

Spec-Driven Development

One Feature, End to End

Hekuran Islamaj

A practitioner handbook for engineering teams:

Feature → Spec (EARS) → DDD → Tasks → Architecture →
BDD → TDD → Implement → Verify → DORA

Spec first. Verify always. Measure honestly.

This handbook is a materials-only compilation. Every claim, number, artifact, command, rule, and aphorism in it derives from the three source documents reproduced verbatim in Appendices H-J. No external sources, industry references, or invented case studies were used. Where a code block or table appears in a chapter, it is reproduced from the source runbooks; the only edits are typographic (soft-wrapping of over-long lines, noted where applied, with semantics unchanged). The running example throughout — PLT-4821, partial refunds for cancelled events — is the worked feature of the source walkthrough.

Contents

Preface	viii
How to Read This Book	x
Who This Book Is For	xii
Conventions and the Cast of Artifacts	xiii
I The Problem of the Vague Ticket	1
1 A Feature Arrives	2
2 Naming the Ambiguities	5
3 Ambiguity at Machine Speed	9
II Specify: The EARS Stage	12
4 Who Writes the Spec, and When	13
5 The Five EARS Patterns	15
6 The Four Rules Enforced While Writing	19
7 The Exit Gate	22
III Place It in the Domain: DDD	24
8 Fifteen Minutes of Event Storming	25

9 Bounded Contexts: One Context, One Folder	27
10 Context Mapping: Frozen Snapshots and Published Events	29
11 Ubiquitous Language and the Tactical Sketch	31
 IV Split It into Tasks	 34
12 Vertical Slices and the Three Laws of a Task	35
13 The Six Tasks of PLT-4821	37
14 Estimation Becomes Counting	40
 V Architecture: Patterns and Principles, Applied	 42
15 The Shape: Hexagonal, with a Functional Core	43
16 Ten Principles, Each with Its Payoff	45
17 Where the Tests Attach	49
 VI Contracts: The BDD Dialect	 51
18 The Shared Dialect	52
19 Rules for Scenarios That Stay Healthy	54
20 One Oracle, Two Dialects	56
21 Red BDD: Proving the Detector Detects	58
 VII Red, Green, and the Anticlimactic Implementation	 59
22 The Red: Derived from the Contract	60
23 Red, Green, Refactor — with Triangulation	62
24 The Agent-Assisted Variant	64

25The Anticlimactic Green	66
VIII Verify: The Exit Gate	68
26The Anatomy of verification.md	69
27The Demo Is the Verification	72
IX Enforcement over Persuasion: The Tooling	74
28The Five Commands	75
29The Resource Gate	77
30Providers, Sessions, and the One Toggle Rule	79
31MCP: When the Agent Reads Jira Itself	81
32The Repository, Mapped	83
X Measure: DORA, the SDD Signals, and the Worked Dashboard	85
33DORA, Exactly as Measured Here	86
34The Two SDD-Native Signals	88
35Beyond DORA: SPACE, DevEx, and GSM	90
36Can Jira Track These? The Split	94
37The Worked Dashboard: Two Sprints, Six Numbers	97
38Reading the Table	99
39Reading It Honestly	101
XI Instrumentation: Two Hours to a Self-Computing Dash-	

board	102
40 The Goal and the Budget	103
41 Part A — Jira as the Workflow Clock	105
42 Part B — GitLab as the Delivery Clock	109
43 Part C — Sanity Checks, Operating Rules, and the Source-of-Truth Table	113
 XII The Twelve-Minute Demo	 115
44 What the Demo Is, and Its One Design Law	116
45 Prerequisites: Ten Minutes, Once	117
46 The Repo, and the Two Modes	119
47 Configuring the Jira Source: Pick One	120
48 The Six Beats	121
49 Fallbacks and Reset — Do Not Skip	124
50 Troubleshooting	125
51 What Was Verified Where	126
 XIII The Pilot: Making It a Team’s Habit	 127
52 From One Feature to One Team	128
53 Move 1 — The Kickoff, and the One Thing to Say Out Loud	129
54 Move 2 — Baseline Before the Pilot	130
55 Move 3 — Two Sprints, Expecting the Dip	132
56 Move 4 — The Retro Reads the Spec	133

57 Move 5 — Scale on Trends; Never Absolutes; Never Individuals	135
58 Closing: The Three-Line Method	137
 XIV Appendices	 138
A Stage → Artifact → Command	139
B The Spec Artifacts of checkout-refunds, Assembled	140
C Configuration Reference (Verbatim)	144
D Repository Map	148
E Troubleshooting	149
F Glossary	151
G Checklists	155
 XV The Source Documents	 157
H Source Document: WALKTHROUGH.md	159
I Source Document: METRICS-SETUP.md	176
J Source Document: DEMO.md	182

Preface

There is a moment, early in the source walkthrough this book is built from, that explains why the whole method exists. A feature has just arrived as a Jira ticket. The description is three sentences. The acceptance criterion is five words: *“refunds work for cancelled events.”* And buried in a comment from the product owner is the sentence that should terrify anyone who has ever pointed a coding agent at a backlog:

“Fees = our service fees, not payment-provider fees. **I think.**”

I think. That is the state of the requirement at the moment implementation is about to begin. For most of the history of software, this was survivable, because the implementer was a human being who would stall on “I think,” feel the discomfort, and go ask. The ambiguity was expensive, but it was slow, and the slowness bought the organization time to notice that it did not actually agree with itself.

That protection is gone. The source materials put it plainly: every one of those gaps *will* be filled by someone’s assumption — a developer’s, a QA’s, or an agent’s, **sampled fresh on every retry**. An agent does not stall on “I think.” It resolves the ambiguity — confidently, plausibly, differently each time — and hands you a green build that proves nothing except its own self-consistency.

Spec-Driven Development is the machinery that fills those gaps **on purpose**. It is not “write more documentation.” It is a pipeline — Feature → Spec (EARS) → DDD → Tasks → Architecture → BDD → TDD → Implement → Verify → DORA — in which every stage produces a checkable artifact, every gate is enforced by tooling rather than by memory, and the whole thing closes with a dashboard that turns the next argument about process into an argument about data.

This handbook works that pipeline in full, on one real feature — **PLT-4821, partial refunds for cancelled events** — with the reasoning, the patterns, and the artifacts. The promise of the source walkthrough is the promise of

this book: follow it once and you can run the method on any feature.

The method has a voice, and this book keeps it, because the aphorisms *are* the method compressed:

- *A hook is enforcement; a prompt is persuasion.*
- *One oracle, two dialects.*
- *If you can't write the failing test, rewrite the criterion.*
- *Coverage dips at every sprint boundary; the dip is the signal working.*
- *Jira measures what people transition.*
- *Instrument once. Argue with data.*
- *Spec first. Verify always. Measure honestly.*

You will meet each of these where it earns its keep.

How to Read This Book

The book follows the pipeline in order, because the pipeline’s order is the argument. Each part corresponds to a stage; each stage is worked on PLT-4821; each chapter ends with the artifact the stage produces.

If you are an engineer who will run this daily, read Parts I-VIII straight through — they are the walkthrough, expanded: the villain ticket, the EARS spec, the domain placement, the task slicing, the architecture, the two contract dialects, the red-green loop, and verification. Then keep Part IX (enforcement and tooling) and the appendices at your elbow.

If you are a tech lead or staff engineer, add Part X (the measurement frame — DORA plus the two SDD-native signals, and the SPACE/DevEx/GSM extensions) and Part XI (the exact Jira and GitLab instrumentation). You will own the boundaries in Part III and referee the rules in Part II.

If you are a delivery lead or engineering manager, Parts X, XI, and XIII are yours: the worked dashboard with real numbers, the two-hour instrumentation setup, and the pilot playbook — baseline first, two sprints, trends per team, never individuals.

If you have to give the demo, Part XII is a self-contained expansion of the 12-minute stage script, with prerequisites, three ways to source the Jira ticket, six beats with timings, fallbacks for every failure mode, and the reset procedure. Rehearse from it; run from Appendix G’s checklist.

The appendices collect everything twice-useful: the stage→artifact→command map, the complete configuration reference (every YAML job, JQL filter, and CI variable, verbatim), the repository map, both troubleshooting tables, a glossary defined strictly from usage in the materials, and operational checklists. Appendices H-J reproduce the three source runbooks in full, so this volume is self-contained: if this book and the repo ever disagree, the appendices *are* the repo’s documents.

Two reading conventions. First, code, YAML, JQL, and shell blocks are reproduced from the sources; where a line was too long for the page, it has been soft-wrapped in a way that preserves semantics (shell continuations, or breaks inside quoted strings that remain valid), and the appendix notes say so. Second, when a sentence appears in italics and sounds like a proverb, it is a direct load-bearing quotation from the materials, not decoration.

Who This Book Is For

- **Backend and full-stack engineers** who will write EARS criteria, bind Gherkin and Jest to one harness, and drive /implement under the resource gate.
- **Tech leads** who will co-sign specs with the PO inside the refinement hour, own the bounded-context map in docs/ddd.md, and hold the line that an L-sized task is a splitting instruction, not an estimate.
- **QA engineers**, whose review is built into the spec stage and whose verification bandwidth is *booked before development capacity* — because with agents typing, verification is the constraint.
- **Product owners**, at least for Parts I, II, and VI: the spec is co-authored, the Gherkin is written to be read by you, and the “...I think” comment is yours to retire.
- **Platform and DevOps engineers** who will stand up the Jira Data Center workflow, the Automation rule, the three GitLab CI jobs, and the Monday-morning dashboard — the whole instrument, in about two hours, once, per project.
- **Anyone who has to convince a room.** The demo part and the worked dashboard exist so that the case for the method is made by a running pipeline and six moving numbers, not by slides alone — though, as the fallback plan says, slides 10–11 *are* the demo if everything else burns down.

Assumed background: comfort with Git, CI/CD, Jira, TypeScript/Node, and the basic mechanics of a coding agent. No prior EARS, DDD, BDD, or DORA experience is required; every term is defined at the point of use and again in the glossary.

Conventions and the Cast of Artifacts

The same small cast of files appears throughout the book. Meet them once here:

Artifact	Role in the story
jira/PLT-4821.json	The feature as it arrives — the vague ticket. In the demo runbook it is literally called <i>the villain</i> .
specs/checkout-refunds/spec.md	The EARS specification: criteria EARS-1...5, non-functionals as numbers, an Out-list, and an Open-questions section that must reach zero.
docs/ddd.md	The bounded-context map: Ordering, Payments, Refunds , Notification.
specs/checkout-refunds/plan.md	The tactical design: aggregate, value objects, ports, risks.
specs/checkout-refunds/tasks.md	Six vertical slices, T1-T6, each 1 : 1 : 1 with declared Touches: globs.
contracts/refunds.feature + features/steps/	The BDD dialect: Gherkin the PO can read, bound to the shared harness.
refunds.service.spec.ts	The TDD dialect: the failing Jest test derived from the contract.
src/refunds/	The Refunds bounded context: pure domain core, application service, adapters.
refund-calculator.ts	The anticlimactic green: the whole feature's math in a handful of lines.
specs/checkout-refunds/verification.md	The exit gate: criteria coverage, per-task evidence, verdict, first-pass boolean.
.claude/commands/	The pipeline as slash-commands: /specify-from-jira, /plan, /tasks, /implement, /verify.
.claude/hooks/resource_gate.sh	Enforcement: a PreToolUse hook; exit 2 = blocked.

Contents

Artifact	Role in the story
<code>.claude/state/ACTIVE_TASK</code>	The leash: the one task an agent may touch right now.
<code>scripts/contract_coverage.sh</code> , <code>scripts/dora_dashboard.py</code>	The measurement scripts CI runs.

Typography: *italic proverbs* are quotations from the materials; monospace names files, commands, statuses, and identifiers; **EARS-n** identifiers are always bold on first use in a chapter. Money is always integer cents. Statuses are Jira workflow statuses. And “the pipeline,” unqualified, always means the full chain on the title page.

Part I

The Problem of the Vague Ticket

Chapter 1

A Feature Arrives

It arrives, as features do, as a Jira ticket. The walkthrough opens with it verbatim, and so will we, because everything that follows — nine stages, six metrics, one demo — is a response to these few lines (`jira/PLT-4821.json`):

Summary: Partial refund for cancelled events **Description:** When an event gets cancelled we need to refund the buyers. Refund the ticket price minus fees. Should be automatic, buyers shouldn't have to call support. Marketing wants this before the autumn on-sales. **AC:** refunds work for cancelled events. **Comment (PO):** "Fees = our service fees, not payment-provider fees. I think." **Comment (Support):** "Make sure people can't claim a refund twice."

Read it the way it would be read in a refinement meeting: quickly, nodding. It seems fine. It has a business motivation (autumn on-sales), a user-experience stance (no calling support), a stated behavior (refund price minus fees), even a security-ish concern from Support (no double claims). Compared to many tickets, this is a *good* ticket.

That is precisely what makes it dangerous. The demo runbook, which turns this same ticket into a stage prop, calls the file **the villain** — and the villainy is not emptiness but plausibility. An empty ticket gets bounced back. A plausible one gets built.

1.1 The anatomy of plausible vagueness

Take the ticket apart, phrase by phrase, and count the decisions it *sounds like* it made but didn't.

“Refund the ticket price minus fees.” Which fees? The company’s service fees? The payment provider’s processing fees? Both? The PO’s comment tries to answer — and ends with *“I think.”* A requirement whose author hedges in the same breath is not a requirement; it is a coin waiting to be flipped by whoever implements first. And this coin has money on both faces: refund the gross and the company eats its service fee on every cancelled event; deduct the wrong fee and buyers are short-changed at scale.

“Should be automatic.” Automatic *triggered by what*, exactly, and *within what time*? Triggered by the cancellation event itself? By a nightly batch? By an operations person pressing a button that is merely not a support call? “Automatic” without a trigger and a deadline is a mood, not a behavior. Note also what the deadline decision drags with it: synchronous versus asynchronous processing, retry semantics, and the definition of “done” for the buyer.

“Refunds work for cancelled events.” This is the acceptance criterion — the sentence that is supposed to tell everyone, including a test suite, when the feature is finished. It is untestable as written. What observable behavior equals “works”? A ledger entry? An API response? Money visible on a statement? The walkthrough’s litmus test (Chapter 6) will execute this sentence and find it returns nothing.

“Make sure people can’t claim a refund twice.” Good instinct from Support — and still underspecified in the one dimension that matters for implementation: *what is the response to a duplicate?* A silent no-op that pretends success? An error? Which status code? The difference is not pedantry; it is the difference between a client that can distinguish “already refunded” from “refunded just now,” and one that cannot.

And then the questions the ticket does not even know it has. What currency precision does the refund math use — floats, decimals, integer cents? What rounding rule applies when fee math produces fractions? Is “partial refund” — the word sitting in the ticket’s own title — in scope now, per line item, or later? Nobody asked, so nobody answered; and yet the code will answer, because code cannot ship an ambiguity. It can only ship a choice.

1.2 Why this ticket, of all tickets

The walkthrough chose PLT-4821 deliberately, and it repays study for three reasons.

First, it is *representative*. Every phrase pattern above — the hedged comment, the untestable AC, the adjective doing a requirement’s job (“automatic”), the unstated numeric policy — recurs across real backlogs. Defeat the patterns here and you can recognize them anywhere.

Second, it is *consequential*. Refunds are money movement. The gap between “minus fees” and “minus *service* fees” is a recurring financial delta, and the gap between “can’t claim twice” and “409 on PENDING or ISSUED” is the difference between idempotency by design and double refunds under retry.

Third, it is *small enough to finish*. The whole feature, properly specified, becomes six tasks totalling at most four developer-days (Chapter 14). The pipeline’s claim is not that ceremony is worth it for a moonshot; it is that the ceremony is *cheap enough for a four-day feature* — 45 to 60 minutes of spec work, inside the refinement hour that already exists — and that the payoff shows up in the very first sprint’s metrics.

The rest of Part I does the one thing the walkthrough insists on before any tooling appears: it names the ambiguities, out loud, in a table, and prices them.

Chapter 2

Naming the Ambiguities

The walkthrough’s instruction is explicit and comes with a claim about leverage:

Before touching anything, name the ambiguities. This ten-minute exercise is the highest-leverage act in the whole pipeline.

Ten minutes. Not a workshop, not a document — a table with two columns: *the vague phrase* and *the question nobody answered*. Here is the table for PLT-4821, exactly as the walkthrough draws it:

Vague phrase	The question nobody answered
“minus fees”	Which fees? Service fees? PSP fees? Both? (a comment says “...I think”)
“automatic”	Triggered by what, exactly? Within what time?
“refunds work”	Untestable. What observable behaviour = “works”?
“can’t claim twice”	What’s the response to a duplicate? Silent no-op? Error? Which code?
(unstated)	What currency precision? Rounding rule on fee math?
(unstated)	Partial refunds per line item — now or later?

Six rows. Spend a moment with each, because these six rows are the raw material for the entire specification stage, and each one models a *class* of ambiguity you will meet again.

2.1 Row 1 — “minus fees”: the hedged decision

The question is *which fees*, and the ticket’s best answer is a comment ending in “...I think.” This is the **hedged decision** class: a stakeholder has an opinion but not a commitment. The failure mode it invites is silent adoption — the implementer treats the hedge as a decision because it is the only text available. The pipeline’s answer, previewed here and executed in Chapter 6, is to convert the hedge into an *open question* addressed to the person who can commit (the PO), and to make planning mechanically impossible until it is answered. When it is answered — *service fees, from the order snapshot* — the answer is written into the spec and the “I think” is retired forever.

2.2 Row 2 — “automatic”: the adjective doing a requirement’s job

“Automatic” gestures at a trigger and a deadline without supplying either. This is the **unbound adjective** class (“fast,” “secure,” “seamless,” “automatic”). The cure is always the same: force the adjective through the interrogation *triggered by what? within what time?* until it becomes a trigger-response-constraint sentence. For PLT-4821 the answer will be: WHEN an event is cancelled — that event, nothing else — the system shall refund within **5 business days**. Suddenly there is something to test, something to monitor, and something for Marketing’s autumn deadline to actually mean.

2.3 Row 3 — “refunds work”: the untestable AC

This is the **untestable acceptance criterion** class, and it is the most important row in the table, because the walkthrough builds its central quality rule on it: *if you can’t write the failing test, rewrite the criterion*. “Refunds work” can never produce a failing test, because it names no observable. It will be replaced by criteria for which the failing tests are shown, in code, later in this book — and the replacement is the moment the feature first becomes *finishable*.

2.4 Row 4 — “can’t claim twice”: the behavior without a response

Support’s comment names the invariant but not the interface. The **unspecified failure response** class covers every “must not,” “should prevent,” and “can’t” that omits what the caller actually experiences. The eventual answer is precise to the status code: WHILE a refund is PENDING or ISSUED, further requests for that order are rejected with 409. That number — 409 — will echo through the OpenAPI contract, a Gherkin scenario, a Jest test, and the idempotency design of the aggregate itself. One row of interrogation, four artifacts aligned.

2.5 Rows 5 and 6 — the unstated: precision, rounding, and scope

The last two rows have no vague phrase at all, which is the point: the **unstated policy** class hides in what the ticket never mentions. Currency precision and rounding sound like implementation trivia; they are not. Fee math on floats produces drift; fractional fees need a rounding rule; and an unwritten rounding rule gets re-litigated by every engineer who touches the code — the walk-through will pin it as banker’s rounding, half-to-even, and log the decision as **ADR-017** precisely *because a decision that isn’t written gets re-litigated*. And “partial refunds” — in the ticket’s title! — is a scope question: now, per line item, or later? Left unstated, it becomes gold-plating bait. Named, it becomes an explicit optional criterion flagged out of this slice.

2.6 Why the table works

Notice what the exercise did *not* require: no tooling, no template, no agent. It required only the discipline to read the ticket as an adversary would — hunting for every place where two reasonable people could implement different systems and both claim fidelity to the text. The table’s two-column shape is itself a forcing function: you may not write a vague phrase in column one without writing the unanswered question in column two, and an unanswered question, once written, demands an owner.

The walkthrough closes the exercise with the sentence that motivates everything after it:

Every one of these gaps *will* be filled by someone's assumption — a developer's, a QA's, or an agent's, sampled fresh on every retry. The rest of this document is the machinery that fills them **on purpose**.

Chapter 3 takes that sentence seriously, and literally.

Chapter 3

Ambiguity at Machine Speed

The claim to internalize from the previous chapter has three parts, and each part is doing work.

“Every one of these gaps *will* be filled.” Not “might.” Code cannot contain an ambiguity; a running system always embodies exactly one interpretation. The only question is whether the interpretation was chosen by the people accountable for it or by whoever happened to be holding the keyboard when the compiler demanded an answer.

“...by someone’s assumption — a developer’s, a QA’s, or an agent’s.” The list is ordered by history. Developers have always filled specification gaps; QA has always filled them a second time, sometimes differently, which is where “works on my machine” arguments are born. The new entrant is the agent — and the agent changes the economics, not just the actor.

“...sampled fresh on every retry.” This is the phrase that distinguishes the agent era. A human developer’s assumption, once made, is at least *stable*: wrong, perhaps, but consistently wrong, discoverable, and fixable once. An agent’s assumption is a sample from a distribution. Ask again — retry a failed run, regenerate a file, re-run the task tomorrow — and the gap may be filled *differently*. Yesterday “minus fees” meant service fees; today it means both; the diff looks plausible both times. Ambiguity plus agents does not produce one wrong system. It produces a *family* of subtly different systems, one per invocation, each internally consistent, none anchored to a decision anyone made.

3.1 Why speed makes it worse, not just faster

Under human implementation, ambiguity generated *friction* — questions in standup, a stalled PR, a raised eyebrow — and friction, for all its cost, was also *detection*. The organization found out it disagreed with itself before the disagreement compiled.

Under agent implementation, ambiguity generates *confident output*. There is no stall. The demo script for Beat 1 gives the sentence to say out loud in front of the ticket, and it belongs here too:

“Every word of ambiguity here will be typed into code at machine speed. Let’s stop it now.”

Machine speed removes the last natural checkpoint. The interpretation gap opens in seconds, spans many files, and arrives wearing a green test suite — green because the agent’s tests verify the agent’s interpretation. Green stops being evidence of correctness and becomes evidence of self-consistency. That inversion is the deep problem, and it explains why the pipeline’s later stages are so insistent that tests be derived *from the contract, never from the implementation* (Chapter 21): a detector built by the thing it is meant to detect detects nothing.

3.2 Two non-solutions, and why the pipeline rejects them

It is worth naming the two popular responses the materials decline to rely on, because their inadequacy shapes the design.

“Prompt more carefully.” A better prompt is better persuasion — and the framework’s sharpest line, met properly in Part IX, draws the boundary: *a hook is enforcement; a prompt is persuasion*. You may ask an agent to stay inside `src/refunds/`; the resource gate *makes* out-of-scope edits fail with exit 2. The pipeline uses prompts for orchestration and hooks for guarantees, and never confuses the two.

“Write more tests afterward.” A test written after the code *passes by construction and proves nothing* — the walkthrough’s own words. The order of operations is the whole game: the failing test comes from the contract, before

implementation exists, precisely so its redness proves the detector can detect absence.

3.3 Filling the gaps on purpose

So the thesis, stated positively: since the gaps will be filled regardless, build machinery that fills them **on purpose** — with decisions made by the accountable people, recorded in checkable form, enforced by tools, and measured afterward. Concretely, over the coming parts:

1. **Decide in the spec** (Part II): each vague phrase is interrogated into one of five EARS patterns with a stable ID; open questions gate planning; numeric policies become ADRs.
2. **Place the decision in the domain** (Part III): bounded contexts ensure the decision lives in exactly one folder, expressed in the ubiquitous language, with the fee source frozen on the order snapshot so a whole class of “fees changed mid-refund” bugs cannot exist.
3. **Slice and scope it** (Part IV): one criterion, one contract, one ticket; the Out-list makes “later” explicit and physical.
4. **Contract it twice** (Parts VI–VII): the same oracle answered in two dialects — Gherkin the PO reads, Jest the engineer trusts — both red before code.
5. **Enforce the blast radius** (Part IX): `ACTIVE_TASK` plus the `PreToolUse` gate turn task scope from a request into a property of the environment.
6. **Verify and measure** (Parts VIII, X–XI): a conformance verdict per criterion, and six numbers on a dashboard whose direction tells you whether the machinery is working.

One more framing before the pipeline begins. The ten-minute ambiguity table cost nothing and required no belief in any methodology. If your team adopts nothing else from this book, adopt the table: read the next ticket as an adversary, write the six rows, and watch how many of them end in “...I think.” The rest of the method is what you will want the moment you see the count.

Part II

Specify: The EARS Stage

Chapter 4

Who Writes the Spec, and When

Before the patterns, the logistics — because the most common objection to specification is not that it is useless but that it is *unaffordable*, and the walk-through answers the objection in its first two lines about the stage:

Who & when: the engineer drafts, the PO co-signs, QA reviews — inside the existing refinement hour. Timebox: 45–60 min for a feature this size. **Where:** specs/checkout-refunds/spec.md, from templates/spec.template.md (or live: /specify-from-jira PLT-4821).

Read the affordability argument out of those lines. No new meeting is created: the work happens *inside the existing refinement hour*, the slot the team already spends nodding at tickets like PLT-4821. No new headcount is created: the three roles that must agree anyway — the engineer who will build it, the PO who owns the intent, the QA who will verify it — are the three authors. And the cost is bounded up front: **45 to 60 minutes for a feature this size**, a feature that Part IV will show totals at most four developer-days. Ten to fifteen percent of a feature’s effort, spent where a decision costs a sentence instead of a rewrite.

The division of labor matters as much as the timebox. **The engineer drafts** — because the draft requires knowing what is implementable and what a test can observe. **The PO co-signs** — because the open questions (which fees? in scope now?) are intent questions only the PO can close, and a co-signature converts “I think” into a commitment with a name on it. **QA reviews** — because QA will run the verification stage later, and the fastest way to make verification cheap is to let the verifier veto untestable criteria before they are written down. Part X will later count this co-authorship as a collaboration signal in its

own right: *specs co-authored PO + engineer: 6 / 6* in the pilot sprint.

Two entry points exist and produce the same artifact. The manual path starts from `templates/spec.template.md` and fills it in during refinement. The live path is the pipeline's first command:

```
/specify-from-jira PLT-4821
```

The command reads the ticket — description *and* comments, because as PLT-4821 proves, half the requirement lives in the comments — and drafts EARS criteria, carrying anything it cannot resolve into the Open-questions section rather than guessing. The demo (Part XII, Beat 2) runs this live and calls the resulting conversation “the demo”: the room watches “...I think” become an open question instead of an assumption. Either way, the destination is the same file: `specs/checkout-refunds/spec.md`, versioned in git next to the code it governs.

The stage's contract with the rest of the pipeline is a gate, stated now and enforced in Chapter 7: the spec is done when it is **approved in review** and its **Open questions are empty** — and *only then does /plan unlock*.

Chapter 5

The Five EARS Patterns

EARS — Easy Approach to Requirements Syntax, applied here via the ears-authoring skill — gives every requirement one of five rigid sentence shapes. The rigidity is the feature: a criterion that fits a pattern has necessarily named its trigger, its response, and its constraint, and a criterion with all three is a criterion a test can fail.

The walkthrough demonstrates the stage as a *transformation table*: each vague phrase from Part I is taken through an interrogation and landed in a pattern. Here is the table, verbatim, followed by a study of each row:

Before (ticket)	Interrogation	After (spec)
“refund the ticket price minus fees”	which fees? when?	EARS-2 (Event): WHEN an event is cancelled, the system shall refund the ticket price minus <i>service fees</i> within 5 business days.
implied money handling	floats? rounding?	EARS-1 (Ubiquitous): The refunds API shall return amounts in minor units (cents).
“can’t claim twice”	duplicate response?	EARS-3 (State): WHILE a refund is PENDING or ISSUED, further refund requests for that order shall be rejected with 409.
(security hygiene)	expired token?	EARS-4 (Unwanted): IF the access token is expired, THEN the API shall return 401 with WWW-Authenticate.

Before (ticket)	Interrogation	After (spec)
“partial refund” in the title	now?	EARS-5 (Optional): WHERE partial refunds are enabled, the system shall support line-item refunds. → flagged out of this slice.

Five criteria; five patterns; every ambiguity from the Part I table accounted for. Take them one at a time.

5.1 EARS-2 — the Event pattern: WHEN

WHEN an event is cancelled, the system shall refund the ticket price minus *service fees* within 5 business days.

The Event pattern is trigger-response-constraint, and this single sentence retires three of the six ambiguities at once. *Which fees?* Service fees — the hedged PO comment, now decided (and Part III will pin *where the number comes from*: frozen on the order snapshot). *Triggered by what?* The cancellation of the event — not a user request, not a batch job. *Within what time?* Five business days — a number Marketing’s deadline can be reconciled against, monitoring can alarm on, and a test can assert.

This is also the criterion the whole book follows to the finish line: it becomes a Gherkin scenario in Part VI, the failing Jest test in Part VII, task **T3** — the demo’s live task — in Part IV, and the row that flips from Red – T3 open to **Green** in `verification.md` in Part VIII.

5.2 EARS-1 — the Ubiquitous pattern: always true

The refunds API shall return amounts in minor units (cents).

The Ubiquitous pattern has no trigger because it holds always. Notice what generated it: *nothing in the ticket*. The interrogation of the unstated — floats? rounding? — produced a requirement the ticket did not know it needed. Minor units, integer cents, everywhere: this one sentence forecloses float drift by fiat, and Part V will make it unrepresentable in the type system via the `Money` value object, which *holds integer cents and throws on anything else*. Ubiquitous

criteria are where cross-cutting policy belongs — precision, encoding, units — the decisions that otherwise get made locally, differently, per file.

5.3 EARS-3 — the State pattern: WHILE

WHILE a refund is PENDING or ISSUED, further refund requests for that order shall be rejected with 409.

The State pattern conditions behavior on a mode the system is in. Support’s “can’t claim twice” has become fully mechanical: the protected states are named (PENDING, ISSUED — nouns that Part III will adopt into the ubiquitous language), and the duplicate response is specified to the status code. The 409 is load-bearing: it is what the OpenAPI contract for task T2 promises, what a Gherkin scenario asserts, and what makes idempotency a *domain rule, not middleware* — the aggregate keyed by `orderId`, the service’s first act a store lookup. Retries, double-clicks, and replayed events all collapse into the same non-event.

5.4 EARS-4 — the Unwanted pattern: IF/THEN

IF the access token is expired, THEN the API shall return 401 with WWW-Authenticate.

The Unwanted-behavior pattern specifies the system’s response to things going wrong. Its provenance row in the table reads simply “(security hygiene)” — no one asked for it; the pattern *itself* prompted the question “and what about an expired token?” This is the quiet second benefit of a pattern catalogue: it functions as a completeness checklist. Teams that write only happy-path Events discover, via the Unwanted pattern, the failure envelope they forgot. EARS-4 will get its own slice — task T6 — because *the unwanted-behaviour pattern gets its own slice*, keeping guard logic from smearing across the happy path.

5.5 EARS-5 — the Optional pattern: WHERE

WHERE partial refunds are enabled, the system shall support line-item refunds. → flagged out of this slice.

The Optional pattern scopes behavior to a configuration or feature flag — and here it performs the stage’s most underrated service: **scope control in the spec itself**. “Partial refund” sat in the ticket’s *title*, the single most gold-plating-prone position a phrase can occupy. Rather than silently building it or silently dropping it, the spec names it, patterns it, and flags it out of the current slice. Later stages honor the flag: EARS-5 stays – in the verification table, *documented* as out of slice; contract coverage counts it as consciously excluded, not missing; and the architecture (Part V) deliberately does **not** pre-build the abstraction for it — *the flag stays in the spec until it’s real*.

5.6 Why five shapes beat free prose

Three properties, all exploited downstream. **Checkability**: trigger-response-constraint is exactly the information a failing test needs; the litmus test of the next chapter is only possible because the shapes guarantee the ingredients. **Countability**: criteria carry stable IDs — EARS-1 through EARS-5 here — and stable IDs are what let a CI script compute contract coverage (EARS-n IDs in specs versus IDs referenced by tests) and what let `verification.md` have one row per requirement. **Translatability**: the shapes map nearly mechanically onto Given-When-Then — Part VI’s table shows WHEN-criteria becoming *When* steps almost word for word, and the walkthrough counts that mechanical feel as *a feature*, because a translation with no creativity in it is a translation that cannot smuggle in new meaning.

Chapter 6

The Four Rules Enforced While Writing

Patterns give criteria their shape; four rules give them their quality. The walk-through lists them as *rules enforced while writing* — enforced, not suggested — and each earns its place on PLT-4821.

6.1 Rule 1 — the litmus test

Every criterion passes the **litmus test** — *if you can't write the failing test, rewrite the criterion.*

This is the quality bar for the entire stage, and it is self-verifying: the walk-through notes that EARS-2 passes because its failing test is literally exhibited later (Part VII shows it, red, with NotImplemented – task T3), while “*refunds work*” *never could* — no observable, no test, no criterion. The litmus test is best applied live, during the refinement hour: for each drafted criterion, sketch the test's first line aloud. If the sketch stalls — what would we assert? against what? — the criterion is rewritten on the spot, while rewriting costs a sentence.

6.2 Rule 2 — non-functionals as numbers

Non-functionals as numbers: p95 issuance ≤ 300 ms (excl. PSP);
zero double refunds under retry.

“Fast” and “reliable” are unbound adjectives (Part I, row 2’s disease) and are banned in this stage. PLT-4821’s spec carries two numeric non-functionals: **p95 refund issuance at or under 300 milliseconds, excluding the payment-service provider’s own latency** — a percentile, a bound, and an explicit exclusion, so the number is both measurable and fair — and **zero double refunds under retry**, an invariant stated as a count. The second one quietly restates EARS-3 as a system property, which is exactly the redundancy you want: the same truth guarded once as behavior and once as an invariant.

6.3 Rule 3 — scope has an Out-list

Scope has an Out-list: notification wiring ($\rightarrow$ T4), refund UI ($\rightarrow$ T5), line-item refunds (behind flag, later). Gold-plating dies here, in writing.

The Out-list is scope made explicit *in the spec*, with each exclusion routed somewhere: notification wiring is out of the core slice but lands as task T4; the refund UI likewise as T5; line-item refunds sit behind the EARS-5 flag for later. The phrase to keep is the walkthrough’s own: *gold-plating dies here, in writing*. And the Out-list is not merely prose — Part V makes it *physical* (domain events at the seam mean T4 ships without touching T3’s code), and Part VII makes it *law* during implementation: T4’s event templates and T5’s UI stay untouched *even when “it would only take a minute.”*

6.4 Rule 4 — open questions must reach zero

Open questions must reach zero before planning. Ours had two: *which fees* (PO confirms: service fees, from the order snapshot) and *rounding on fractional fee math* (decided: banker’s, half-to-even — logged as **ADR-017**, because a decision that isn’t written gets re-litigated).

The spec template carries an Open-questions section, and the section is a *counter with a gate on it*. PLT-4821’s counter reached two. Question one — *which fees* — went to the PO, who confirmed service fees, sourced from the order snapshot; the hedge from the ticket comment is now a signed decision in a versioned file. Question two — *rounding on fractional fee math* —

was decided as **banker's rounding, half-to-even**, and, crucially, *logged as ADR-017*.

Dwell on the ADR move, because it is a pattern you will reuse constantly. Some spec-stage decisions are pure intent (which fees) and live happily as spec text. Others are *policies with a rationale* that future engineers will be tempted to “fix” by intuition — rounding is the canonical example; the walkthrough later imagines the engineer who “fixes” $2.5 \rightarrow 3$ and *finds the ADR, not a debate*. Such decisions get an Architecture Decision Record: numbered, citable, and referenced from the code (`Money.roundBankers`, per ADR-017). The justification is the walkthrough's aphorism, worth its italics: *a decision that isn't written gets re-litigated*.

Chapter 7

The Exit Gate

Every stage in this pipeline ends the same way: not with a feeling of doneness but with a **gate** — a mechanical condition that either holds or doesn't. The specification stage's gate is stated in one line:

Exit gate: spec.md approved in review, Open questions **empty**. Only now does /plan unlock.

Two conditions, both checkable. **Approved in review** — the engineer-PO-QA trio has signed the artifact; the co-signature is what makes “service fees” a commitment rather than a comment. **Open questions empty** — the counter is at zero; every hedge has been converted into either a decision in the spec or an ADR.

And then the sentence that gives the gate teeth: *only now does /plan unlock*. This is not etiquette. The /plan command **refuses to run while the Open-questions section is non-empty** — the demo makes a beat out of the refusal, pointing at it on stage: answer the fees question live, watch the list empty, watch planning unlock. The refusal pattern established here recurs at every stage boundary in the pipeline, and it embodies the design stance this book will keep restating: stages *assert their preconditions and decline rather than degrade*. A pipeline that proceeds politely on top of an unresolved ambiguity manufactures exactly the machine-speed confusion Part I described; a pipeline that refuses generates its friction at the one moment friction is cheap — before any code exists.

What passes through the gate is a small, dense artifact: five identified criteria in five patterns, two numeric non-functionals, an Out-list with routing, zero open questions, and one ADR. Everything downstream — the domain place-

ment, the six tasks, the two contract dialects, the four-line implementation, the verification verdict, and two of the six dashboard metrics — will trace back to the IDs minted here.

Part III

Place It in the Domain: DDD

Chapter 8

Fifteen Minutes of Event Storming

With the spec approved, the next question is not *how* to build the feature but *where it lives and what language it speaks*. The walkthrough is explicit that this ordering is the point of strategic design:

Strategic design answers *where does this live and what language does it speak* — before any class diagram.

Before any class diagram. The instinct after a spec is approved is to open an editor and start sketching services and tables; the pipeline interposes a fifteen-minute exercise first, because a class diagram drawn in the wrong context is a liability with good formatting.

The exercise is **lightweight event storming on the flow**: walk the feature’s story as a sequence of *domain events* — things that happen, named in past tense, in the business’s own words. Fifteen minutes on PLT-4821 surfaces this chain:

EventCancelled → RefundRequested → RefundCalculated → RefundIssued → BuyerNotified

Five events, and each one is quietly doing architectural work.

EventCancelled is the trigger — the WHEN of EARS-2, now a first-class noun. Naming it as an event settles the “automatic” ambiguity structurally: the refund flow is *event-driven from the cancellation*, not polled, not manual.

RefundRequested → **RefundCalculated** → **RefundIssued** decompose “refund the buyer” into three distinct happenings — and the seams between them are

where responsibilities will split. Requesting is not calculating (the request must survive the duplicate check of EARS-3 first); calculating is not issuing (the math is pure; the issuance moves money and changes state). The three-way split is what will let Part V give the calculator to a pure domain service and the orchestration to an application service, with a straight face.

BuyerNotified is deliberately *last and detachable*. It is the event that the Out-list already routed to task T4 — notification wiring — and its position at the end of the chain previews the context mapping: whoever owns refunds will *publish* that something was issued and know nothing about how buyers get told.

The storming's output is small — one line of arrows — but it is the skeleton on which the bounded contexts, the ports, and even the Gherkin steps will hang. When Part VI writes “*When the event is cancelled*” as a scenario step, it is quoting this chain.

Chapter 9

Bounded Contexts: One Context, One Folder

The event chain crosses territory, and the next move is to draw the borders. The walkthrough names four **bounded contexts**, with the full map living in `docs/ddd.md`:

Bounded contexts (full map in `docs/ddd.md`): Ordering owns orders and their frozen totals; Payments owns money movement; **Refunds owns refund policy** — the math, the state machine, the idempotency; Notification owns outbound messages.

Read the ownership grammar carefully — each context is defined by what it *owns*, not what it does:

- **Ordering** owns orders *and their frozen totals*. The adjective “frozen” is not decoration; it is the single most consequential word in the map, and Chapter 10 is built on it.
- **Payments** owns *money movement* — the actual transfer of funds, the PSP integration, the part of the world where latency is excluded from our p95.
- **Refunds** owns *refund policy* — and the walkthrough enumerates exactly what that means: **the math, the state machine, the idempotency**. The calculation (EARS-2, EARS-1), the PENDING/ISSUED lifecycle (EARS-3), and the duplicate-rejection rule are all one context’s business.
- **Notification** owns *outbound messages* — templates, channels, delivery — none of which Refunds will ever know about.

From the ownership map follows the placement decision, stated with the walk-

through's characteristic finality:

The calculator therefore lives in the Refunds context — `src/refunds/` — and nowhere else. One context, one folder: when a refund contract goes red, there is exactly one directory to open.

One context, one folder. The map is not a wall poster; it is a filesystem convention with a debugging guarantee attached. The guarantee — *when a refund contract goes red, there is exactly one directory to open* — sounds modest until Part X prices it: the pilot's MTTR fell from roughly six hours to **forty minutes**, and the mechanism cited is precisely this — *the red contract test named the folder before the call ended*. Localize → reproduce → bisect, and the map did the localizing for free.

The demo compresses this whole chapter into its Beat 3, one sentence long by design, and the sentence is worth memorizing as the context map's elevator pitch:

"Refunds is its own bounded context — the calculator lives in `src/refunds/`, downstream of Ordering's frozen order snapshot — so when a refund test goes red, there is exactly one folder in the universe to look in."

Chapter 10

Context Mapping: Frozen Snapshots and Published Events

Borders drawn, the next question is how the contexts *relate* — who conforms to whom, and through what contract. The walkthrough maps Refunds’ two relationships, and each one kills a class of bugs by construction.

10.1 Downstream-conformist to Ordering: the frozen snapshot

Refunds is *downstream-conformist* to Ordering: totals and service fees come **frozen on the order snapshot**, never recomputed from live fee config (this single decision kills a whole class of “fees changed mid-refund” bugs).

Unpack the term and then the payoff. *Downstream* means Refunds consumes what Ordering produces; *conformist* means Refunds accepts Ordering’s model as-is rather than translating it — the order snapshot’s shape *is* Refunds’ input contract. And *frozen* is the decision: the totals and the service fees used in refund math are the ones **captured on the order at purchase time**, never re-read from live fee configuration at refund time.

Consider the alternative to see what the freeze buys. Suppose fees were recomputed live: an order is placed under a 10% service fee; the company changes the fee to 12%; the event is cancelled; the refund math now runs against a fee the buyer never paid. Every fee-schedule change becomes a latent refund bug with a time-delay fuse. The walkthrough’s verdict — *this*

single decision kills a whole class of “fees changed mid-refund” bugs — is exact: the class does not become rarer; it becomes **inexpressible**, because the live fee config is simply not an input to the calculation.

The freeze also resolves, at the data-source level, the PO’s answered question from Part II. “Service fees, *from the order snapshot*” — the phrase the PO confirmed — is this mapping. Spec decision and context mapping are the same fact viewed from two stages.

And the freeze pays a purity dividend Part V will spend: because the snapshot is immutable, the refund becomes a *pure function of the snapshot* — `total.cents - fees.cents`, then a rounding rule — which is what makes a four-line, deterministic, laptop-reproducible implementation possible at all.

10.2 Upstream to Notification: **publish and know nothing**

Refunds is *upstream* to Notification: it publishes `refund.issued v1` and knows nothing about templates.

The other direction, the other discipline. Refunds *publishes* a versioned domain event — `refund.issued`, version 1 — through a seam, and its knowledge stops there: no templates, no channels, no delivery status. Notification subscribes on its own side of the border.

Two consequences are cashed in later. First, the Out-list becomes *physical*: task T4 (the notification wiring) *ships without touching T3’s code*, because the only coupling is an event schema — the walkthrough calls this *the Out-list made physical*, and it is the difference between scope discipline as a promise and scope discipline as a property of the dependency graph. Second, the **v1** matters: the event is a published contract with a version, which is what allows Notification to evolve independently and what gives task T4 its own contract artifact (*event schema v1*) in the task table.

Chapter 11

Ubiquitous Language and the Tactical Sketch

11.1 The same nouns, from the PO's mouth to the code

Strategic design's last deliverable is a vocabulary — the **ubiquitous language** — and the walkthrough states both the list and the law:

Ubiquitous language — the same nouns from the PO's mouth to the code: *service fees* (ours, on the snapshot — the "...I think" comment, now decided), *ISSUED/PENDING* (refund states; duplicates → 409), *minor units* (integer cents everywhere). These words appear verbatim in spec.md, in the Gherkin, in the type names, and in the test descriptions. That is the point.

Three terms, each dragging its decision behind it. **Service fees**: ours, sourced from the snapshot — every time anyone says the phrase, the retired "...I think" is retired again. **ISSUED / PENDING**: the refund states, whose very mention implies the 409 rule for duplicates. **Minor units**: integer cents everywhere — EARS-1 as vocabulary.

The law is the *verbatim* requirement: the same words in spec.md, in the Gherkin, in the type names, in the test descriptions. *That is the point*. When the spec says *service fees*, the scenario says *service fees*, the type is fees, and the test description says "*refunds price minus fees*", then a conversation with the PO, a failing test name, and a stack trace are all speaking one language

— and translation errors, the quietest class of defect, have nowhere to occur. The discipline also gives review a cheap heuristic: a PR that introduces a synonym (“charges,” “commission,” “surcharge”) for an existing ubiquitous noun is either a bug or a missing spec amendment.

11.2 The tactical sketch: aggregate, value object, pure service

Only now — context placed, language fixed — does tactical design get its paragraph, and it lands in `plan.md` (produced by `/plan checkout-refunds`):

Tactical sketch (lands in `plan.md`): the *Refund* is the aggregate, keyed by `orderId` (which is what makes idempotency trivial); *Money* is a value object; the calculator is a pure domain service. Risks table: fee model varies by market → mitigated by the snapshot decision; double refunds under retry → mitigated by EARS-3 + the keyed aggregate.

Three tactical choices, each aimed at a criterion:

The Refund aggregate, keyed by `orderId`. The key choice is the idempotency design — *which is what makes idempotency trivial*. One order, one refund aggregate, ever; a second request for the same `orderId` finds the existing aggregate in `PENDING` or `ISSUED` and is rejected with 409. EARS-3 stops being a rule someone must remember to check and becomes a collision on a key.

Money as a value object. EARS-1’s *minor units* made structural: a type that holds integer cents and (Part V) *throws on anything else*, with the rounding rule (ADR-017) as its method. Illegal amounts don’t fail review; they fail construction.

The calculator as a pure domain service. The math of EARS-2, with no I/O — the choice that Part V’s functional-core architecture will generalize and that makes T3’s test *four lines and deterministic*.

And, soberly, the **risks table** — the plan’s habit of writing down what could go wrong next to what already handles it. Two rows for PLT-4821: *fee model varies by market* → mitigated by the snapshot decision (frozen fees are per-

order facts, so market variance is Ordering's problem at purchase time, not Refunds' at refund time); *double refunds under retry* → mitigated by EARS-3 plus the keyed aggregate — the spec rule and the tactical design named together, because a risk mitigated twice, at two different layers, is the pipeline's idea of handled.

The stage's artifacts are now on disk: the context map in `docs/ddd.md`, the tactical sketch and risks in `plan.md`. The feature has a home (`src/refunds/`), a language, two border treaties, and a design whose every element points back at an EARS ID. Next, the work gets cut into slices small enough to verify.

Part IV

Split It into Tasks

Chapter 12

Vertical Slices and the Three Laws of a Task

The spec says *what*; the plan says *where*; the task stage says *in what increments* — and the walkthrough opens it with the slicing method and its rationale in a single breath:

Slicing method: **vertical slices** that each ship alone, ordered so a *walking skeleton* exists early.

Vertical slices that each ship alone. A slice cuts through the whole stack for one behavior — contract, service, domain, test — rather than layering horizontally (“first all the models, then all the endpoints”). The test of verticality is shippability: each slice, done, could go to production behind its flag without waiting for its siblings. This is not an aesthetic preference; it is what makes the pilot’s deployment-frequency number possible later (*twenty verified tasks merged behind flags produced 16 production deploys in 10 working days*) and what makes lead time fall for the stated mechanism — *less queueing, because one-day slices don’t wait behind each other*.

Ordered so a walking skeleton exists early. The first slices assemble a minimal end-to-end path — fixtures, harness, endpoint shell — so that every later slice lands into a running, testable world instead of a pile of parts. On PLT-4821, T1 and T2 are the skeleton; T3, the domain heart, arrives third, into a world that can already exercise it.

Then come the three laws, and the walkthrough is precise about their enforcement — *the tasks command refuses to break them*, the same refusal pattern as /plan’s gate:

Rules (the tasks command refuses to break them): every task is **1 : 1 : 1** — one EARS criterion, one contract, one ticket; sizes **S ($\leq \frac{1}{2}$ day)** or **M (≤ 1 day)**, **never L** — an L is a splitting instruction; every task declares Touches: globs for the resource gate.

12.1 Law 1 — 1 : 1 : 1

One EARS criterion, one contract, one ticket, per task. The ratio is the traceability engine of the whole method: it is what lets `verification.md` map criteria to tasks to named green tests row by row, what lets contract coverage be computed by counting IDs, and what lets a retro trace a first-pass failure back to *which spec section* was ambiguous. Break the ratio — a task serving two criteria, a criterion smeared across three tasks — and every one of those ledgers blurs.

12.2 Law 2 — S or M, never L

Sizes are **S (at most half a day)** or **M (at most one day)**. There is no L, and the phrasing of the ban is the point: *an L is a splitting instruction*. If a task estimates beyond a day, that is not information about the estimate; it is an instruction to slice again — usually a sign that the task is secretly serving more than one criterion or crossing a context border. The one-day ceiling is also what turns estimation into counting (Chapter 14) and what keeps rollbacks thin (Part X's MTTR story: *thin slices = thin rollbacks*).

12.3 Law 3 — declared Touches: globs

Every task declares, up front, the file globs it is allowed to modify. This is where the task stage shakes hands with the enforcement machinery of Part IX: the Touches: list is the allow-list the **resource gate** enforces. Set the active task, and an edit outside its declared globs is answered by the `PreToolUse` hook with exit 2 — *a hook is enforcement; a prompt is persuasion*. Note what this does to decomposition quality: a task whose Touches: list sprawls across contexts is confessing, in its own metadata, that it was sliced wrong. Write the globs and the gate will keep you honest; struggle to write them and you have found an L wearing an M's clothes.

Chapter 13

The Six Tasks of PLT-4821

The derivation — *criterion* → *contract* → *task* — lands in `specs/checkout-refunds/tasks.md` (via `/tasks checkout-refunds`), and the walkthrough presents it as a table worth reproducing exactly:

Task	From	Contract	Size	Why this slice
T1 · fixtures & harness	EARS-1	feature background	S	walking skeleton: gives every later test its verbs
T2 · endpoint skeleton	EARS-3	OpenAPI 201/409	S	contract-first: FE can mock <i>today</i> ; 409 path proven early
T3 · refund calculation	EARS-2	POST + “refund on cancellation” scenario	M	the domain heart — the demo’s live task
T4 · refund.issued event	(out-of-scope wiring)	event schema v1	S	isolates Notification coupling
T5 · status UI slice	scope §2	pact checkout-web ↔ orders-svc	M	consumer-driven contract pair
T6 · expired-token guard	EARS-4	OpenAPI 401	S	unwanted-behaviour pattern gets its own slice

Six rows; read them as six small case studies in slicing.

T1 — fixtures & harness (EARS-1, S). The walking skeleton’s first bone. Its

contract is the *feature background* — the shared Given-setup every scenario will use — and its purpose is stated as a gift to the future: *gives every later test its verbs*. T1 is where the shared in-memory harness (the “one oracle” of Part VI) and the Money/minor-units fixtures come to exist, which is why its criterion is the ubiquitous EARS-1: the always-true rule about cents is baked into the fixtures everything else stands on.

T2 — endpoint skeleton (EARS-3, S). Contract-first in the most literal sense: the OpenAPI contract promising 201 on success and **409** on duplicate is published before the behavior behind it exists. Two payoffs are named. *FE can mock today* — the front end develops against the contract immediately, which is what will let T5 proceed in parallel. And *the 409 path is proven early* — the duplicate-rejection rule, the feature’s riskiest invariant, has an executable promise from day one rather than being bolted on at the end.

T3 — refund calculation (EARS-2, M). *The domain heart — the demo’s live task*. One criterion (the WHEN of cancellation), one contract (the POST plus the “refund on cancellation” Gherkin scenario), one folder (`src/refunds/`), one day at most. This is the task whose ACTIVE_TASK line the demo sets, whose red test Part VII exhibits, and whose green is four lines.

T4 — refund.issued event (out-of-scope wiring, S). The Out-list, embodied. Its “From” column honestly says (*out-of-scope wiring*) — it serves the spec’s Out-list rather than a criterion — and its contract is the *event schema v1* from the context mapping. Its stated purpose, *isolates Notification coupling*, is the physicalized Out-list again: all knowledge of notification lives here, so T3 never learns what a template is.

T5 — status UI slice (scope §2, M). The other Out-list item, and the table’s lesson in cross-service contracts: its contract is a **Pact** — the consumer-driven contract pair between checkout-web and orders-svc. The consumer states what it needs; the provider verifies it; and because the pact stub exists, T5 runs *parallel* to the backend critical path rather than behind it.

T6 — expired-token guard (EARS-4, S). The Unwanted pattern’s slice, with its OpenAPI 401 contract. The rationale column repeats the principle from Part II: *the unwanted-behaviour pattern gets its own slice* — failure handling as first-class work with its own criterion, contract, and ticket, not a `// TODO: auth` inside T3.

Notice, finally, what the table does *not* contain: EARS-5. Line-item refunds are behind the flag, out of the slice, and therefore have no task — which is exactly how the coverage arithmetic of Part XI will want it: *4/4 in-scope, EARS-5 consciously excluded*.

Chapter 14

Estimation Becomes Counting

With the table drawn, the walkthrough sequences and prices it in three dense lines:

Critical path: $T1 \rightarrow T2 \rightarrow T3$; T4/T6 hang off it; T5 parallel via the Pact stub. Estimation summary: $4 \times S + 2 \times M \leq 4$ **dev-days**, with **1.5 days of review & verification bandwidth booked first** — with agents typing, verification is the constraint, so it gets capacity before development does. Notice what happened to estimation: it became **counting**.

14.1 The shape of the schedule

The **critical path is $T1 \rightarrow T2 \rightarrow T3$** : harness, then endpoint, then the domain heart — the walking skeleton growing its organs in dependency order. **T4 and T6 hang off it**: the event publication needs T3's seam to exist; the token guard needs T2's endpoint to guard; both are S-sized decorations on a finished spine. **T5 runs parallel via the Pact stub**: because T2 published the contract and the pact provides a stub, the UI slice never waits for the backend at all. The dependency diagram is short, and it is short *because* the slices are vertical — horizontal layers would have made everything depend on everything.

14.2 The arithmetic

Four S tasks (each $\leq \frac{1}{2}$ day) plus two M tasks (each ≤ 1 day) bounds the build at **four developer-days**. The estimate required no poker, no story points, no

negotiation — only the size ceilings and the count. That is the meaning of the closing aphorism: *notice what happened to estimation: it became counting*. When every task is provably ≤ 1 day or it isn't a task, the sum is arithmetic; the uncertainty that estimation rituals exist to manage was removed upstream, by the slicing laws, where removing it is cheap.

14.3 The line most teams skip

And then the allocation that reads strangely until you remember whose hands are on the keyboard: **1.5 days of review & verification bandwidth booked first** — before development capacity, not after. The reasoning is the era's economics in one clause: *with agents typing, verification is the constraint*. Agents have made the production of candidate code cheap and fast; what has not gotten cheaper is a human reviewing every diff *against the criteria* and a verifier flipping rows to green with evidence. In queueing terms, the bottleneck moved — and capacity planning that still books “dev days” first is planning for the previous decade. The pipeline books the constraint first, which is also why Part X measures the wait from Green to Verify (four hours median, target under a day) and treats *verification never starves the pipeline* as a goal worth a GSM chain of its own.

The stage's artifact — `tasks.md` — now holds six slices, three laws, a critical path, and a bounded price. What it does not yet hold is a single line of design for *how* the code inside those slices should be shaped. That is deliberate, and it is next.

Part V

Architecture: Patterns and Principles, Applied

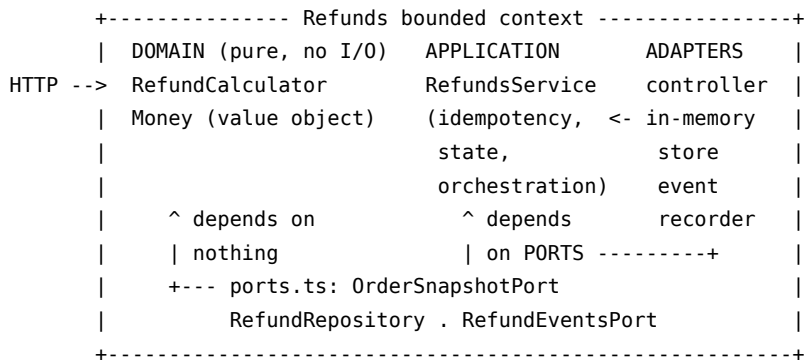
Chapter 15

The Shape: Hexagonal, with a Functional Core

The architecture stage opens with a disclaimer that sets the tone for everything in this part:

The shape is **hexagonal (ports & adapters)** with a **functional core and an imperative shell** — chosen not for fashion but because every principle below buys a measurable debugging or testing property.

Not for fashion. Every pattern in this stage must pay rent in one of two currencies — a debugging property or a testing property — and the walkthrough audits each one’s payoff explicitly. The code in `src/refunds/` mirrors the shape exactly; here is the walkthrough’s diagram of the Refunds bounded context, lightly re-set for the page:



Three rings, strict dependency direction, and every element already familiar from earlier stages:

The domain ring — pure, no I/O. RefundCalculator (the pure domain service from the tactical sketch) and Money (the value object carrying EARS-1’s minor units and ADR-017’s rounding). The annotation is absolute: *depends on nothing*. No imports of storage, transport, or framework — which is what will make the T3 test four deterministic lines.

The application ring. RefundsService, owning exactly the three things the context-ownership sentence assigned to Refunds beyond the math: *idempotency, state, orchestration*. It performs the store lookup that yields EARS-3’s 409, drives the PENDING→ISSUED transition, and publishes refund.issued — and it *depends on ports*, never on concrete infrastructure.

The ports. Three narrow interfaces in ports.ts — OrderSnapshotPort (the frozen snapshot from Ordering, the context mapping as a type), RefundRepository (the keyed aggregate’s home), RefundEventsPort (the seam to Notification). The ports are the architecture’s border checkpoints: everything outside speaks to everything inside through them.

The adapters ring. The HTTP controller, the in-memory store, the event recorder — the replaceable outside. In tests and in the demo, the in-memory adapters *are* the infrastructure, which is precisely what makes the shared harness of Part VI possible.

One wiring rule completes the shape: main.ts is the **composition root** — *wiring only*. The one file allowed to know both an interface and its implementation, and allowed to know nothing else.

Chapter 16

Ten Principles, Each with Its Payoff

The walkthrough then walks its principles “*principle by principle — each with its payoff*,” and the payoff column is the part to steal: it is how you defend an architecture review with something other than taste. Ten entries, verbatim in spirit, expanded in commentary.

16.1 SRP — Single Responsibility

`RefundCalculator` computes an amount. Nothing else — no storage, no events, no HTTP. Payoff: the T3 test is four lines and deterministic.

The classic principle, with an unusually concrete receipt. Because the calculator does exactly one thing, its test needs exactly one arrangement and one assertion — the four lines Part VII will display. Every responsibility you add to a unit is a fixture you add to its test.

16.2 Functional core, imperative shell

The calculator and `Money` are pure; all I/O lives in the service + adapters. Payoff: bugs in refund *math* reproduce on a laptop in milliseconds — never “only in staging, around noon.” Purity is search-space compression.

The Bernhardt pattern, and the payoff phrase deserves a frame: *purity is search-space compression*. A pure function’s behavior is fully determined by its arguments, so a math bug reproduces anywhere, instantly, with no environment, no clock, no database state — the search space for “why is this refund wrong?” compresses from *the running system* to *one function and its inputs*. The despair-inducing bug class — “only in staging, around noon” — is definitionally an impurity leak, and the shell is where all such leaks are quarantined.

16.3 DIP + DI — Dependency Inversion, with injection

RefundsService depends on the three **ports** (interfaces in `ports.ts`), never on Dynamo, HTTP or the bus. `main.ts` is the *composition root* — wiring only. Payoff: swap a fake in one line and answer “my logic or their service?” in one test run.

The payoff is a diagnostic superpower stated as a question: *my logic or their service?* When production misbehaves, swap the real adapter for a fake in one line of the composition root; if the failure persists, the bug is yours; if it vanishes, it is theirs. Dependency inversion turns the oldest finger-pointing argument in distributed systems into a one-test-run experiment.

16.4 ISP — small interfaces

Three narrow ports instead of one god Repository. Payoff: test doubles are five lines; adapters can’t grow hidden coupling.

Interface segregation, priced in test-double lines. A god interface breeds forty-method fakes and lets adapters silently depend on each other’s corners; three narrow ports keep every fake trivial and every dependency visible in a signature.

16.5 Value objects — make illegal states unrepresentable

Money holds *integer cents* and throws on anything else; refund status is a closed union. Payoff: the float-drift and half-cent bugs from the origin story don't sneak past review — they **don't compile**.

EARS-1 as a type. Money is constructed only from integer cents; the refund status is a closed union of the ubiquitous nouns (PENDING, ISSUED, ...). The payoff escalates the enforcement ladder one final rung: review catches what reviewers notice; hooks catch what hooks check; the *type system* catches everything, before the program exists. Float drift is not discouraged; it is unrepresentable.

16.6 Idempotency as a domain rule, not middleware

The Refund aggregate is keyed by `orderId`; the service's first act is a store lookup → 409 (EARS-3). Payoff: retries, double-clicks and replayed events are all the same non-event.

The tactical sketch's key choice, restated as principle. Idempotency implemented as middleware protects one transport; idempotency as a *domain rule* — a key collision on the aggregate — protects every path into the domain, present and future. Hence the payoff's sweep: retries, double-clicks, replayed events — *all the same non-event*.

16.7 Domain events at the seam

The service publishes `refund.issued v1` through the events port; Notification subscribes. Payoff: T4 ships without touching T3's code — the Out-list made physical.

The context mapping's downstream edge, now load-bearing in the task plan. Because the only coupling to Notification is a versioned event through a port, the Out-list is not a promise kept by discipline but a fact of the dependency graph: *the Out-list made physical*.

16.8 One decision, one place, one record

Rounding happens once, in `Money.roundBankers`, cited by **ADR-017**.
Payoff: the next engineer who “fixes” $2.5 \rightarrow 3$ by intuition finds the ADR, not a debate.

The rule of the audited single point. Every refund’s rounding passes through one method, and the method cites its decision record. The payoff scene is vivid and universal: an engineer, certain that 2.5 obviously rounds to 3, opens the code to fix it — and finds ADR-017 explaining banker’s half-to-even. *Finds the ADR, not a debate*. A decision that isn’t written gets re-litigated; a decision that is written and *cited from the code* gets read.

16.9 OCP, gently

New refund policies (line-item, EARS-5) extend by a new calculator behind the same call shape — no modification of the verified path. We do *not* pre-build the abstraction: the flag stays in the spec until it’s real. (Patterns are applied where they pay, not collected.)

Open-closed, with the emphasis on *gently*. When EARS-5’s day comes, line-item refunds arrive as a new calculator behind the same call shape, leaving the verified path unmodified. But — and this is the sentence that separates this architecture chapter from a patterns catalogue — the abstraction is **not** pre-built. No strategy interface, no policy registry, no speculative generality; *the flag stays in the spec until it’s real*. The parenthetical is the whole philosophy: *patterns are applied where they pay, not collected*.

16.10 CQS — Command-Query Separation

`calculate()` answers; `requestRefund()` changes state and reports. No method does both quietly.

The quiet closer. Queries answer without changing anything; commands change state and say so. The operative word is *quietly*: the sin is not mixing effect and answer per se, but hiding the mix — a “getter” that mutates is a debugging trap with a friendly name.

Chapter 17

Where the Tests Attach

The chapter closes by turning the rings into a testing map, and the walk-through’s paragraph is a complete test strategy in five lines:

Tests attach at two seams: **domain** (calculator: pure unit tests — fast, exhaustive) and **application** (service against in-memory adapters: the Gherkin and the 409 path). The HTTP adapter is covered by the OpenAPI contract; the cross-service promise by Pact (T5). Every layer has exactly one kind of test, and every test knows which layer it accuses when it fails.

Read it as a matrix. **Domain seam:** the calculator gets pure unit tests — *fast, exhaustive*, because purity makes exhaustive cheap. **Application seam:** the service runs against the in-memory adapters, and this is where both Gherkin scenarios and the 409 path execute — the shared harness of “one oracle, two dialects” is, architecturally, exactly this seam. **HTTP adapter:** covered not by more unit tests but by the **OpenAPI contract** that T2 published — the promise is the test. **Cross-service:** the **Pact** pair from T5 covers the promise between checkout-web and orders-svc.

Then the two closing clauses, which are the payoff of the entire part compressed. *Every layer has exactly one kind of test* — no redundancy to maintain, no gaps to argue about; the layer determines the test kind mechanically. And *every test knows which layer it accuses when it fails* — the debugging guarantee again, now finer-grained than “one folder”: a red calculator test accuses pure math; a red 409 scenario accuses the service; a red OpenAPI check accuses the controller; a red pact accuses the border. Failure comes pre-localized.

The architecture stage produces no dramatic artifact — `ports.ts`, a service skeleton, `docs/code-structure.md` — because its output is mostly *constraints*. But every constraint was purchased for a payoff, and the invoice is now on record. Next, the constraints meet the contracts.

Part VI

Contracts: The BDD Dialect

Chapter 18

The Shared Dialect

Everything so far has produced text that humans agree on. This stage produces text that humans agree on *and a machine can execute* — and the walkthrough defines it by its audience:

BDD is the **shared dialect**: the EARS criteria, re-expressed as Given-When-Then that the PO can read *and* the pipeline can execute. The translation is nearly mechanical — that’s a feature.

Two claims, both structural. First, the *shared* in shared dialect is literal: the same sentences serve the product owner reviewing intent and the CI pipeline detecting regressions. There is no “business version” and “technical version” to drift apart; there is one `.feature` file, and both constituencies read it. Second — and this is the counterintuitive one — *the translation is nearly mechanical, and that’s a feature*. A translation requiring creativity is a translation that can smuggle in new meaning; a mechanical one cannot. The EARS shapes were chosen in Part II precisely so that this stage would be boring.

Watch the mechanism in the walkthrough’s own translation table for `contracts/refunds.feature`:

EARS	Gherkin (in <code>contracts/refunds.feature</code>)
EARS-2: WHEN an event is cancelled...	<i>Given</i> a paid order for event “EVT-303” · <i>When</i> the event is cancelled · <i>Then</i> a refund is issued for price minus service fees · <i>And</i> the order status is “REFUNDED”
EARS-3: WHILE PENDING/ISSUED... 409	<i>Given</i> the order already has a refund · <i>When</i> a refund is requested again · <i>Then</i> the request is rejected with status 409

Study the mapping grammar, because it generalizes to any criterion you will ever translate:

Trigger → When. EARS-2's *WHEN an event is cancelled* becomes, almost character for character, *When the event is cancelled*. The Event pattern's trigger slot and Gherkin's When slot are the same slot.

Precondition → Given. What EARS leaves implicit — that there exists something to refund — Gherkin makes concrete with an example: *Given a paid order for event "EVT-303"*. The Given step is where a criterion acquires its specimen. Similarly, EARS-3's WHILE-state ("PENDING or ISSUED") becomes the Given of the second scenario: *Given the order already has a refund* — the state precondition, instantiated.

Response and constraint → Then/And. *The system shall refund the ticket price minus service fees* becomes *Then a refund is issued for price minus service fees*; the observable side effect gets its And (*the order status is "REFUNDED"*); the rejection rule keeps its number (*rejected with status 409*).

The ubiquitous language rides along untranslated. *Service fees, the event is cancelled*, the states, the 409 — every noun crossing the table is a noun from Part III's vocabulary, verbatim. The dialect is shared because the language already was.

Two scenarios for a five-criterion spec is not an omission — it is the task plan speaking: EARS-1 lives in the feature *background* (task T1's contract — the fixtures every scenario stands on), EARS-4's 401 belongs to T6's OpenAPI contract, and EARS-5 is out of the slice. The Gherkin file carries exactly the behaviors whose home is the application seam, which Part V already identified as *where the Gherkin and the 409 path attach*.

Chapter 19

Rules for Scenarios That Stay Healthy

Gherkin has a famous failure mode — scenarios that rot into brittle UI scripts — and the walkthrough forestalls it with four terse rules:

Rules for scenarios that stay healthy: declarative, domain-language steps (“*the event is cancelled*” — never “click the cancel button”); one behaviour per scenario; shared setup in Background;; ubiquitous-language nouns only. If a scenario needs UI verbs, it’s testing the wrong layer.

19.1 Declarative, domain-language steps

The contrast pair is chosen to be memorized: “***the event is cancelled***” — ***never “click the cancel button.”*** The first states a domain fact and survives any redesign of how cancellation happens — a button, an API, an admin batch. The second welds the specification to one afternoon’s UI and breaks the day a designer moves the button. Declarative steps specify *what became true*; imperative steps script *how someone made it true*, and the how is not the requirement.

19.2 One behaviour per scenario

Each scenario asserts exactly one criterion’s worth of behavior. The discipline pays at failure time — the moment this pipeline optimizes for relentlessly: a

red scenario with one behavior names its accusation precisely; a red scenario with five assertions across three behaviors names a neighborhood. It also keeps the 1 : 1 : 1 traceability intact — one criterion, one contract — at the scenario level.

19.3 Shared setup in Background:

The *Given a paid order* style of setup, repeated across scenarios, is hoisted into the feature's Background: — which, not coincidentally, is exactly the contract task T1 was assigned. The walking skeleton's first slice built the verbs; Background: is where the scenarios borrow them. Setup written once is setup that cannot drift into two subtly different versions of "a paid order."

19.4 Ubiquitous-language nouns only

The vocabulary law of Part III, applied at the sentence level: if the noun is not in the ubiquitous language, it does not appear in a step. A scenario that suddenly speaks of "charges" instead of *service fees* is either wrong or announcing an unratified vocabulary change — both worth catching in review, and both trivially catchable when the rule is absolute.

And the summary heuristic, which doubles as a layer-placement test from Part V's matrix: ***if a scenario needs UI verbs, it's testing the wrong layer.*** Clicking belongs to whatever covers the UI; the application seam speaks domain.

Chapter 20

One Oracle, Two Dialects

Now the mechanism that gives this book its refrain. Scenarios are prose until something binds them to code, and the binding is where the design choice lives:

Step definitions (`features/steps/refunds.steps.ts`) bind the scenarios to the **same in-memory harness the unit tests use** — one oracle, two dialects.

The step definitions translate each Gherkin sentence into calls — and the calls land on **the same in-memory harness the unit tests use**. Not a parallel test rig, not a mock universe maintained separately for the business-facing tests: the *same* service wired to the *same* in-memory adapters at the *same* application seam that Part V designated. One implementation of truth, interrogated in two languages.

Sit with what the alternative costs, because most teams pay it without noticing. Two harnesses — one for “acceptance tests,” one for “unit tests” — means two setups that can drift, two definitions of “a paid order,” and, worst, the possibility of the dialects *silently disagreeing*: the business suite green, the unit suite green, each against its own private reality. With one oracle, disagreement between dialects is not a maintenance smell — it is a **loud contradiction about a single object**, surfaced the moment it exists. The dialects check the system; the shared oracle lets them also check each other.

Each dialect earns its keep by covering the other’s blind spot. Gherkin holds *intent fidelity*: the PO can read that a refund is *price minus service fees* and object to the sentence itself. Jest (next part) holds *numeric and edge fidelity*: exact cents, the rounding boundary, the retry path. Neither alone is the con-

tract; together, bound to one oracle, they are.

Chapter 21

Red BDD: Proving the Detector Detects

The stage ends with an execution, and the execution is supposed to fail:

```
npm run bdd
```

Run: `npm run bdd` → both scenarios **fail**, correctly, because nothing is implemented. Red BDD before code is the proof that the scenarios can detect absence — the same logic as TDD’s red, one level up.

Fail, correctly. Both scenarios go red, and the walkthrough insists on the adverb: this is the desired outcome, not a setback on the way to one. The logic is the epistemics of testing stated in one clause — **red before code proves the scenarios can detect absence**. A detector that has never fired on a known-empty world is a detector of unknown sensitivity; these two scenarios have now demonstrably fired on the emptiest world there is. When they later go green, the green will mean something, because the red already did.

The walkthrough’s tag — *the same logic as TDD’s red, one level up* — is also the bridge to the next part. The discipline is fractal: the failing scenario proves the business-level detector; the failing unit test will prove the code-level one; and the demo’s sanity check (`npm test` → RED on T3, `npm run bdd` → scenario fails) institutionalizes red as *the correct starting state* of the whole repository. In this pipeline you do not begin from zero; you begin from a verified, meaningful red — and everything after is the disciplined act of turning exactly that red, and nothing else, green.

Part VII

Red, Green, and the Anticlimactic Implementation

Chapter 22

The Red: Derived from the Contract

The TDD stage opens with its one absolute rule of provenance, and the parenthesis is the proof:

The failing test is derived **from the contract, never from the implementation** (there is no implementation).

There is no implementation. The clean-room condition is not a discipline the team performs; at this point in the pipeline it is a fact of the repository. The test cannot copy the code's assumptions because the code does not exist — which is precisely the property Part I demanded when it observed that an agent's tests, written after its code, merely certify the agent's own interpretation. Here the direction of information flow is fixed by construction: contract → test → (later) code.

For T3, the walkthrough says, *the contract names the oracle and the test writes itself* — and then shows it, in `src/refunds/application/refunds.service.spec.ts`:

```
it("refunds price minus fees when the event is cancelled", async () => {
  const order = await paidOrder("EVT-303");
  await events.cancel("EVT-303");
  expect(await refundFor(order.id)).toEqual(
    order.total.minus(order.fees) // RED – no implementation yet (T3)
  );
});
```

Four lines of body — the count SRP's payoff promised — and every line is a quotation from an earlier stage. The description string, *"refunds price*

minus fees when the event is cancelled," is EARS-2 in the ubiquitous language. `paidOrder("EVT-303")` is the Gherkin's Given, speaking through T1's fixtures — the same specimen, the same event ID, because the same harness. `events.cancel("EVT-303")` is the When — the trigger as a domain event, exactly as the storming named it. And the assertion, `order.total.minus(order.fees)`, is the criterion's arithmetic expressed through the Money value object — the *expected* value computed from the frozen snapshot's own fields, so the test asserts the *relationship* the spec demands rather than a magic number someone once believed.

Run it:

```
npm test
```

Run `npm test` → **RED**, with `NotImplemented – task T3`. This failure is an asset: it proves the detector works. A test written after the code passes *by construction* and proves nothing.

Two details in that red are worth the frame. The failure message — `NotImplemented – task T3` — is *labelled with its task*: the stub in `solution/` fails by name, so even the starting redness participates in the traceability ledger. And the epistemics get their sharpest statement in the whole walkthrough: *a test written after the code passes by construction and proves nothing*. The red is not a to-do item; it is **evidence** — the only moment you will ever directly observe that this detector can detect this absence. The pipeline treats that evidence as an asset to be manufactured deliberately, which is why the demo's sanity check insists the repo *start* red and why the reset procedure restores redness before every run.

Chapter 23

Red, Green, Refactor — with Triangulation

The loop itself is stated as four numbered disciplines, and each number carries a rule most retellings soften:

1. **Red** — smallest failing test from the contract. No production code without one.
2. **Green** — the *minimum* to pass. Resist generality; the next red test earns it (triangulation: EARS-1's minor-units test and the 409 test pin the edges the first test doesn't).
3. **Refactor** — only while green: names to ubiquitous language, extract, de-dupe.
4. Repeat until the task's Done: line is literally true.

Red: smallest failing test, from the contract — and no production code without one. The second clause is the enforcement half: production code exists only in response to a red test, which means every behavior in the codebase has, somewhere in history, a moment when its absence was observed. “Smallest” keeps the accusation sharp — a small red names one missing thing.

Green: the minimum to pass — resist generality. The counterintuitive rule, with its own justification: *the next red test earns it*. Generality written speculatively is generality untested; generality demanded by a second failing test is generality with evidence. The walkthrough names the mechanism **triangulation** and shows PLT-4821's instance: the first test pins price-minus-fees; **EARS-1's minor-units test** and **the 409 test** then *pin the edges the first test doesn't* — the precision policy and the duplicate rule arrive as their

own reds, and only then does the code grow to meet them. Three criteria, three reds, three minimal greens: the spec drives the code's growth ring by ring.

Refactor: only while green. Structure changes happen exclusively under a passing suite, so every rename and extraction is continuously certified behavior-preserving. And note the pipeline's house style for what refactoring *is*: *names to ubiquitous language, extract, de-dupe* — the first refactoring listed is linguistic, pulling the code ever closer to the vocabulary the PO speaks, because Part III's law ("the same nouns... in the type names") is maintained here, in the refactor step, forever.

Repeat until the task's Done: line is literally true. Not "feels done" — *literally* true: the task's own completion sentence, read against the suite, with no interpretive charity. The loop's exit condition is a string match against reality, which is the only kind of exit condition this pipeline trusts.

Chapter 24

The Agent-Assisted Variant

Everything above described a human at the keyboard. The variant paragraph swaps the typist and changes almost nothing else — which is the point:

Agent-assisted variant: set the leash — echo "checkout-refunds T3" > .claude/state/ACTIVE_TASK — then /implement T3. The agent runs the same loop; the human reviews every diff **against the criteria** (conformance, not style). Try an edit outside Touches: and the hook answers exit 2: a hook is enforcement; a prompt is persuasion.

The sequence is three moves. **First, the leash:**

```
echo "checkout-refunds T3" > .claude/state/ACTIVE_TASK
```

One line declaring, to the enforcement machinery of Part IX, which single task the agent may touch. The metaphor is the walkthrough's own — *set the leash* — and it is set *before* the agent is invoked, because permission precedes action in a fail-closed system.

Then the invocation: /implement T3. The agent runs *the same loop* — red from the contract, minimal green, refactor under green — not a special agent workflow. The pipeline's stages were designed to be typist-agnostic; the contract, the fixtures, and the labelled red stub give an agent the same unambiguous starting state they give a human.

And throughout, the review rule: the human reviews *every diff against the criteria* — *conformance, not style*. This is the human role, redefined for the era Part IV priced when it booked verification bandwidth first. The question at review is never “would I have written it this way?” but “does this diff satisfy EARS-2, respect the Out-list, and stay inside Touches:?” Conformance

review scales with agent output in a way style review never will, and it is the review the artifacts make cheap: the criteria are enumerated, the contract is executable, and the blast radius is declared.

The paragraph closes by demonstrating the enforcement rather than describing it: *try an edit outside Touches: and the hook answers exit 2* — the Pre-ToolUse gate, blocking the write before it lands. The demo makes theater of exactly this (Beat 5 asks the agent to touch `src/payments/infrastructure/psp.ts` and lets the room watch the refusal), and the aphorism lands here in its native habitat: **a hook is enforcement; a prompt is persuasion.** You asked the agent to stay in scope when you wrote the command; the hook is what makes staying in scope not a choice.

Chapter 25

The Anticlimactic Green

And then, the implementation — introduced with the sentence that reframes what a pipeline is *for*:

The green for T3 is deliberately anticlimactic — the whole pipeline exists to make it so.

Here it is, from `solution/refund-calculator.solution.ts` (or live-typed on stage):

```
export class RefundCalculator {  
  calculate(order: Order): Money {  
    const cents = Money.roundBankers(order.total.cents - order.fees.cents);  
    return Money.ofCents(cents); // EARS-2 · minor units per EARS-1 · ADR-017  
  }  
}
```

Read it slowly; every token has a chapter behind it. `order.total.cents - order.fees.cents` — EARS-2’s arithmetic, on the frozen snapshot’s fields, in EARS-1’s integer cents, where subtraction is exact. `Money.roundBankers(...)` — ADR-017’s audited single rounding point. `Money.ofCents(...)` — the value object refusing to exist except as legal money. And the comment is a citation trail — *EARS-2 · minor units per EARS-1 · ADR-017* — the code annotating itself with the decisions it implements. The demo calls it what it is: about four lines, the two-hour Sprint-14 story.

The walkthrough’s commentary defuses the one apparent oddity before anyone asks:

Commentary: integer-cent subtraction is exact, so today the rounding guard is a no-op — it exists because ADR-017 says *all* refund math passes through one audited rounding point, so tomorrow’s

percentage-fee change can't reintroduce the origin-story bug.

A no-op *today*, and deliberately present anyway: the guard is not for this arithmetic but for the *invariant* — all refund math through one rounding point — so that the day someone introduces a percentage fee, the half-cent bug has nowhere to be born. This is defensive design justified by a written record rather than by superstition, which is the only kind that survives review.

The function stays pure; the shell keeps its duties: *the service (imperative shell) owns the store lookup → 409, the state transition, and the refund.issued publish — all behind ports.* The rings of Part V, honored in the final artifact.

Two disciplines govern the implementer while typing, and both are restraints rather than techniques:

Discipline while implementing: **the Out-list is law** — T4's event templates and T5's UI stay untouched even when "it would only take a minute"; anything learned that contradicts the spec goes back as an **open question or ADR**, never a silent interpretation.

The Out-list is law. The quoted temptation — "*it would only take a minute*" — is the exact voice in which scope creep speaks, and the answer is categorical: T4's templates and T5's UI stay untouched. The architecture already made the Out-list physical; this rule keeps the human from un-making it.

No silent interpretations. Implementation is where reality talks back — an edge the spec missed, a constraint nobody knew. The pipeline's routing rule for such discoveries is absolute: back upstream, as an *open question* (if it needs an owner's decision) or an *ADR* (if it is a decision with a rationale) — **never a silent interpretation**, because a silent interpretation is Part I's machine-speed assumption, merely typed by a person.

Then the close, both dialects at once:

```
npm test && npm run bdd    # GREEN — both dialects agree on the same oracle
```

Green, twice, against one harness: the Jest edges and the Gherkin intent, agreeing about a single object. The feature now *does* what the spec *says* — as far as these detectors can tell. Turning "as far as these detectors can tell" into an explicit, signed verdict is the next stage's entire job.

Part VIII

Verify: The Exit Gate

Chapter 26

The Anatomy of verification.md

Every stage so far ended in a gate; this stage is the gate — the walkthrough’s definition is a single clause:

Verification is the exit gate — conformance made explicit, in
specs/checkout-refunds/verification.md (or /verify checkout-refunds).

Conformance made explicit. Not quality assurance in the general sense, not exploratory testing, not sign-off vibes: a written, versioned answer to one question — does the implementation conform to the specification, criterion by criterion, with evidence? The artifact is `verification.md`, produced by hand or by the pipeline’s fifth command, and the walkthrough specifies its four components as a numbered anatomy. Take them in order, because the order is also a logic: coverage, then evidence, then verdict, then the metric the verdict feeds.

26.1 Component 1 — the criteria coverage table

Criteria coverage table — every EARS row maps to named green tests. EARS-2 flips Red – T3 open → **Green**. EARS-5 stays – (out of slice, *documented* as such).

One row per EARS criterion; each row maps the criterion to the *named* tests that prove it, and carries a status. The table is the 1 : 1 : 1 law of Part IV paying its final dividend: because every task served one criterion and one contract, the mapping from requirement to proof is a lookup, not an investigation.

Two rows are singled out, and each teaches a state of the table. **EARS-2 flips Red – T3 open → Green** — note that even the *red* was informative, carrying

the name of the task that owed it; the flip to Green is the moment this book's running example completes, and it is the exact moment the demo's final beat points at on stage. **EARS-5 stays** — not red, not green, but dash: *out of slice, documented as such*. The dash is a first-class status, and it is what keeps scope honesty measurable: coverage arithmetic later counts EARS-5 as *consciously excluded*, which is a different fact from *forgotten*, and the table is where the difference is recorded.

26.2 Component 2 — per-task evidence

Per-task evidence — suite names, results, and **deviations** → **ADRs**
(T3's row cites ADR-017: decided up-front, not during coding).

Below the criteria table, each task presents its receipts: which suites ran, what they reported — and, crucially, its **deviations**, each routed to an ADR. The rule of Part VII (“never a silent interpretation”) gets its ledger here: any place the implementation departed from, or elaborated beyond, the spec's letter must point at a written decision. T3's row cites ADR-017 — and the walkthrough's parenthesis savors the timeline: *decided up-front, not during coding*. The rounding policy was settled in the spec stage, logged, and merely *honored* during implementation; the evidence row proves the pipeline's promised ordering actually happened on this feature.

26.3 Component 3 — the verdict

Verdict — CONFORMS — <date> only when every in-scope row is green; one red row and the feature does not merge — it reopens the spec or the task, visibly.

The document ends in a single dated line, and the line is binary. CONFORMS — <date> may be written only when **every in-scope row is green** — in-scope, because the dashes of consciously excluded criteria do not block. And the contrapositive is the gate's teeth: *one red row and the feature does not merge*. No “known issues” rider, no shipping with an asterisk — the walkthrough's DORA table will later restate this as a mechanism (*Red never ships as “known issue”*), and the pilot's change-failure story will bank on it.

But notice what a red row triggers, because it is not merely “fix the code.” *It*

reopens the spec or the task, visibly. A verification failure is treated as information about an upstream artifact: either the spec was ambiguous (reopen the spec) or the slice was mis-cut (reopen the task) — and the reopening is *visible*, on the board, where Part XI’s automation will label it. The loop tightens instead of drifting: failure feeds the artifact that caused it, not a private branch of heroics.

26.4 Component 4 — the first-pass entry

First-pass entry — did this task conform without rework? That single boolean, aggregated, is the purest spec-quality metric you will ever own.

The last field is one boolean per task: *conformed without rework — yes or no*. Its cost is zero — the fact is already known the moment the verdict is written — and its value, aggregated, is the walkthrough’s superlative: *the purest spec-quality metric you will ever own*. The reasoning arrives in Part X, but the essence belongs here: a task that fails verification on the first pass almost always traces to an ambiguity that survived spec review — so the aggregate first-pass rate is a direct, trailing measurement of how good the specs were, harvested as a by-product of a gate you run anyway. Zero extra tooling; Part XI will show the Jira automation that collects it with a single label.

Chapter 27

The Demo Is the Verification

One paragraph remains in the stage, and it replaces an entire genre of process theater:

The demo *is* the verification: run the live slice behind its flag. No screenshots. If Verify found a gap, the spec — not the definition of done — would have changed, and the loop tightens instead of drifting.

Run the live slice behind its flag. No screenshots. The sprint demo, in this method, is not a slideshow about the work — it is the verification stage performed in public. The slice runs, live, behind the feature flag it shipped under; the audience watches behavior, not captures of behavior. The prohibition on screenshots is not aesthetic: a screenshot is evidence that something worked *once, somewhere*; a live run behind the production flag is evidence the verified thing is the deployed thing. (The demo runbook of Part XII applies the same standard to itself — its closing motto is *Spec first. Verify always — including this demo* — and its offline verification appendix is that promise kept.)

If Verify found a gap, the spec — not the definition of done — would have changed. This clause is the stage’s philosophy compressed. In most processes, a gap discovered at the end is absorbed by softening the finish line — the definition of done grows an exception, the ticket ships with a caveat, and the process learns nothing. Here the finish line is fixed (every in-scope row green; CONFORMS), so a gap has exactly one place to go: back into the spec, as a new or amended criterion, where the *next* feature inherits the lesson. The definition of done is invariant; the specification is what evolves. That is what

the loop tightens instead of drifting means mechanically — every verification failure ratchets the specs, and the first-pass metric exists precisely to watch the ratchet work, sprint over sprint, toward its target of ninety percent.

With CONFORMS – <date> written, PLT-4821’s story as a feature is over: specified, placed, sliced, shaped, contracted twice, implemented in four lines, and verified row by row. What remains is everything around the story — the machinery that enforced it (Part IX), the numbers that prove it moved the needle (Parts X–XI), the twelve minutes that sell it to a room (Part XII), and the playbook for making it a team’s habit rather than one feature’s biography (Part XIII).

Part IX

Enforcement over Persuasion: The Tooling

Chapter 28

The Five Commands

The pipeline’s stages are driven by five Claude Code slash-commands, each defined by a markdown file in the repo — the demo runbook’s inventory lists them with their one-line jobs:

```
.claude/commands/  
  specify-from-jira.md      Jira → spec.md, gated on EARS quality  
  plan.md · tasks.md · implement.md · verify.md
```

Map them to the stages you have now walked: `/specify-from-jira` PLT-4821 turns the ticket into the EARS spec, *gated on EARS quality* — the command itself refuses drafts that fail the patterns, and carries unresolvables into Open questions rather than guessing. `/plan` checkout-refunds produces the DDD placement and tactical sketch — and refuses while Open questions is non-empty. `/tasks` checkout-refunds derives the slice table — and refuses to break the three laws (1 : 1 : 1; S or M, never L; declared Touches:). `/implement` T3 runs the red→green loop on exactly one task. `/verify` checkout-refunds writes the coverage table, evidence, and verdict.

Notice the recurring verb: *refuses*. Each command asserts its stage’s preconditions. But a command file is, in the end, a structured prompt — and the framework’s sharpest distinction, the one this whole part exists to elaborate, is that prompts and guarantees are different species:

A hook is enforcement; a prompt is persuasion.

Commands orchestrate. The command text tells the agent what to read, what to produce, which rules to honor — and a capable agent will usually honor them. *Usually* is the operative word, and “usually” is not a property you build a pipeline on. For the rules that must hold — an agent may touch only its

declared task's files — the pipeline does not ask. It installs a hook.

Chapter 29

The Resource Gate

The enforcement backbone is one shell script registered as a **PreToolUse hook**:

```
.claude/hooks/resource_gate.sh    the PreToolUse gate (exit 2 = blocked)
```

Its operating contract is brutally simple. Before the agent’s tool use lands — before an edit is written — the hook fires. It learns the active task from `.claude/state/ACTIVE_TASK` (the file whose one line, `checkout-refunds T3`, the walkthrough calls *the leash*), checks the attempted path against that task’s declared Touches: globs from `tasks.md`, and answers with an exit code. **Exit 2 = blocked**: the write never happens. Not “the agent is warned,” not “a violation is logged for later” — the action is *prevented*, at the same layer of reality where the action would have occurred.

The demo runbook’s offline-verification section enumerates the gate’s behavior as **four modes**, all exercised in a clean container before the script is trusted on stage:

the gate’s four modes (allow in-scope, block out-of-scope, allow specs/docs, block without `ACTIVE_TASK`)

Read them as the gate’s whole decision table:

1. **Allow in-scope.** The edit’s path matches the active task’s Touches: globs → permitted. The gate is not an obstacle to sanctioned work; T3’s four lines land in `src/refunds/` without friction.
2. **Block out-of-scope.** The path is outside the globs → exit 2. This is the mode the demo stages deliberately: ask the agent to edit `src/payments/infrastructure/psp.ts` while T3 is active, and *the hook*

answers with exit 2: outside T3's declared resources. The Payments context is another bounded context; the gate is the context map with teeth.

3. **Allow specs/docs.** Specification and documentation paths remain writable — because the pipeline's own rules *require* upstream writes: a discovery during implementation must go back as an open question or an ADR, never a silent interpretation. A gate that blocked the spec would block the loop-tightening the method depends on.
4. **Block without ACTIVE_TASK.** No active task declared → *everything* is blocked. This is the fail-closed default, and the troubleshooting table insists on reading it correctly: *"Agent edits refused everywhere — .claude/state/ACTIVE_TASK missing — **that's the gate working**; set the task."* The safe failure of a scoping system is refusal, not free rein; a gate that failed open would be a prompt with extra steps.

One operational note from the same troubleshooting table: the hook resolves paths via `CLAUDE_PROJECT_DIR`, so when testing it manually outside a Claude Code session, export that variable — *"Hook doesn't block: CLAUDE_PROJECT_DIR unset when testing manually — run via Claude Code, or export it."* A gate you cannot test by hand is a gate you will not trust; the runbook makes hand-testing a supported mode.

Step back and see what the gate closes. Part I's threat model was ambiguity resolved at machine speed across many files; Parts II-IV shrank the ambiguity and declared the blast radius; the gate makes the declared radius *physical*. With the leash set, the worst an over-eager agent can do is fail loudly inside one task's territory — and a loud failure inside a small territory is exactly the kind of failure this pipeline knows how to convert into a reopened task and a tighter spec.

Chapter 30

Providers, Sessions, and the One Toggle Rule

The agent behind the commands needs a model, and the prerequisites table gives the two supported routes and the single operational rule that governs them:

Model access: `claude` starts a session — Anthropic API key **or** Amazon Bedrock (`CLAUDE_CODE_USE_BEDROCK=1` + AWS creds). If you keep a provider-toggle script, run it before the session and *never* switch mid-demo.

The mechanics are one environment variable: the default route is the Anthropic API; setting `CLAUDE_CODE_USE_BEDROCK=1` (with AWS credentials in place) routes the same Claude Code through Amazon Bedrock instead — the enterprise path when model traffic must stay inside an AWS boundary. The pipeline is identical either way; the commands, hooks, and artifacts neither know nor care which door the model came through.

The rule is the part teams violate at the worst moments: *run the toggle before the session and never switch mid-demo* — and by extension, treat provider as a **session-level constant** in ordinary work too. Mid-session switches change authentication, latency, and environment at once, which is three variables too many while a red test is on the screen. Baseline prerequisites, then hold them: the same discipline the metrics part will demand of its instruments (*keep the same instruments running*), applied to the agent itself.

The remaining prerequisites complete the environment in five checks — Node.js $\geq$ 20 (22 recommended), Claude Code installed globally (`npm i`

-g @anthropic-ai/claude-code), the repo unzipped, npm install for jest/ts-jest/cucumber/ts-node, and then the sanity check that defines this repository's idea of health: npm test → **RED on T3**, npm run bdd → **scenario fails** — *red is the correct starting state*. Plus one flourish with a purpose: npm run typecheck:noinstall *proves the skeleton compiles before any deps* — *nice opener line* — a zero-dependency typecheck that works on a machine with nothing installed, which is also why the offline-verification list could certify the stub *and* the solution compile without network access.

Chapter 31

MCP: When the Agent Reads Jira Itself

Three ways exist to put the villain in front of the pipeline (Part XII details all three for the stage); the third is the interesting one for tooling, because it changes *who* does the reading. Via the Model Context Protocol, the agent connects to Atlassian directly — one entry in `.mcp.json`:

```
{ "mcpServers": { "atlassian": {  
  "type": "url", "url": "https://mcp.atlassian.com/v1/mcp" } } }
```

Authenticate once beforehand, confirm with `claude mcp list`, and `/specify-from-jira` PLT-4821 can pull the issue live — no export script, no committed JSON, the freshest possible ticket.

And then the runbook attaches two disciplines to the power, both worth generalizing beyond the demo:

Name the token cost out loud. The live-pull comes with a built-in talking point: *“note the token cost of that server — slide 23 in action.”* An MCP server’s tools and results occupy context; freshness is purchased in tokens. The pipeline’s default (offline JSON) is not technophobia — it is the recognition that a deterministic, free, committed input beats a live, metered one whenever freshness isn’t the point.

Remove the server after the demo; scope per repo, as preached. Connectivity is configured where it is needed, for as long as it is needed, and torn down after — *as preached*, because the same minimalism already governs everything else in this part: the leash names one task, the gate allows one blast radius, the session pins one provider. An MCP server left configured “because

it might be handy” is standing capability with standing cost, and the reset procedure (Part XII) treats removing it as part of returning the repo to a known state.

Chapter 32

The Repository, Mapped

Close the part with the whole machine on one screen — the demo runbook’s inventory of what ships in the repo, verbatim:

```
jira/PLT-4821.json           the vague ticket (the villain)
.claude/commands/
  specify-from-jira.md       Jira → spec.md, gated on EARS quality
  plan.md · tasks.md · implement.md · verify.md
.claude/hooks/resource_gate.sh the PreToolUse gate (exit 2 = blocked)
.claude/state/ACTIVE_TASK    which task an agent may touch
specs/checkout-refunds/     the six artifacts (already worked, for reference)
docs/ddd.md                 bounded-context map – the DDD talking slide
features/steps/             BDD step definitions binding the .feature file
solution/                   RED stub + GREEN solution + demo:red / demo:green
```

Every line is a chapter you have read. The villain (Part I). The five commands with their gates (Parts II–IV, VIII). The hook and the leash (this part). The six worked artifacts of `specs/checkout-refunds/` — spec, plan, tasks, contracts, steps’ feature, verification — the pipeline’s paper trail for one feature (Parts II–VIII). The context map that is also *the DDD talking slide* (Part III). The step definitions binding Gherkin to the shared harness (Part VI). And `solution/` — the RED stub that makes the repository start honest and the GREEN solution behind `demo:red / demo:green`, the pair of scripts that let the repo travel between its two canonical states on command (Part XII will lean on them as the reset and the fallback).

Add the two scripts the metrics stack ships with — `scripts/contract_coverage.sh` and `scripts/dora_dashboard.py`, met in Part XI — and the inventory is complete. Nothing in the method lives outside the repository: the specs are files, the gates are files, the enforcement is a file, and the demo’s determinism is two

npm scripts. Which is the quiet thesis of this part: a process you can `git clone` is a process you can trust, because every rule in it is inspectable, versioned, and — where it matters — enforced by something with an exit code.

Part X

Measure: DORA, the SDD Signals, and the Worked Dashboard

Chapter 33

DORA, Exactly as Measured Here

The walkthrough’s final stage opens by naming what metrics are *for* in this method:

Metrics close the loop from *practice* to *proof*.

Everything before this part was practice — gates, contracts, four-line greens. This part is proof: numbers a skeptic can audit, with definitions precise enough to audit them. The frame is **DORA — DevOps Research & Assessment — four measures, each with its exact definition here and the mechanism by which this pipeline moves it**, presented as a table worth reading column by column:

DORA metric	Measured here as	How this pipeline moves it	Instrument from
Lead time for changes	spec-approved → running in prod	≤1-day slices kill queueing; contracts unblock parallel FE/BE/QA (PLT-4821: 4.2 days median spec→verified)	board timestamps (Spec col → Done)
Deployment frequency	deploys/week	small batches behind flags → daily merges of verified slices	CI/CD pipeline events

DORA metric	Measured here as	How this pipeline moves it	Instrument from
Change failure rate	% deploys causing incident/rollback	contracts catch regressions pre-prod ; Red never ships as “known issue”	incident tracker ÷ deploys
MTTR (mean time to restore)	failure → restored	thin slices = thin rollbacks; a red contract test <i>names the folder</i> (localize → reproduce → bisect)	incident timestamps

Three disciplines are embedded in the table’s very shape. **The “measured here as” column** pins each metric to a local, checkable definition — lead time is *spec-approved to running in prod*, not “commit to deploy” or any other variant, so no two people on the team can mean different things by the same word. **The “how this pipeline moves it” column** is the causal claim, stated before the numbers arrive: slicing kills queueing (lead time); small flagged batches enable frequent merges (frequency); contracts catch regressions pre-prod and *Red never ships as “known issue”* (CFR — the verification verdict from Part VIII, now visible as a failure-rate mechanism); thin slices roll back thin, and *a red contract test names the folder* (MTTR — Part III’s one-folder guarantee, priced). **The “instrument from” column** commits, in advance, to where each number will come from — board timestamps, CI events, the incident tracker — so that when the dashboard later shows movement, nobody can relitigate the measurement.

Chapter 34

The Two SDD-Native Signals

DORA measures delivery outcomes; it cannot see *why*. The pipeline adds two leading indicators of its own — **the two SDD-native signals (leading indicators DORA can't see)** — and both fall out of artifacts the pipeline already produces:

First-pass verification = tasks conforming without rework ÷ total.

Sprint 14: **81 %**, target 90. Falls out of §8's boolean — zero extra tooling. When it dips, the retro reads *which spec section* failed.

The boolean from Part VIII's verification anatomy, aggregated. Eighty-one percent of Sprint 14's tasks conformed on the first pass; the target is ninety. Two properties make it precious. It is *free* — the fact is recorded the moment a verdict is written. And it is *diagnostic* — because a first-pass failure almost always traces to an ambiguity that survived spec review, a dip is not a morale event but a reading assignment: *the retro reads which spec section failed*.

Contract coverage = EARS criteria with executable tests ÷ total.

At Sprint-14 close: **4/4 in-scope** (EARS-5 consciously excluded). The Sprint-15 amendment (EARS-6/7) and the new wallet-credit spec enter *uncovered* — coverage dips at every sprint boundary and recovers as their tasks land: the dip is the signal working. Drift downward = spec rot, detected before the incident detects it for you.

The countability of EARS IDs (Part II) cashed in. At Sprint-14's close, coverage is **4/4 in-scope** — and the denominator's honesty matters: EARS-5 is *consciously excluded*, a dash in the verification table, not a gap. Then the time-series behavior every adopter must be pre-warned about: when Sprint

15 amends the spec (EARS-6/7) and a brand-new wallet-credit spec arrives (EARS-101...104), those criteria enter the count *uncovered* — their contracts land with their tasks, later. Coverage *dips at every sprint boundary and recovers as tasks land*, and the walkthrough's verdict is the aphorism to tattoo on the dashboard: **the dip is the signal working**. A coverage line that never dips is a line measuring nothing — it means either no new requirements or new requirements entering untracked. What the signal exists to catch is not the healthy sawtooth but *drift downward — spec rot, detected before the incident detects it for you*.

Chapter 35

Beyond DORA: SPACE, DevEx, and GSM

One instrumentation pass feeds more than DORA. The walkthrough works three further frameworks *with the same sprint*, and the point of showing all three is the closing sentence — they share instruments, not just a philosophy.

35.1 SPACE — sample at least three dimensions, one perceptual

SPACE — sample ≥ 3 dimensions, always including one perceptual (survey) measure. ✓ marks the three this team samples:

Dim.	Question it answers	Worked example (Sprint 14)	Instrument	Sampled
S · Satisfaction & well-being	Do people feel good about how work works?	Spec-quality pulse: 4.2 / 5 (“criteria were clear before I started”)	5-question pulse	✓ (perceptual)
P · Performance	Outcomes, not effort	First-pass verification: 81 %	verification.md	optional
A · Activity	Output volume (context, never a target)	20 verified tasks	board count	✓

35.2. DevEx — pair every perception with a workflow measure

Dim.	Question it answers	Worked example (Sprint 14)	Instrument	Sampled
C · Communication & collaboration	Does knowledge flow?	Specs co-authored PO + engineer: 6 / 6	spec review log	optional
E · Efficiency & flow	How much waiting between steps?	Wait Contract → Red: 3 h median	board timestamps	✓

The framework’s rule is in its first line: sample **at least three** dimensions, and always include **one perceptual** measure — numbers about systems must be accompanied by at least one number about how the work *feels*, gathered by survey. This team samples S, A, and E: the spec-quality pulse (4.2/5 on “*criteria were clear before I started*” — a perceptual echo of the first-pass metric), verified-task count (with the table’s own guardrail attached: *context, never a target*), and the Contract→Red wait (3 hours median — how long a finished contract waits for its failing test). P and C are marked optional because their numbers already exist elsewhere: first-pass lives in *verification.md*, and co-authorship (6/6 specs written PO-plus-engineer) lives in the spec review log — Part II’s authorship discipline, now a collaboration signal.

35.2 DevEx — pair every perception with a workflow measure

DevEx — pair every *perception* with a *workflow* measure; feelings are data too:

Driver	What it captures	Perception (pulse)	Workflow (system)
Feedback loops	How fast the system answers you	“I get test/review results quickly” — 4.5 / 5	suite 2 m 40 s · review turnaround 4 h
Cognitive load	How hard it is to think here	“It’s easy to know exactly what to build” — 4.1 / 5	new dev → first verified task: 3 days
Flow state	How often deep work survives the calendar	“I get uninterrupted focus time” — 3.9 / 5	WIP = 1 → 0 mid-task handoffs

The pairing rule is the framework: *feelings are data too*, but never alone. Each driver gets a pulse question **and** a system measurement, and the pairs are chosen to check each other. Feedback loops: *people say* results come quickly (4.5/5), and the suite *runs* in 2 minutes 40 seconds with review turning around in 4 hours. Cognitive load: “*easy to know exactly what to build*” scores 4.1/5 — the spec stage’s entire purpose, perceived — and the system corroborates: a new developer reaches their first *verified* task in 3 days. Flow: 3.9/5 on uninterrupted focus, alongside the structural facts that produce it — work-in-progress limited to one, zero mid-task handoffs. When a pair diverges — pleasant feelings over ugly workflow numbers, or vice versa — the divergence itself is the finding.

35.3 GSM — Goal, Signal, Metric, strictly in that order

GSM — Goal → Signal → Metric, strictly in that order; two worked chains:

Step	Rule	Chain 1 — spec quality	Chain 2 — review bottleneck	Goodhart trap avoided
Goal	In words; no numbers yet	Refunds ship with zero rework caused by ambiguity	Verification never starves the pipeline	starting from “what’s easy to count”
Signal	How you’d <i>know</i> , even if unmeasurable	Verified work rarely reopens	Finished work rarely waits on review	mistaking activity for the goal
Metric	A measurable <i>proxy</i> , held loosely	First-pass 81 % (→ 90) · reopened: 1	Wait Green → Verify: 4 h median (< 1 day)	optimizing the proxy (rushed reviews)

GSM’s discipline is the *ordering*: goal in words with no numbers; then the signal — how you would know, even if you couldn’t measure; only then a metric, explicitly *a proxy, held loosely*. The two worked chains are the pipeline’s own anxieties formalized: spec quality (goal: refunds ship with zero ambiguity-caused rework; metric: first-pass 81% trending to 90, with reopened count 1)

and the review bottleneck Part IV predicted when it booked verification bandwidth first (goal: *verification never starves the pipeline*; metric: Green→Verify wait, 4 hours median, kept under a day). And the last column names the trap each row exists to dodge — starting from what’s easy to count, mistaking activity for the goal, optimizing the proxy with rushed reviews — Goodhart’s law walked, row by row, instead of merely cited.

The synthesis sentence is the operational payoff of all three frameworks:

One instrumentation pass — the board, `verification.md`, and the five-question pulse — feeds all three frameworks *and* DORA: watch the pipeline, watch the humans, keep the “why” honest.

Chapter 36

Can Jira Track These? The Split

Before the numbers, the plumbing question every adopter asks, answered with a precondition and a table. The precondition first:

Precondition for everything: the Jira workflow statuses must mirror the pipeline columns (*Spec* → *Contract* → *In Progress* → *Green* → *Verify* → *Done/Released*), because the issue **changelog timestamps every transition** — and those timestamps are the metrics.

The changelog *is* the instrument; the workflow must therefore speak the pipeline’s language, status for status (Part XI performs the configuration). Then the split, metric by metric:

Metric	In Jira?	How (Data Center)
Lead time	Yes — best source	changelog: <i>Spec approved</i> → <i>Released</i> ; Kanban Control Chart out of the box, or a nightly <code>/rest/api/2/search?expand=changelog</code> job / Time-in-Status app
First-pass verification	Yes — easiest	Automation rule: transition <i>Verify</i> → <i>In Progress</i> adds label <i>rework</i> ; then status = <i>Done</i> AND labels != <i>rework</i> ÷ all <i>Done</i>

Metric	In Jira?	How (Data Center)
MTTR	Yes, if incidents are issues	Incident issue type; created → resolved (or a <i>Restored</i> status) via resolution reports/JQL
Change failure rate	Partially	numerator: incidents linked to the causing release/story (linking discipline is the whole game); denominator (deploys) must be mirrored in CD job calls Jira REST on each prod deploy (transition to <i>Released</i> / deployed label), or GitLab-for-Jira feeds the development panel; counting stays truthful in GitLab
Deployment frequency	Mirror from CI	a CI step counts EARS-n IDs in specs vs. test annotations and posts to a dashboard gadget; source of truth stays in git
Contract coverage	No — CI owns it	

The answers grade from *yes, best source* to *no, CI owns it*, and the grading is itself the architecture:

Working architecture, matching “instrument once”: **Jira = work-flow clock** (lead time, first-pass, MTTR) · **GitLab CI = delivery clock** (frequency, coverage) · incidents-as-issues bridge the two for CFR · one nightly REST script joins them into the dashboard.

Two clocks, one bridge, one join. And the caveat the walkthrough says *to say out loud*, because it is the failure mode of every Jira-based measurement program:

Jira only measures what people actually transition — batch-moving tickets on Friday turns lead time into fiction, which is exactly why an SDD board helps: status is *test status*, driven by the pipeline’s reality, not by memory.

Followed immediately by the operating rules — stated here in the walkthrough and repeated verbatim in the setup guide because *they bear repeating*: capture the **baseline before the pilot**; re-measure after two sprints; judge

trends per team — never absolutes across teams; and never at individual level — *Goodhart's law: the fastest way to turn a measurement program into a gaming program.* One instrumentation pass — the board, CI events, `verification.md`, and a 5-question pulse — feeds DORA today and SPACE/DevEx/GSM tomorrow.

Chapter 37

The Worked Dashboard: Two Sprints, Six Numbers

Now the numbers themselves — the walkthrough’s worked example, with its methodology stated first because the methodology is what makes the numbers arguable-with:

Baseline captured over Sprints 11-12 (pre-pilot, same team of five); re-measured over Sprints 13-14 (the pilot). Same board, same CI, same incident tracker — only the practice changed.

Same team, same instruments, before and after; the only changed variable is the practice. Then the table:

Metric	Baseline · S11-12	SDD pilot · S13-14	Δ
Lead time (spec-approved → prod, median)	9.0 days	4.2 days	−53 %
Deployment frequency	2 / week	8 / week (16 deploys in 10 days, behind flags)	×4
Change failure rate	18 % (7 incidents / 39 deploys)	6 % (1 / 16)	−12 pp
MTTR	~6 h (multi-slice rollbacks, log archaeology)	40 min (flag-off + red test named the folder)	−89 %
<i>First-pass verification</i> (SDD)	— (not measurable pre-pilot)	74 % → 81 % (S13 → S14; target 90)	+7 pp

Metric	Baseline · S11-12	SDD pilot · S13-14	Δ
<i>Contract coverage</i> (SDD)	0 %	93 % (38 / 41 in-scope EARS criteria)	+93 pp

And, in the walkthrough’s words, *where each number came from* — *no new tooling, just the artifacts this repo already produces*:

- **Lead time:** board timestamps, Spec approved → Verified in prod. The five verified Sprint-14 stories took 3.1 · 3.8 · **4.2** · 4.6 · 5.0 days — median 4.2 (PLT-4830, rolled, is excluded from the median and *visible* as a roll, not hidden inside an average).
- **Deployment frequency:** CI deploy events. Twenty verified tasks merged behind flags produced 16 production deploys in 10 working days.
- **CFR:** incident tracker ÷ deploys. The one pilot incident was a flag mis-configuration — notably *not* a contract regression; the contracts caught those pre-merge (PLT-4830’s rounding gap cost 2 h, not an incident).
- **MTTR:** flag-off restored service in 40 minutes; the failing contract test pointed at src/refunds/ before the call ended — localize → reproduce → bisect, as designed.

The provenance bullets are half the argument. Every headline number arrives with its raw observations (five story durations, listed; 7/39 and 1/16, shown), its exclusions declared (PLT-4830 rolled — excluded from the median *and visible as a roll*), and its instrument named. A dashboard that shows its arithmetic invites audit; a dashboard that survives audit ends arguments.

Chapter 38

Reading the Table

The walkthrough then does something rarer than publishing numbers: it teaches how to read them, starting with the direction rule —

Did they improve? Yes — all six. “Better” points in different directions, so read the arrows with this rule:

Direction of good	Metrics	What happened	Verdict
Lower is better	Lead time · Change failure rate · MTTR	9.0→4.2 d · 18→6 % · 6 h→40 min — all down	✓ improved
Higher is better	Deployment frequency · First-pass verification · Contract coverage	2→8 /wk · 74→81 % · 0→93 % — all up	✓ improved

— and then the observation that carries more evidential weight than any single delta:

The strongest signal is the pair that normally *fights*: deploying more often usually means breaking more often. Here frequency rose ×4 **while** failure rate fell — improving both sides of that trade-off at once is what tells you the process, not luck, did it.

Then each metric gets its deep read; these six paragraphs are the dashboard’s user manual, and they repay near-verbatim study.

Lead time (median), 9.0 → 4.2 days (–53 %). *Clock time from spec approved to running in production — the full journey of one slice, including all*

the waiting. It's a **median** on purpose: half of all items now arrive within 4.2 days, and one disaster story can't hide inside an average. And the causal honesty: *the mechanism behind the drop is not faster typing — it's less queueing, because one-day slices don't wait behind each other.*

Deployment frequency, 2 → 8 / week (×4). Sixteen deploys in ten working days, *behind flags* — and the qualifier matters: *each deploy is dark until the flag flips, so shipping often ≠ exposing users often.* Then the reframe that turns a vanity count into a diagnostic: *frequency is really a proxy for **batch size*** — you can only deploy eight times a week if changes are small, and small changes are the easy ones to review, test, and undo.

Change failure rate, 18 % → 6 % (−12 pp). Of all deploys, the fraction causing an incident or rollback: 7/39 before, 1/16 after. The unit gets its own tutorial — **pp = percentage points**, *the absolute gap between two percentages (18 − 6 = 12), written that way because “−12 %” would ambiguously suggest a relative drop.* And the qualitative read: *the shape of the remaining failure matters too* — a flag misconfig, not a contract regression; *that class died pre-merge (2 h, not an outage).*

MTTR, ~6 h → 40 min (−89 %). Mean Time To Restore, detection to healthy. *Forty minutes is what “designed for debugging” buys — flag-off restores instantly, and the red contract test names the folder.* And the philosophical payload: *MTTR reframes risk: the goal isn't never failing; it's making failure cheap.*

First-pass verification, 74 → 81 % (target 90). Seventeen of twenty-one tasks right the first time — *the purest spec-quality signal there is: a first-pass failure almost always traces to an ambiguity that survived spec review.* No baseline exists *because the gate itself didn't exist before the pilot* — an honest dash where a lesser dashboard would have invented a zero.

Contract coverage, 0 → 93 %. Thirty-eight of forty-one in-scope criteria have an executable test bound to them — and *the missing three are named, not forgotten.* The reading rule: *watch the **direction** more than the level — drifting down means specs are outrunning tests: the early smell of spec rot.*

Chapter 39

Reading It Honestly

Two closing moves keep the dashboard credible, and both are quotable in full. First, the meta-observation the walkthrough saves *for the Q&A*:

All six move for the *same underlying reason* — smaller, unambiguous, verified slices — which is what makes them credible together. If only one had moved, be suspicious.

Coherence as evidence: six independent instruments, two directions of good, one mechanism. A single improved number can be gamed or lucked into; six, moving together, for a stated shared cause, is a process signature.

And then the paragraph that inoculates the whole exercise against its own success:

Read it honestly: two sprints is a **direction, not a proof** — the deltas are large partly because the baseline was soft. Keep the same instruments running, report the trend per team every sprint, and let first-pass verification climb toward 90 % before declaring victory. The dashboard's job is to make the next argument about data instead of vibes.

A direction, not a proof. The deltas are large partly because the baseline was soft; the remedy is not celebration but continuation — same instruments, per-team trend every sprint, victory deferred until first-pass reaches its target. The dashboard's job, stated in its last line, is the whole part in one sentence: **make the next argument about data instead of vibes.** How to wire that dashboard so it computes itself — two hours, once, per project — is Part XI.

Part XI

Instrumentation: Two Hours to a Self-Computing Dashboard

Chapter 40

The Goal and the Budget

The setup guide — `METRICS-SETUP.md`, reproduced in full as Appendix I — opens by stating exactly what “done” looks like and what it costs:

Goal: after this setup, the six-metric dashboard from `WALK-THROUGH.md` §9 computes itself — **Jira is the workflow clock** (lead time, first-pass verification, MTTR, and the incident side of CFR), **GitLab CI is the delivery clock** (deployment frequency, contract coverage, and the deploy side of CFR), and one scheduled script joins them.

Time budget: ~2 hours once, per project. Everything below uses only built-in features, Automation for Jira, and two scripts that ship in this repo (`scripts/contract_coverage.sh`, `scripts/dora_dashboard.py`).

Read the design commitments out of those two paragraphs. *Computes itself* — after setup, no human compiles the dashboard; a Monday-morning schedule does. *Two clocks* — the split from Part X, now an assignment of every metric to its owning system, with change-failure rate deliberately straddling both (incidents in Jira, deploys in GitLab). *Two hours, once, per project* — the affordability claim, kept honest by the constraint that follows it: **only built-in features**, Jira’s bundled Automation, and two repo-shipped scripts. No marketplace apps, no data warehouse, no new tooling — the same discipline the worked dashboard bragged about (*no new tooling, just the artifacts this repo already produces*), applied to its own installation.

The guide targets Jira **Data Center** and **self-managed GitLab**, with Cloud differences noted where they exist. The steps below follow its lettering —

Part A for Jira (A1–A6), Part B for GitLab (B0–B4), Part C for sanity checks and rules — so this chapter and the appendix can be read side by side.

Chapter 41

Part A — Jira as the Workflow Clock

41.1 A1 · Workflow statuses that mirror the pipeline

The guide marks this step “(*prerequisite for everything*)” and gives the reason in one line:

The changelog timestamps every status transition — those timestamps **are** the metrics. So the workflow must speak the pipeline’s language.

Three configuration moves:

Statuses. Under *Admin* → *Issues* → *Statuses*, create Spec, Spec Approved, Contract, Green, Verify, Released — In Progress and Done already exist. Categories matter for Jira’s reporting: Spec*/Contract map to *To Do*; In Progress/Green/Verify to *In Progress*; Released/Done to *Done*.

Workflow. In the project’s workflow designer, order them:

Spec → Spec Approved → Contract → In Progress → Green → Verify → Done → Released

with two deliberate edges: **allow Verify → In Progress** — *the rework path* — *we want it visible*, because that visible edge is what the first-pass automation will watch — and **a global transition to Released**, because CI, not a human, will drive that one (§B2).

Board columns. Map one status per column so the board reads exactly like the deck’s slide 19: *Spec · Contract · In Progress (Red) · Green · Verify · Done*.

Note the board's own vocabulary lesson: the In-Progress column is subtitled **(Red)** — on this board, being in progress *means* having a failing test, because status is test status.

And then the boxed warning the guide wants said in the team kickoff, since no configuration can enforce it:

Discipline note: Jira measures what people transition. If tickets get batch-moved on Friday, lead time is fiction. The SDD board resists this because status = test status — but say it out loud in the team kickoff.

41.2 A2 · Lead time — nothing to install

Two reads, one instrument:

- **Quick read:** Board → **Reports** → **Control Chart**, columns set from Spec Approved to Released — *the median line is your lead time; watch the trend, not single points.*
- **Exact numbers:** the nightly script (§C1) reads `/rest/api/2/search?expand=changelog` and computes the median from the **first entry into Spec Approved** to the **first entry into Released** — “first entry,” so a ticket that bounces through rework doesn't restart its own clock.

41.3 A3 · First-pass verification — the rework label rule

One Automation rule makes rework self-recording:

1. **Project settings** → **Automation** → **Create rule.**
2. **Trigger:** Issue transitioned · From Verify · To In Progress.
3. **Action:** Edit issue → Labels → **add** rework.
4. Name it SDD · mark rework, scope: this project. Done.

Four steps, and Part VIII's boolean becomes infrastructure: the visible rework edge from A1 now stamps every task that fails verification, with no one remembering anything. Reading the ratio takes two saved filters — dashboard gadgets or the script:

```
-- all finished
project = PLT AND issuetype in (Story, Task) AND statusCategory = Done AND resolved >= -14d
-- finished first-pass
project = PLT AND issuetype in (Story, Task) AND statusCategory = Done AND resolved >= -14d
  AND (labels IS EMPTY OR labels != rework)
```

First-pass % = second count ÷ first count. Note the JQL’s one subtlety, easy to get wrong when reinventing it: the numerator’s label clause is `(labels IS EMPTY OR labels != rework)` — an issue with *no labels at all* must count as first-pass, which a bare `labels != rework` would silently exclude.

41.4 A4 · Incidents — the CFR numerator and MTTR

1. **Issue type:** add Incident (standard type) to the project’s scheme. Workflow can be a simple Open → Restored → Done — *or keep the default and treat resolution as restoration.*
2. **The team rule the guide calls “the whole game”:** every production incident becomes an Incident issue **at detection time**, and gets an issue link **caused by** → **<the story or the Deployment issue>** (add the causes / caused by link type if missing). The link is what turns a pile of incidents into a change-failure *rate*: numerator meets denominator through it.
3. **MTTR** = `mean(resolutiondate – created)` over Incidents — the script computes it; the built-in *Average Resolution Time* gadget approximates it.

41.5 A5 · Deployments in Jira — the mirrored denominator

Jira DC has no native deployments feed (that’s a Cloud feature), *so CI puts them there* — two complementary conventions, both driven from GitLab in §B2:

- **One Deployment issue per production deploy** (add the Deployment issue type). Counting them per week = deployment frequency *inside Jira*; linking incidents to them gives clean CFR.
- **Transition shipped stories to Released** — *so lead time ends at the truth*, at the deploy, not at a human’s memory of one.

41.6 A6 · The dashboard, and the Cloud differences

Assemble a Jira dashboard from what now exists: the Control Chart link, the two first-pass filter counters, *Created vs Resolved* for Incidents — plus the script’s weekly table (§C1) *posted wherever the team reads (Slack/Confluence/CI artifact)*. The last clause is its own operating principle: the dashboard lives where the audience already is.

Jira Cloud differences: Automation is built-in the same way; deployments can come natively from the GitLab for Jira Cloud app (development panel + deployments), so §A5’s Deployment issues become optional; JQL identical.

Chapter 42

Part B — GitLab as the Delivery Clock

42.1 B0 • Connect GitLab ↔ Jira

GitLab project → **Settings** → **Integrations** → **Jira issues**: Web URL `https://jira.yourcompany.com`, auth = Jira **PAT (Bearer)**. From then on, any commit, MR, or branch containing PLT-123 appears in the Jira issue’s development panel — and **smart commits** work:

```
git commit -m "PLT-4821 verify green #transition Released"
```

One commit message that references the story, records the act, and drives the workflow — the developer’s keyboard becomes a Jira client.

42.2 B1 • CI variables

GitLab project → **Settings** → **CI/CD** → **Variables**, *masked*, *protected*:

Variable	Value
JIRA_BASE_URL	<code>https://jira.yourcompany.com</code>
JIRA_TOKEN	a service-account PAT (not a personal one)
JIRA_PROJECT	PLT

The parenthesis is a small governance decision worth honoring: the token that writes to Jira belongs to a *service account*, so the audit trail says “CI did this,” and no one’s departure breaks the pipeline.

42.3 B2 · The deploy job — tell Jira

Append to `.gitlab-ci.yml`; it *runs after every **production** deploy* and performs both A5 conventions in one `.post` job. Reproduced verbatim from the guide (with one over-long line soft-wrapped inside its quoted string; semantics unchanged — see the appendix note):

```
notify-jira:
  stage: .post
  rules:
    - if: '$CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH && $CI_ENVIRONMENT_NAME == "production"'
  script:
    # 1 · one Deployment issue per prod deploy (the CFR denominator)
    - |
      curl -sfS -X POST "$JIRA_BASE_URL/rest/api/2/issue" \
        -H "Authorization: Bearer $JIRA_TOKEN" -H "Content-Type: application/json" \
        -d '{"fields":{"project":{"key":"$JIRA_PROJECT"},
          "issuetype":{"name":"Deployment"},
          "summary":"deploy $CI_COMMIT_SHORT_SHA · $CI_PIPELINE_URL",
          "labels":["env:prod"]}}'

    # 2 · transition every story shipped in this deploy to Released
    - |
      for KEY in $(git log "$CI_COMMIT_BEFORE_SHA..$CI_COMMIT_SHA" --pretty=%s%b \
        | grep -oE "$JIRA_PROJECT-[0-9]+" | sort -u); do
        TID=$(curl -sfS -H "Authorization: Bearer $JIRA_TOKEN" \
          "$JIRA_BASE_URL/rest/api/2/issue/$KEY/transitions" \
          | python3 -c 'import json,sys;
ts=json.load(sys.stdin)["transitions"];
print(next((t["id"] for t in ts if t["to"]["name"]=="Released"), ""))'
        [ -n "$TID" ] && curl -sfS -X POST \
          "$JIRA_BASE_URL/rest/api/2/issue/$KEY/transitions" \
          -H "Authorization: Bearer $JIRA_TOKEN" -H "Content-Type: application/json" \
          -d '{"transition":{"id":"$TID"}}' || true
      done
```

Walk it once, because every line implements a decision from earlier chapters. The **rules** clause fires only on the default branch deploying to the production environment — production deploys and nothing else, so the frequency count stays honest. **Step 1** creates the Deployment issue: summary carrying the short SHA and a link back to the pipeline, label `env:prod` — the CFR denominator, born at the moment of deploy. **Step 2** finds every story shipped in this deploy by grepping `PLT-[0-9]+` out of the commit subjects and bodies between `CI_COMMIT_BEFORE_SHA` and `CI_COMMIT_SHA` — the 1 : 1 : 1 discipline paying an

infrastructure dividend: because every task is one ticket and commits cite their ticket, *the git log is a manifest*. For each key, it fetches the issue's available transitions, picks the one whose destination is Released (the small embedded Python — soft-wrapped above — does the lookup, returning empty if the transition isn't available), and drives it. The trailing `|| true` keeps one untransitionable issue from failing the deploy: telling Jira is important; blocking production on Jira is not.

And the guide's closing observation on this job is a design feature disguised as redundancy:

Deployment frequency now exists in *both* systems: GitLab (pipelines on the production environment — Deployments page / API) and Jira (Deployment issues) — they must agree, which is itself a useful health check.

42.4 B3 · The contract-coverage job — the CI-owned SDD signal

```
contract-coverage:
  stage: test
  script:
    - scripts/contract_coverage.sh "${COVERAGE_MIN:-0}"
```

The script counts EARS-*n* IDs declared in `specs/*/spec.md` against IDs referenced by any `*.feature` or `*.spec.ts` — the repo's convention being generous by design: *a header line like Covers EARS-2 and EARS-3 or a test title (EARS-1) both count*. On the repo as it stands, the guide reports, it prints — *truthfully*:

```
contract coverage: 3/11 criteria (27%)
uncovered: EARS-4 EARS-5 EARS-6 EARS-7 EARS-101 EARS-102 EARS-103 EARS-104
```

Twenty-seven percent — and the guide immediately performs the reading lesson this book has been building toward, uncovered ID by uncovered ID: *EARS-4 lands with T6; EARS-5 is superseded by Amendment A1; EARS-6/7 arrived with the Sprint-15 amendment and EARS-101...104 with the brand-new wallet-credit spec — their contracts land with their tasks*. Every uncovered criterion has a name and a reason: one awaits its scheduled slice, one has been superseded by a formal amendment, and eight are the leading edge of new intent whose tests are, correctly, not written yet. Hence the boldfaced

law and its corollary:

Coverage dips at every sprint boundary; the dip is the signal working. Watch it recover as tasks land. Set `COVERAGE_MIN` once the pilot stabilizes to make drift a red pipeline instead of a surprise.

The threshold's default of 0 is the pilot posture — observe, don't enforce. The graduation move is raising `COVERAGE_MIN` once the sawtooth's floor is known, at which point spec rot stops being a chart you might notice and becomes *a red pipeline*.

42.5 B4 · The weekly dashboard — a scheduled pipeline

```
dora-dashboard:
  stage: .post
  rules: [{ if: '$CI_PIPELINE_SOURCE == "schedule"' }]
  script:
    - python3 scripts/dora_dashboard.py --project "$JIRA_PROJECT" --days 14 | tee dora.txt
  artifacts: { paths: [dora.txt], expire_in: 90 days }
```

Then GitLab → **Build** → **Pipeline schedules** → **New**, cron `0 6 * * 1` — Monday mornings. The script is *stdlib-only*, *PAT auth*, and prints the table: lead-time median from the changelog, deployments/week from Deployment issues, CFR = incidents ÷ deployments, MTTR from incident resolution times, first-pass from the rework label — *contract coverage it defers to §B3's job*, honoring the source-of-truth split to the end. The `--days 14` window is the two-sprint cadence the operating rules keep invoking; `expire_in: 90 days` keeps a rolling quarter of dashboards as CI artifacts — the trend line, self-archiving.

Chapter 43

Part C — Sanity Checks, Operating Rules, and the Source-of-Truth Table

C1 • Dry-run the script from any machine that can reach Jira:

```
export JIRA_BASE_URL=https://jira.yourcompany.com JIRA_TOKEN=<PAT>
python3 scripts/dora_dashboard.py --project PLT --days 14
```

C2 • Verify the plumbing, one item each: move a test issue Verify → In Progress and confirm the rework label appears; run one production deploy and confirm a Deployment issue plus a Released transition; open the Control Chart and confirm the Spec Approved → Released span. Three checks, one per subsystem — the instrument gets its own verification stage, which is only consistent.

C3 • Operating rules — *“from the walkthrough, they bear repeating,”* and this book repeats them a final time because the guide does: capture the **baseline before the pilot**; re-measure after two sprints; judge **trends per team, never absolutes across teams**; never at individual level (Goodhart’s law); and when a number moves, ask *which spec section or pipeline stage* moved it — **that’s the retro’s job**.

The guide closes with the table that settles, in advance, every future argument about whose number is right:

Metric	Source of truth	Where configured
Lead time	Jira changelog	\$A1-A2

Chapter 43. Part C — Sanity Checks, Operating Rules, and the Source-of-Truth Table

Metric	Source of truth	Where configured
Deployment frequency	GitLab prod pipelines (mirrored to Jira)	\$B2 (\$A5)
Change failure rate	Jira Incidents ÷ Deployment issues	\$A4–A5, \$B2
MTTR	Jira Incident timestamps	\$A4
First-pass verification	Jira rework label	\$A3
Contract coverage	git + CI	\$B3

Six metrics, six owners, six pointers into the configuration that produced them.
And the sign-off, four words that are the whole part:

Instrument once. Argue with data.

Part XII

The Twelve-Minute Demo

Chapter 44

What the Demo Is, and Its One Design Law

The demo runbook — `DEMO.md`, reproduced in full as Appendix J — describes itself with precision: it is the *stage script only*, twelve minutes, while the walk-through remains *the fully reasoned, stage-by-stage method*. Its purpose statement is the pipeline in miniature:

This runbook turns the repo into a live, end-to-end demonstration of the whole pipeline: a deliberately vague Jira ticket becomes EARS criteria, the criteria become executable Gherkin and a failing Jest test, an agent (or you) drives red → green under a resource gate, and verification closes the loop.

And its one design law, stated in the same opening paragraph and honored in every section after: **every step has an offline fallback, so the demo cannot be killed by Wi-Fi**. A demo of a method about removing single points of failure must itself have no single point of failure; the runbook takes its own medicine, down to a closing motto that includes itself in the discipline — *Spec first. Verify always — including this demo*.

Chapter 45

Prerequisites: Ten Minutes, Once

The §0 table is the environment contract; here it is, verbatim:

Need	Check	Notes
Node.js ≥ 20	<code>node -v</code>	22 recommended
Claude Code	<code>claude --version</code>	<code>npm i -g @anthropic-ai/claude-code</code>
Model access	<code>claude</code> starts a session	Anthropic API key or Amazon Bedrock (CLAUDE_CODE_USE_BEDROCK=1 + AWS creds). If you keep a provider-toggle script, run it before the session and <i>never</i> switch mid-demo.
This repo	<code>unzip sdd-starter.zip && cd sdd-starter</code>	
Dependencies	<code>npm install</code>	installs jest, ts-jest, cucumber, ts-node
Sanity	<code>npm test</code> → RED on T3 · <code>npm run bdd</code> → scenario fails	red is the correct starting state
No-install check	<code>npm run typecheck:noinstall</code>	proves the skeleton compiles before any deps — nice opener line

Two rows deserve a presenter’s underline. The **sanity row** encodes the repository’s philosophy of health: you verify readiness by confirming the tests *fail* — RED on T3, scenario failing — because *red is the correct starting state*; a green repo before the demo means someone forgot to reset. And the **no-install**

check is both an engineering nicety and stagecraft: a zero-dependency type-check proves the skeleton compiles on a bare machine, and saying so is a *nice opener line*.

Then the stage direction most technical presenters skip and regret:

Terminal setup for stage: font $\geq$ 18 pt, dark theme, two panes — left for Claude Code, right for `npm test / npm run bdd`. Rehearse the pane switch.

Eighteen-point font so the back row reads EARS-2; two panes so the agent's conversation and the test results are visible simultaneously — the demo's argument *is* the correspondence between those panes; and the closing imperative, *rehearse the pane switch*, because the beat structure below turns on it repeatedly.

Chapter 46

The Repo, and the Two Modes

Section §1's inventory was mapped in Part IX; what remains is the runbook's fork in the road:

Two demo modes: - **From scratch (impressive):** delete specs/checkout-refunds/spec.md body sections and rebuild them live via /specify-from-jira PLT-4821. - **From the worked example (safe):** keep the spec, *walk* it, and spend the live minutes on the red → green loop. Recommended for the webinar.

The trade is stated in the parentheses. From-scratch maximizes spectacle — the room watches a spec materialize from the villain — at the cost of live-generation variance eating your clock. From-the-worked-example keeps the finished spec on screen, *walks* it, and spends the scarce live minutes where liveness matters most: the red→green loop under the gate. The runbook's own recommendation for a webinar is the safe mode, and the reasoning generalizes: put the live risk where the payoff is highest, and the worked example everywhere else.

Chapter 47

Configuring the Jira Source: Pick One

Section §2 offers three ways to put PLT-4821 in front of the pipeline — *pick ONE*, before the session, per the provider rule’s same logic.

A. Offline JSON — default, zero risk. Nothing to configure. `jira/PLT-4821.json` is a *faithful issue export: vague description, untestable AC, two half-decisions hiding in comments* — and `/specify-from-jira` reads it first. Deterministic, free, Wi-Fi-proof: the default for a reason.

B. Live Jira via REST (Data Center or Cloud).

```
export JIRA_BASE_URL=https://jira.yourcompany.com
export JIRA_TOKEN=<personal-access-token>      # DC: PAT bearer
scripts/jira-fetch.sh PLT-4821                # writes jira/PLT-4821.json
```

Jira Cloud instead: *replace the Bearer header in the script with `-u you@company.com:$JIRA_API_TOKEN`*. And the rule that keeps option B from becoming a live grenade: **fetch before the session and commit the JSON — never do live auth on stage**. Live-sourced, offline-presented: the freshness of B with the determinism of A.

C. Atlassian MCP — the agent reads Jira itself. The `.mcp.json` entry from Part IX; authenticate beforehand, confirm with `claude mcp list`, and the `specify` command pulls the issue live — with the bonus talking point (“*note the token cost of that server — slide 23 in action*”) and the teardown discipline (*remove the server after the demo; scope per repo, as preached*).

Chapter 48

The Six Beats

Twelve minutes, six beats, each with its command, its artifact, and its line. The script below is the runbook's, expanded with what each beat is *doing* to the room.

48.1 Beat 1 · The villain (1 min)

Open `jira/PLT-4821.json`. Read the AC out loud: *"refunds work for cancelled events."* Read the comment: *"fees = our service fees... I think."* Then the line this book met in Part I:

"Every word of ambiguity here will be typed into code at machine speed. Let's stop it now."

One minute, two quotations, one thesis. The beat's job is to make the audience *hear* the vagueness they normally skim — the untestable AC and the hedge — so that everything after registers as a response to a problem they just witnessed.

48.2 Beat 2 · Jira → EARS, the /specify moment (3 min)

In Claude Code:

```
/specify-from-jira PLT-4821
```

Watch it mine the description AND the comments; the *"...I think"* becomes an **open question**, not an assumption. Then the interactive core — steer

one criterion live, asking the room the interrogation from Part II: “*‘automatic refunds’ — WHEN exactly?*” — until it reads: WHEN an event is cancelled, the system shall refund the ticket price minus service fees within 5 business days. Point at the gate: */plan refuses while Open questions is non-empty*. Answer the fees question on the spot (*service fees, per the PO*) and watch the list empty.

The beat closes with the runbook’s own emphasis: **This conversation is the demo**. Not the artifact — the conversation: the room watching a hedge become a question, a question become a decision, and a refusal become an unlock. Three minutes that contain Parts I and II entire.

48.3 Beat 3 • DDD in one breath (1 min)

Open docs/ddd.md. One sentence — Part III’s elevator pitch, verbatim:

“Refunds is its own bounded context — the calculator lives in src/refunds/, downstream of Ordering’s frozen order snapshot — so when a refund test goes red, there is exactly one folder in the universe to look in.”

One breath by design: the context map earns its minute by promising the debugging payoff the audience will want later, and moves on.

48.4 Beat 4 • BDD — the shared dialect (2 min)

`npm run bdd`

Open specs/checkout-refunds/contracts/refunds.feature side by side: *the same EARS criterion as Given-When-Then that the PO can read — and it executes, and it fails, because nothing is implemented*. Then the refrain, delivered where it was born: **one oracle, two dialects** — *the Gherkin and the Jest spec bind to the same in-memory harness*. The pane discipline pays here: feature file on one side, red run on the other, and the audience sees readable prose *fail like a test*, which is the whole point of the dialect.

48.5 Beat 5 · TDD + the gate (4 min)

The longest beat, in three movements.

Red, by design:

```
npm test          # RED – refunds.service.spec.ts, by design
```

Prove the gate before using the agent. Set the leash — echo "checkout-refunds T3" > .claude/state/ACTIVE_TASK — then, *first*, ask Claude to edit `src/payments/infrastructure/hooks` answers with `exit 2`: outside T3's declared resources. And say the line: **"a hook is enforcement; a prompt is persuasion."** The order matters theatrically and epistemically: the audience watches the constraint *refuse* before they watch the agent *comply*, so the later compliance reads as enforced rather than lucky.

Then implement: /implement T3, letting the agent drive red → green against the contract — *it's ~4 lines: total.cents – fees.cents, banker's rounding via Money.roundBankers, per ADR-017* — and mention *the two-hour Sprint-14 story* while it lands. Close the beat with both dialects:

```
npm test && npm run bdd    # GREEN – both dialects agree
```

48.6 Beat 6 · Verify (1 min)

/verify checkout-refunds — *the EARS-2 row flips to Green in verification.md; the exit gate opens*. And the closing line, which is the demo's entire arc in eleven words:

"Spec on the ticket this morning; verified behavior now. That's the pipeline."

Chapter 49

Fallbacks and Reset — Do Not Skip

Section §4's title carries its own imperative, and its four bullets are a complete failure-mode analysis:

- **Reset before every run:** `npm run demo:red && npm test` (must be RED), `restore ACTIVE_TASK, git checkout -- specs/` if you edited the spec live. The reset is a *verified* reset — you don't trust `demo:red`; you run the test and confirm the redness — and it covers all three kinds of demo-time mutation: code, leash, and spec.
- **Agent slow or offline?** `npm run demo:green swaps in solution/refund-calculator.solution` — *narrate the diff instead of typing it. The audience still sees red → green.* The GREEN solution shipping in `solution/` exists precisely for this: the demonstration's payload is the transition, and the transition survives the agent's absence.
- **Cucumber install trouble on the venue machine?** Skip Beat 4's run; *show the .feature file and lean on the Jest suite — the story holds*, because one oracle means either dialect alone still testifies.
- **Total tooling failure?** *Slides 10–11 ARE the demo (criterion → contracts → failing test → task). Present them, promise the repo, move on.* The last-resort fallback is the recognition that the demo's content is the *chain* — criterion to contracts to failing test to task — and the chain survives translation to two slides.

Chapter 50

Troubleshooting

The §5 table, verbatim — five symptoms, five fixes, and one reframe:

Symptom	Fix
tsc deprecation error on moduleResolution	you’re on TS 6 globally — the repo pins TS 5.5 via npm install; use npm run typecheck
Hook doesn’t block	CLAUDE_PROJECT_DIR unset when testing manually — run via Claude Code, or export it
Cucumber “no steps”	run from repo root; cucumber.js resolves paths relative to it
Jest can’t find ts	npm install didn’t finish — rerun; typecheck:noinstall needs nothing
Agent edits refused everywhere	.claude/state/ACTIVE_TASK missing — that’s the gate working; set the task

The last row is the table’s philosophy lesson, met already in Part IX and worth its second appearance: universal refusal is not the gate broken but the gate *fail-closed* — the correct response to an undeclared task is to declare one, never to disable the hook.

Chapter 51

What Was Verified Where

The runbook ends by verifying itself, and the split is a model of honest engineering documentation:

Tested in a clean offline container: zero-dependency typecheck (stub **and** solution), the gate's four modes (allow in-scope, block out-of-scope, allow specs/docs, block without ACTIVE_TASK), demo:red/demo:green round-trip, JSON/YAML validity, cucumber config load. Requires npm install on your machine (not testable offline here): the jest run and the cucumber execution — both use pinned, current versions; run \$0's sanity checks the day before.

Everything that *could* be certified without dependencies was — both compile targets, all four gate modes, the red/green round-trip, the config files' validity. Everything that *couldn't* is named, with its mitigation: pinned versions, and \$0's sanity checks *the day before*. The runbook holds itself to the pipeline's evidentiary standard, claim by claim — which is what its sign-off means:

Spec first. Verify always — including this demo.

Part XIII

The Pilot: Making It a Team's Habit

Chapter 52

From One Feature to One Team

Everything to this point has the shape of a biography: one feature, PLT-4821, carried from villain to CONFORMS. This closing part assembles, from the materials' own operating rules and worked history, the playbook for the harder act — making the pipeline a *team's* practice, measured honestly enough that the decision to continue is made by the dashboard rather than by enthusiasm.

The playbook has five moves, and every one of them is a sentence you have already read, now given its operational weight: **baseline before the pilot; run two sprints; re-measure with the same instruments; let the retro read the spec; scale on trends, never absolutes, never individuals.**

Chapter 53

Move 1 — The Kickoff, and the One Thing to Say Out Loud

The instrumentation guide plants a specific sentence in the team kickoff, and it is the pilot's founding disclosure:

Discipline note: Jira measures what people transition. If tickets get batch-moved on Friday, lead time is fiction. The SDD board resists this because `status = test status` — but say it out loud in the team kickoff.

Say it out loud — because the measurement system's one unpatchable vulnerability is transition hygiene, and the mitigation is social: a team that has heard, on day one, that batch-moving turns its own lead time into fiction is a team that polices the habit itself. The structural help is real (`status = test status` — the board's columns are driven by the pipeline's reality, Red and Green and Verify, not by memory), but the guide is explicit that structure assists honesty rather than replacing it.

The kickoff also sets the two non-negotiables the pilot inherits from the method. **Open questions gate planning** — `/plan` refuses on a non-empty list, and the team agrees in advance that the refusal is the method working, not an obstacle to route around. **The gate stays on** — `ACTIVE_TASK`, the hook, exit 2; a pilot that disables its enforcement is piloting a suggestion, and its dashboard will be measuring the suggestion.

Chapter 54

Move 2 — Baseline Before the Pilot

The rule appears in both source documents in nearly the same words, which is the materials' way of saying it is load-bearing:

Capture the **baseline before the pilot** (this month), re-measure after two sprints...

The worked example shows what obeying it looks like: *baseline captured over Sprints 11-12 (pre-pilot, same team of five); re-measured over Sprints 13-14 (the pilot). Same board, same CI, same incident tracker — only the practice changed.* The controls are the credibility: same team, same instruments, one changed variable. A pilot without a baseline can end only in anecdotes; a pilot with one ends in a Δ column.

Practically, the baseline period is also when the instrument itself is installed and shaken down — the two hours of Part XI, plus the C2 plumbing checks: move a test issue Verify → In Progress and watch the rework label appear; run one production deploy and confirm the Deployment issue and the Released transition; open the Control Chart and confirm the Spec Approved → Released span. The instrument gets verified before the experiment it will judge — of course it does; that is the only order this method knows.

One honesty rule for the baseline, taken from the worked dashboard itself: some rows will be blank, and blank is correct. First-pass verification shows “— (not measurable pre-pilot)” because *the gate itself didn't exist before the pilot*; contract coverage starts at 0 %. Record the dash rather than inventing a zero-with-an-asterisk — the dashboard's authority later depends on its refusal

to fake precision now.

Chapter 55

Move 3 — Two Sprints, Expecting the Dip

The pilot runs **two sprints** — the re-measurement cadence both documents fix, and the window the scheduled dashboard's --days 14 mirrors. Two sprints is long enough for the six numbers to move and short enough that the team's patience isn't the experiment's first casualty.

Two expectations, set in advance, keep the middle of the pilot from being misread:

Expect the coverage sawtooth. The moment new intent arrives — a spec amendment, a brand-new spec — contract coverage *dips*, because criteria enter the count before their tasks land. The pilot team will watch its own version of the guide's 3/11 morning, uncovered IDs listed by name, and the pre-briefed reading is the aphorism: *coverage dips at every sprint boundary; the dip is the signal working. Watch it recover as tasks land.* The graduation move — setting `COVERAGE_MIN` so that drift becomes *a red pipeline instead of a surprise* — waits, per the guide, until *the pilot stabilizes*.

Expect verification to be the constraint. The capacity plan already said so (*1.5 days of review & verification bandwidth booked first — with agents typing, verification is the constraint*), and the GSM chain gave the goal its words: *verification never starves the pipeline*, watched via the Green→Verify wait (the pilot's median: 4 hours; the bound: under a day). If mid-pilot the wait balloons, the response is capacity, not gate-loosening.

Chapter 56

Move 4 — The Retro Reads the Spec

The materials assign the retrospective one specific job, stated identically in the walkthrough and the setup guide:

When a number moves, ask *which spec section or pipeline stage* moved it — that’s the retro’s job.

This is the discipline that turns a dashboard from a scoreboard into a steering wheel. A moved metric is a *question*, and the method’s traceability was built so the question has a findable answer:

- **First-pass verification dips?** *The retro reads which spec section failed* — the walkthrough’s own instruction. Each rework-labelled task points, via 1 : 1 : 1, at one criterion; the criterion points at the ambiguity that survived review; the fix is a better pattern, a sharper litmus reading, an earlier open question.
- **Lead time stalls?** The changelog says *where* — which column ate the days. A pile-up entering Verify is the constraint speaking; a pile-up in Spec Approved is planning starving the pipeline from the front.
- **Coverage isn’t recovering after the dip?** The uncovered list is named IDs; each ID either has a scheduled task, a supersession, or a problem — and the list makes the three cases distinguishable.
- **A change failure occurs?** Its caused by link walks from incident to deploy to story to criterion — and the *shape* of the failure gets read the way the pilot’s one incident was: a flag misconfiguration, *notably not a contract regression*, because that class died pre-merge.

And the verification stage's own rule already routes every gap the retro finds: *if Verify found a gap, the spec — not the definition of done — would have changed, and the loop tightens instead of drifting.* The retro is where the tightening is scheduled.

Chapter 57

Move 5 — Scale on Trends; Never Absolutes; Never Individuals

The judgment rules, from both documents, verbatim and non-negotiable:

Judge **trends per team — never absolutes across teams**, and never at individual level (Goodhart’s law: the fastest way to turn a measurement program into a gaming program).

Three fences, each protecting a different failure. **Trends, not levels:** a team is compared to its own past — the worked dashboard’s honest closer applies (*two sprints is a direction, not a proof — the deltas are large partly because the baseline was soft*), so the deliverable of a pilot is a slope, watched sprint over sprint with *the same instruments running*, not a trophy number. **Per team, never across teams:** absolute cross-team comparison imports every difference in domain, legacy, and luck as noise, and invites exactly the batch-moving fictions the kickoff warned about. **Never individuals:** the Goodhart clause is the materials’ bluntest sentence, and the GSM table already walked its traps row by row — start from goals, hold metrics loosely as proxies, and remember that the fastest way to get rushed reviews is to target the review-wait number at a person.

Scaling, then, is the pilot’s playbook re-run, not relaxed: the next team gets its own kickoff sentence, its own baseline, its own two sprints, its own retro discipline — and the first team keeps its instruments running, because *let first-pass verification climb toward 90 % before declaring victory* is a standing

order, not a pilot-only one.

Chapter 58

Closing: The Three-Line Method

Strip the book to what its sources refuse to compress any further, and three lines remain — the walkthrough’s sign-off, with the demo runbook’s amendment folded in.

Spec first. Kill the ambiguity while it costs a sentence: five patterns, a litmus test, numbers for non-functionals, an Out-list, open questions at zero, and a written record for every decision that would otherwise be re-litigated.

Verify always — *including this demo*. Red before green, so the detector is proven; two dialects on one oracle, so intent and edges agree; a criteria-coverage table with a dated verdict, so “done” is a fact; and the same standard applied to the runbooks, the reset, and the gate’s own four modes.

Measure honestly. Two clocks, six metrics, sources of truth declared in advance; baselines before pilots; medians that exclude the rolled story *visibly*; dips read as the signal working; trends per team, never individuals — so that the next argument is about data instead of vibes.

That is the whole machine. It fit in three source documents; it fits, expanded, in this handbook; and it fits, when you need it to, in twelve minutes on a stage. *Spec first. Verify always. Measure honestly.*

Part XIV

Appendices

Appendix A

Stage → Artifact → Command

The walkthrough's closing map of the entire pipeline — one row per stage, with the artifact each stage produces and the command or file that produces it:

Stage	Artifact	Command / file
Feature arrives	jira/PLT-4821.json	scripts/jira-fetch.sh (live)
Spec (EARS)	specs/.../spec.md	/specify-from-jira PLT-4821
DDD	plan.md, docs/ddd.md	/plan checkout-refunds
Tasks	tasks.md	/tasks checkout-refunds
Architecture	src/refunds/application/ports.ts,— refunds.service.ts, docs/code-structure.md	
BDD	contracts/refunds.feature, features/steps/	npm run bdd
TDD (red)	refunds.service.spec.ts	npm test
Implement	refund-calculator.ts	/implement T3 (demo:green fallback)
Verify	verification.md	/verify checkout-refunds
DORA	metrics table	board + CI + verification.md

Appendix B

The Spec Artifacts of checkout-refunds, Assembled

A single-reference view of the worked feature's specification content, gathered from the chapters.

B.1 The five EARS criteria

- **EARS-1 (Ubiquitous):** The refunds API shall return amounts in minor units (cents).
- **EARS-2 (Event):** WHEN an event is cancelled, the system shall refund the ticket price minus *service fees* within 5 business days.
- **EARS-3 (State):** WHILE a refund is PENDING or ISSUED, further refund requests for that order shall be rejected with 409.
- **EARS-4 (Unwanted):** IF the access token is expired, THEN the API shall return 401 with WWW-Authenticate.
- **EARS-5 (Optional):** WHERE partial refunds are enabled, the system shall support line-item refunds. → flagged out of this slice.

B.2 Non-functionals (as numbers)

p95 issuance $\leq$ 300 ms (excl. PSP); zero double refunds under retry.

B.3 Out-list

Notification wiring (→ T4); refund UI (→ T5); line-item refunds (behind flag, later).

B.4 Open questions (resolved to zero)

Which fees → PO confirms: service fees, from the order snapshot. Rounding on fractional fee math → banker's, half-to-even — logged as **ADR-017**.

B.5 Domain events

EventCancelled → RefundRequested → RefundCalculated → RefundIssued → BuyerNotified

B.6 Bounded contexts and mapping

Ordering owns orders and their frozen totals; Payments owns money movement; **Refunds owns refund policy** (the math, the state machine, the idempotency); Notification owns outbound messages. Refunds is *downstream-conformist* to Ordering (totals and service fees frozen on the order snapshot) and *upstream* to Notification (publishes `refund.issued v1`).

B.7 The task table

Task	From	Contract	Size	Why this slice
T1 · fixtures & harness	EARS-1	feature background	S	walking skeleton: gives every later test its verbs
T2 · endpoint skeleton	EARS-3	OpenAPI 201/409	S	contract-first: FE can mock <i>today</i> ; 409 path proven early
T3 · refund calculation	EARS-2	POST + “refund on cancellation” scenario	M	the domain heart — the demo’s live task

Task	From	Contract	Size	Why this slice
T4 · refund.issued event	(out-of-scope wiring)	event schema v1	S	isolates Notification coupling
T5 · status UI slice	scope §2	pact checkout-web ↔ orders-svc	M	consumer-driven contract pair
T6 · expired-token guard	EARS-4	OpenAPI 401	S	unwanted-behaviour pattern gets its own slice

Critical path $T1 \rightarrow T2 \rightarrow T3$; T4/T6 hang off it; T5 parallel via the Pact stub. $4 \times S + 2 \times M \leq 4$ dev-days; 1.5 days of review & verification bandwidth booked first.

B.8 The EARS → Gherkin translation

EARS	Gherkin (in <code>contracts/refunds.feature</code>)
EARS-2	<i>Given</i> a paid order for event “EVT-303” · <i>When</i> the event is cancelled · <i>Then</i> a refund is issued for price minus service fees · <i>And</i> the order status is “REFUNDED”
EARS-3	<i>Given</i> the order already has a refund · <i>When</i> a refund is requested again · <i>Then</i> the request is rejected with status 409

B.9 The failing test (TDD red)

```
it("refunds price minus fees when the event is cancelled", async () => {
  const order = await paidOrder("EVT-303");
  await events.cancel("EVT-303");
  expect(await refundFor(order.id)).toEqual(
    order.total.minus(order.fees) // RED – no implementation yet (T3)
  );
});
```

B.10 The implementation (green)

```
export class RefundCalculator {  
  calculate(order: Order): Money {  
    const cents = Money.roundBankers(order.total.cents - order.fees.cents);  
    return Money.ofCents(cents); // EARS-2 · minor units per EARS-1 · ADR-017  
  }  
}
```

B.11 verification.md anatomy

1. Criteria coverage table — every EARS row maps to named green tests; EARS-2 flips Red – T3 open → Green; EARS-5 stays – (out of slice, documented).
2. Per-task evidence — suite names, results, deviations → ADRs (T3 cites ADR-017).
3. Verdict — CONFORMS – <date> only when every in-scope row is green; one red row and the feature does not merge.
4. First-pass entry — conformed without rework, yes/no.

Appendix C

Configuration Reference (Verbatim)

Everything Part XI configures, in one place. Long lines are soft-wrapped exactly as in the chapters (semantics unchanged).

C.1 Jira workflow (A1)

Statuses to create: Spec, Spec Approved, Contract, Green, Verify, Released (plus existing In Progress, Done). Categories: Spec*/Contract → To Do; In Progress/Green/Verify → In Progress; Released/Done → Done.

Spec → Spec Approved → Contract → In Progress → Green → Verify → Done → Released

Allow Verify → In Progress (visible rework path) and a global transition to Released (CI-driven). Board columns, one status each: *Spec · Contract · In Progress (Red) · Green · Verify · Done*.

C.2 Automation rule (A3)

Name SDD · mark rework, scope: this project. Trigger: Issue transitioned, From Verify, To In Progress. Action: Edit issue → Labels → add rework.

C.3 First-pass JQL (A3)

```
-- all finished
project = PLT AND issuetype in (Story, Task) AND statusCategory = Done AND resolved >= -14d
```

```
-- finished first-pass
project = PLT AND issuetype in (Story, Task) AND statusCategory = Done AND resolved >= -14d
AND (labels IS EMPTY OR labels != rework)
```

First-pass % = second count ÷ first count.

C.4 Incidents & deployments (A4-A5)

Issue types Incident and Deployment. Incident workflow Open → Restored → Done (or treat resolution as restoration). Link type causes / caused by; every production incident becomes an Incident issue at detection time, linked caused by → <story or Deployment issue>. MTTR = mean(resolutiondate – created) over Incidents. One Deployment issue per production deploy; transition shipped stories to Released.

C.5 GitLab ↔ Jira (B0) and CI variables (B1)

Settings → Integrations → Jira issues: Web URL <https://jira.yourcompany.com>, auth Jira PAT (Bearer). Smart commits: `git commit -m "PLT-4821 verify green #transition Released"`.

Variable	Value
JIRA_BASE_URL	https://jira.yourcompany.com
JIRA_TOKEN	a service-account PAT (not a personal one)
JIRA_PROJECT	PLT

C.6 notify-jira job (B2)

```
notify-jira:
  stage: .post
  rules:
    - if: '$CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH && $CI_ENVIRONMENT_NAME == "production"'
  script:
    # 1 · one Deployment issue per prod deploy (the CFR denominator)
    - |
      curl -sfS -X POST "$JIRA_BASE_URL/rest/api/2/issue" \
        -H "Authorization: Bearer $JIRA_TOKEN" -H "Content-Type: application/json" \
        -d '{"fields":{"project":{"key":"$JIRA_PROJECT"},
          "issuetype":{"name":"Deployment"},
```

```
\ "summary\":"deploy $CI_COMMIT_SHORT_SHA · $CI_PIPELINE_URL\",
\ "labels\":[\ "env:prod\"]]}"
# 2 · transition every story shipped in this deploy to Released
- |
for KEY in $(git log "$CI_COMMIT_BEFORE_SHA..$CI_COMMIT_SHA" --pretty=%s%b \
    | grep -oE "$JIRA_PROJECT-[0-9]+" | sort -u); do
    TID=$(curl -sfS -H "Authorization: Bearer $JIRA_TOKEN" \
        "$JIRA_BASE_URL/rest/api/2/issue/$KEY/transitions" \
        | python3 -c 'import json,sys;
ts=json.load(sys.stdin)["transitions"];
print(next((t["id"] for t in ts if t["to"]["name"]=="Released"),""))')
    [ -n "$TID" ] && curl -sfS -X POST \
        "$JIRA_BASE_URL/rest/api/2/issue/$KEY/transitions" \
        -H "Authorization: Bearer $JIRA_TOKEN" -H "Content-Type: application/json" \
        -d "{\"transition\":{\"id\":\"$TID\"}}" || true
done
```

C.7 contract-coverage job (B3)

```
contract-coverage:
  stage: test
  script:
    - scripts/contract_coverage.sh "${COVERAGE_MIN:-0}"
```

Counts EARS-n IDs in specs/*/spec.md vs. IDs referenced by any *.feature or *.spec.ts (Covers EARS-2 and EARS-3 headers or (EARS-1) test titles both count).
Current repo output:

```
contract coverage: 3/11 criteria (27%)
uncovered: EARS-4 EARS-5 EARS-6 EARS-7 EARS-101 EARS-102 EARS-103 EARS-104
```

Set COVERAGE_MIN once the pilot stabilizes.

C.8 dora-dashboard job (B4)

```
dora-dashboard:
  stage: .post
  rules: [{ if: '$CI_PIPELINE_SOURCE == "schedule"' }]
  script:
    - python3 scripts/dora_dashboard.py --project "$JIRA_PROJECT" --days 14 | tee dora.txt
  artifacts: { paths: [dora.txt], expire_in: 90 days }
```

Pipeline schedule cron: 0 6 * * 1 (Monday mornings).

C.9 Dry run (C1)

```
export JIRA_BASE_URL=https://jira.yourcompany.com JIRA_TOKEN=<PAT>
python3 scripts/dora_dashboard.py --project PLT --days 14
```

C.10 Source-of-truth table (C3)

Metric	Source of truth	Where configured
Lead time	Jira changelog	\$A1-A2
Deployment frequency	GitLab prod pipelines (mirrored to Jira)	\$B2 (\$A5)
Change failure rate	Jira Incidents ÷ Deployment issues	\$A4-A5, \$B2
MTTR	Jira Incident timestamps	\$A4
First-pass verification	Jira rework label	\$A3
Contract coverage	git + CI	\$B3

C.11 MCP source (demo option C)

```
{ "mcpServers": { "atlassian": {
  "type": "url", "url": "https://mcp.atlassian.com/v1/mcp" } } }
```

Authenticate beforehand (`claude mcp list`); remove the server after the demo.

C.12 Live REST source (demo option B)

```
export JIRA_BASE_URL=https://jira.yourcompany.com
export JIRA_TOKEN=<personal-access-token>          # DC: PAT bearer
scripts/jira-fetch.sh PLT-4821                     # writes jira/PLT-4821.json
```

Jira Cloud: replace the Bearer header with `-u you@company.com:$JIRA_API_TOKEN`.
Fetch before the session and commit the JSON.

Appendix D

Repository Map

```
jira/PLT-4821.json          the vague ticket (the villain)
.claude/commands/
  specify-from-jira.md      Jira → spec.md, gated on EARS quality
  plan.md · tasks.md · implement.md · verify.md
.claude/hooks/resource_gate.sh  the PreToolUse gate (exit 2 = blocked)
.claude/state/ACTIVE_TASK    which task an agent may touch
specs/checkout-refunds/      the six artifacts (already worked, for reference)
docs/ddd.md                  bounded-context map – the DDD talking slide
features/steps/              BDD step definitions binding the .feature file
solution/                   RED stub + GREEN solution + demo:red / demo:green
scripts/contract_coverage.sh · scripts/dora_dashboard.py · scripts/jira-fetch.sh
```

Architecture inside `src/refunds/`: domain (pure — `RefundCalculator`, `Money`), application (`RefundsService`, `ports.ts`: `OrderSnapshotPort` · `RefundRepository` · `RefundEventsPort`), adapters (controller, in-memory store, event recorder); `main.ts` is the composition root — wiring only.

Appendix E

Troubleshooting

E.1 Demo troubleshooting (from DEMO.md §5)

Symptom	Fix
tsc deprecation error on moduleResolution	you're on TS 6 globally — the repo pins TS 5.5 via npm install; use npm run typecheck
Hook doesn't block	CLAUDE_PROJECT_DIR unset when testing manually — run via Claude Code, or export it
Cucumber "no steps"	run from repo root; cucumber.js resolves paths relative to it
Jest can't find ts	npm install didn't finish — rerun; typecheck:noinstall needs nothing
Agent edits refused everywhere	.claude/state/ACTIVE_TASK missing — that's the gate working; set the task

E.2 Instrumentation plumbing checks (from METRICS-SETUP.md §C2)

Move a test issue `Verify` → `In Progress` → the `rework` label appears. Run one production deploy → a `Deployment` issue is created and shipped stories transition to `Released`. Open the `Control Chart` → the `Spec Approved` → `Released` span is measured.

E.3 Metric pathologies and their reads

Batch-moved tickets → lead time is fiction (say the discipline note at kickoff).
Coverage dip at sprint boundary → the signal working; watch it recover. Coverage drifting down over time → spec rot, detected before the incident detects it.
Only one of six metrics moves → be suspicious. Deltas huge after two sprints → a direction, not a proof.

Appendix F

Glossary

Terms as used in the materials; definitions derive strictly from usage.

ACTIVE_TASK — `.claude/state/ACTIVE_TASK`; the one task an agent may touch (“the leash”). Missing → the gate blocks everything (fail-closed).

ADR (Architecture Decision Record) — a written, numbered decision; ADR-017 records banker’s rounding, half-to-even, and is cited from `Money.roundBankers`.

Aggregate — DDD tactical element; the *Refund* aggregate is keyed by `orderId`, which makes idempotency trivial.

Amendment — a formal spec change (e.g., Amendment A1 supersedes EARS-5; the Sprint-15 amendment adds EARS-6/7).

Banker’s rounding — round half to even; the ADR-017 policy applied in `Money.roundBankers`.

Baseline — metrics captured before the pilot (Sprints 11-12 in the worked example), on the same instruments used after.

BDD — Behaviour-Driven Development; the shared dialect: EARS re-expressed as Given-When-Then the PO can read and the pipeline can execute.

Bounded context — a DDD boundary owning its model and language; here Ordering, Payments, Refunds, Notification. One context, one folder.

CFR (Change failure rate) — % of deploys causing incident/rollback; numerator Jira Incidents (linked caused by), denominator Deployment issues / GitLab prod pipelines.

Composition root — `main.ts`; the only place wiring happens.

Contract coverage — EARS criteria with executable tests $\div$ total; computed by `scripts/contract_coverage.sh` in CI; dips at sprint boundaries by design.

Control Chart — Jira board report used for the lead-time quick read (Spec Approved $\rightarrow$ Released).

CQS — Command-Query Separation; `calculate()` answers, `requestRefund()` changes state and reports; no method does both quietly.

Critical path — $T1 \rightarrow T2 \rightarrow T3$; $T4/T6$ hang off it; $T5$ parallel via the Pact stub.

DDD — Domain-Driven Design; strategic (contexts, mapping, language) before tactical (aggregate, value objects, services).

Deployment issue — one Jira issue per production deploy (DC convention, created by the `notify-jira` job); the CFR denominator and frequency mirror.

DevEx — framework pairing each *perception* (pulse) with a *workflow* (system) measure: feedback loops, cognitive load, flow state.

DIP / DI — Dependency Inversion / Injection; the service depends on ports, never on Dynamo/HTTP/bus.

Domain event — a past-tense business happening (`EventCancelled ... BuyerNotified`); published at the seam (`refund.issued v1`).

DORA — DevOps Research & Assessment; four measures: lead time, deployment frequency, change failure rate, MTTR.

EARS — Easy Approach to Requirements Syntax; five patterns: Ubiquitous, Event (WHEN), State (WHILE), Unwanted (IF/THEN), Optional (WHERE). Criteria carry stable IDs (EARS-1...).

Exit gate — a stage's mechanical completion condition (spec approved + open questions empty; every in-scope verification row green).

Fail-closed — the gate's default: no `ACTIVE_TASK`, no edits anywhere.

First-pass verification — tasks conforming without rework $\div$ total; from the verification boolean / the rework label; "the purest spec-quality metric you will ever own." Target 90 %.

Frozen order snapshot — totals and service fees captured on the order; Re-funds' input, never recomputed from live fee config.

Functional core, imperative shell — pure domain (calculator, Money); all I/O in service + adapters. "Purity is search-space compression."

Gherkin — Given-When-Then scenario language of the .feature contract.

GSM — Goal → Signal → Metric, strictly in that order; metrics are proxies, held loosely.

Harness (shared, in-memory) — the single in-memory service+adapters rig both dialects bind to; "one oracle, two dialects."

Hexagonal (ports & adapters) — the architecture shape: domain, application, adapters; ports in ports.ts.

Hook — Claude Code PreToolUse script; resource_gate.sh; exit 2 = blocked. "A hook is enforcement; a prompt is persuasion."

Idempotency (as domain rule) — aggregate keyed by orderId; first act is a store lookup → 409; retries/double-clicks/replays are the same non-event.

ISP — small interfaces; three narrow ports instead of one god repository; five-line test doubles.

Lead time — first entry into Spec Approved → first entry into Released, from the Jira changelog; reported as a median.

Litmus test — "if you can't write the failing test, rewrite the criterion."

MCP — Model Context Protocol; the Atlassian server (<https://mcp.atlassian.com/v1/mcp>) lets the agent read Jira live; note the token cost; remove after the demo.

Minor units — integer cents; EARS-1; the Money value object's only representation.

MTTR — mean time to restore; mean(resolutiondate - created) over Incidents; "the goal isn't never failing; it's making failure cheap."

Open question — an unresolved spec item; /plan refuses while the list is non-empty.

Out-list — the spec's explicit exclusions (→ T4, → T5, behind flag); "gold-plating dies here, in writing"; during implementation, "the Out-list is law."

Pact — consumer-driven contract pair (T5: checkout-web ↔ orders-svc); enables parallel work via the stub.

PAT — personal access token; Bearer auth for Jira DC; CI uses a *service-account* PAT.

pp (percentage points) — absolute gap between two percentages (18 – 6 = 12 pp).

PreToolUse — the hook point at which the resource gate runs, before a tool action lands.

Pulse — the 5-question perceptual survey feeding SPACE/DevEx.

Rework label — added automatically on Verify → In Progress; the first-pass metric's raw material.

SPACE — Satisfaction, Performance, Activity, Communication, Efficiency; sample ≥ 3 dimensions, always one perceptual.

SRP — Single Responsibility; the calculator computes an amount, nothing else; the T3 test is four lines.

Touches: — a task's declared file globs; the resource gate's allow-list.

Triangulation — letting the next red test earn generality (minor-units and 409 tests pin the edges the first test doesn't).

Ubiquitous language — the same nouns from the PO's mouth to the code: *service fees*, *ISSUED/PENDING*, *minor units* — verbatim in spec, Gherkin, types, test names.

Value object — e.g., Money: holds integer cents, throws on anything else; illegal states unrepresentable — they don't compile.

Verification (stage/artifact) — conformance made explicit in `verification.md`: coverage table, evidence, CONFORMS – <date> verdict, first-pass boolean.

Vertical slice — a task cutting through the whole stack for one behavior, shippable alone; sized S ($\leq \frac{1}{2}$ day) or M (≤ 1 day), never L — “an L is a splitting instruction.”

Walking skeleton — the early minimal end-to-end path (T1 fixtures/harness, T2 endpoint) later slices land into.

Appendix G

Checklists

G.1 Pre-demo (from §0, §2, §4)

- Node ≥ 20 (`node -v`); Claude Code installed (`claude --version`).
- Provider chosen and toggled *before* the session (API key or `CLAUDE_CODE_USE_BEDROCK=` never switch mid-demo).
- Repo unzipped; `npm install` done; `npm run typecheck:noinstall` passes.
- Sanity: `npm test` → RED on T3; `npm run bdd` → scenario fails. Red is the correct starting state.
- Jira source picked (A offline JSON default / B REST fetched **before** the session, JSON committed / C MCP authenticated, `claude mcp list` confirms; plan to remove after).
- Reset run: `npm run demo:red && npm test` (RED confirmed); `ACTIVE_TASK` restored; `git checkout -- specs/` if the spec was edited live.
- Stage terminal: font ≥ 18 pt, dark theme, two panes; pane switch rehearsed.
- Fallbacks staged: `demo:green` ready to narrate; Jest-only path if cucumber misbehaves; slides 10-11 loaded.

G.2 Spec review (from §1's rules)

- Every criterion fits one of the five EARS patterns and carries an ID.
- Litmus test passed per criterion: the failing test is writable.
- Non-functionals are numbers (percentile, bound, exclusions stated).
- Out-list present; each exclusion routed (task, flag, or later).
- Open questions: zero. Decisions with rationale → ADRs.

- Co-signatures: engineer drafts, PO co-signs, QA reviews — inside the refinement hour, 45–60 min.

G.3 Task review (from §3's laws)

- 1 : 1 : 1 — one EARS criterion, one contract, one ticket.
- Size S or M; any L split before acceptance.
- Touches: globs declared (the gate's allow-list).
- Walking-skeleton ordering; verification bandwidth booked first.

G.4 Pilot (from the operating rules)

- Instrumentation installed (~2 h): A1–A6, B0–B4; C2 plumbing checks all pass.
- Kickoff held; the discipline note said out loud (batch-moving = fiction; status = test status).
- Baseline captured *before* the pilot; dashes recorded where pre-pilot measurement is impossible.
- Two sprints run; coverage dips read as the signal working; `COVERAGE_MIN` raised only after stabilization.
- Re-measure on the same instruments; report trends per team, every sprint.
- Retro traces every moved number to a spec section or pipeline stage.
- Never absolutes across teams; never individual-level metrics (Goodhart).
- Victory deferred until first-pass verification approaches 90 %.

Part XV

The Source Documents

The three runbooks this handbook is compiled from, reproduced in full. Typographic adaptations only, semantics unchanged: headings are demoted one level to nest under each appendix chapter; inter-document links are flattened to plain file names; in the architecture diagram the arrowhead glyphs are rendered as `>`, `<`, `^` and the block is re-set in the compact form used in the Architecture part (identical content); and two over-long single lines (the embedded `python3 -c` one-liner and the one-line MCP JSON) are soft-wrapped at points that preserve their exact behavior.

Appendix H

Source Document: WALKTHROUGH

H.1 WALKTHROUGH.md — one feature, end to end

Feature → Spec (EARS) → DDD → Tasks → Architecture → BDD → TDD → Implement → Verify → DORA

This is the detailed companion to DEMO.md (the 12-minute stage script). Here, every stage of the pipeline is worked in full on one real feature — **PLT-4821, partial refunds for cancelled events** — with the reasoning, the patterns, and the artifacts. Follow it once and you can run the method on any feature.

§0 · We have a feature

It arrives, as features do, as a Jira ticket (jira/PLT-4821.json):

Summary: Partial refund for cancelled events **Description:** When an event gets cancelled we need to refund the buyers. Refund the ticket price minus fees. Should be automatic, buyers shouldn't have to call support. Marketing wants this before the autumn on-sales. **AC:** refunds work for cancelled events. **Comment (PO):** "Fees = our service fees, not payment-provider fees. I think." **Comment (Support):** "Make sure people can't claim a refund twice."

Before touching anything, name the ambiguities. This ten-minute exercise is the highest-leverage act in the whole pipeline:

Vague phrase	The question nobody answered
"minus fees"	Which fees? Service fees? PSP fees? Both? (a comment says "...I think")
"automatic"	Triggered by what, exactly? Within what time?
"refunds work"	Untestable. What observable behaviour = "works"?
"can't claim twice"	What's the response to a duplicate? Silent no-op? Error? Which code?
(unstated)	What currency precision? Rounding rule on fee math?
(unstated)	Partial refunds per line item — now or later?

Every one of these gaps *will* be filled by someone's assumption — a developer's, a QA's, or an agent's, sampled fresh on every retry. The rest of this document is the machinery that fills them **on purpose**.

§1 · Design the spec — EARS

Who & when: the engineer drafts, the PO co-signs, QA reviews — inside the existing refinement hour. Timebox: 45-60 min for a feature this size.

Where: specs/checkout-refunds/spec.md, from templates/spec.template.md (or live: /specify-from-jira PLT-4821).

The transformation. Take each vague phrase through the interrogation and land it in one of the five EARS patterns (skill: ears-authoring):

Before (ticket)	Interrogation	After (spec)
"refund the ticket price minus fees"	which fees? when?	EARS-2 (Event): WHEN an event is cancelled, the system shall refund the ticket price minus <i>service fees</i> within 5 business days.
implied money handling	floats? rounding?	EARS-1 (Ubiquitous): The refunds API shall return amounts in minor units (cents).

Before (ticket)	Interrogation	After (spec)
“can’t claim twice”	duplicate response?	EARS-3 (State): WHILE a refund is PENDING or ISSUED, further refund requests for that order shall be rejected with 409.
(security hygiene)	expired token?	EARS-4 (Unwanted): IF the access token is expired, THEN the API shall return 401 with WWW-Authenticate.
“partial refund” in the title	now?	EARS-5 (Optional): WHERE partial refunds are enabled, the system shall support line-item refunds. → flagged out of this slice.

Rules enforced while writing: 1. Every criterion passes the **litmus test** — *if you can’t write the failing test, rewrite the criterion.* (EARS-2 passes: the test is in §6. “Refunds work” never could.) 2. **Non-functionals as numbers:** p95 issuance ≤ 300 ms (excl. PSP); zero double refunds under retry. 3. **Scope has an Out-list:** notification wiring (→ T4), refund UI (→ T5), line-item refunds (behind flag, later). Gold-plating dies here, in writing. 4. **Open questions must reach zero before planning.** Ours had two: *which fees* (PO confirms: service fees, from the order snapshot) and *rounding on fractional fee math* (decided: banker’s, half-to-even — logged as **ADR-017**, because a decision that isn’t written gets re-litigated).

Exit gate: spec.md approved in review, Open questions **empty**. Only now does /plan unlock.

§2 • DDD — place the feature in the domain

Strategic design answers *where does this live and what language does it speak* — before any class diagram. Fifteen minutes of lightweight event storming on the flow surfaces the domain events:

EventCancelled → RefundRequested → RefundCalculated → RefundIssued → BuyerNotified

Bounded contexts (full map in docs/ddd.md): Ordering owns orders and their

frozen totals; Payments owns money movement; **Refunds owns refund policy** — the math, the state machine, the idempotency; Notification owns outbound messages. The calculator therefore lives in the Refunds context — `src/refunds/` — and nowhere else. One context, one folder: when a refund contract goes red, there is exactly one directory to open.

Context mapping. Refunds is *downstream-conformist* to Ordering: totals and service fees come **frozen on the order snapshot**, never recomputed from live fee config (this single decision kills a whole class of “fees changed mid-refund” bugs). Refunds is *upstream* to Notification: it publishes `refund.issued v1` and knows nothing about templates.

Ubiquitous language — the same nouns from the PO’s mouth to the code: *service fees* (ours, on the snapshot — the “...I think” comment, now decided), *ISSUED/PENDING* (refund states; duplicates → 409), *minor units* (integer cents everywhere). These words appear verbatim in `spec.md`, in the Gherkin, in the type names, and in the test descriptions. That is the point.

Tactical sketch (lands in `plan.md`): the *Refund* is the aggregate, keyed by `orderId` (which is what makes idempotency trivial); *Money* is a value object; the calculator is a pure domain service. Risks table: fee model varies by market → mitigated by the snapshot decision; double refunds under retry → mitigated by EARS-3 + the keyed aggregate.

§3 · Split it into tasks

Slicing method: **vertical slices** that each ship alone, ordered so a *walking skeleton* exists early. Rules (the tasks command refuses to break them): every task is **1 : 1 : 1** — one EARS criterion, one contract, one ticket; sizes **S ($\leq \frac{1}{2}$ day) or M (≤ 1 day), never L** — an L is a splitting instruction; every task declares Touches: globs for the resource gate.

Derivation — criterion → contract → task (`specs/checkout-refunds/tasks.md`):

Task	From	Contract	Size	Why this slice
T1 · fixtures & harness	EARS-1	feature background	S	walking skeleton: gives every later test its verbs
T2 · endpoint skeleton	EARS-3	OpenAPI 201/409	S	contract-first: FE can mock <i>today</i> ; 409 path proven early
T3 · refund calculation	EARS-2	POST + “refund on cancellation” scenario	M	the domain heart — the demo’s live task isolates
T4 · refund.issued event	(out-of-scope wiring)	event schema v1	S	Notification coupling
T5 · status UI slice	scope §2	pact checkout-web ↔ orders-svc	M	consumer-driven contract pair
T6 · expired-token guard	EARS-4	OpenAPI 401	S	unwanted-behaviour pattern gets its own slice

Critical path: T1 → T2 → T3; T4/T6 hang off it; T5 parallel via the Pact stub. Estimation summary: **4×S + 2×M ≤ 4 dev-days**, with **1.5 days of review & verification bandwidth booked first** — with agents typing, verification is the constraint, so it gets capacity before development does. Notice what happened to estimation: it became **counting**.

§4 · Architecture — patterns & principles, applied

The shape is **hexagonal (ports & adapters)** with a **functional core and an imperative shell** — chosen not for fashion but because every principle below buys a measurable debugging or testing property. The code in `src/refunds/` mirrors this exactly:

```

+----- Refunds bounded context -----+
| DOMAIN (pure, no I/O)  APPLICATION      ADAPTERS  |
HTTP --> RefundCalculator  RefundsService  controller |
| Money (value object)   (idempotency, <- in-memory |

```

```
|               state,               store |
|               orchestration)      event |
|   ^ depends on   ^ depends      recorder |
|   | nothing      | on PORTS -----+   |
|   +--- ports.ts: OrderSnapshotPort      |
|               RefundRepository . RefundEventsPort |
+-----+-----+-----+-----+-----+
```

Principle by principle — each with its payoff:

- **SRP (Single Responsibility).** RefundCalculator computes an amount. Nothing else — no storage, no events, no HTTP. Payoff: the T3 test is four lines and deterministic.
- **Functional core, imperative shell** (Bernhardt). The calculator and Money are pure; all I/O lives in the service + adapters. Payoff: bugs in refund *math* reproduce on a laptop in milliseconds — never “only in staging, around noon.” Purity is search-space compression.
- **DIP (Dependency Inversion) + DI.** RefundsService depends on the three **ports** (interfaces in ports.ts), never on Dynamo, HTTP or the bus. main.ts is the *composition root* — wiring only. Payoff: swap a fake in one line and answer “my logic or their service?” in one test run.
- **ISP (small interfaces).** Three narrow ports instead of one god Repository. Payoff: test doubles are five lines; adapters can’t grow hidden coupling.
- **Value objects / make illegal states unrepresentable.** Money holds *integer cents* and throws on anything else; refund status is a closed union. Payoff: the float-drift and half-cent bugs from the origin story don’t sneak past review — they **don’t compile**.
- **Idempotency as a domain rule, not middleware.** The Refund aggregate is keyed by orderId; the service’s first act is a store lookup → 409 (EARS-3). Payoff: retries, double-clicks and replayed events are all the same non-event.
- **Domain events at the seam.** The service publishes refund.issued v1 through the events port; Notification subscribes. Payoff: T4 ships without touching T3’s code — the Out-list made physical.
- **One decision, one place, one record.** Rounding happens once, in Money.roundBankers, cited by **ADR-017**. Payoff: the next engineer who “fixes” 2.5 → 3 by intuition finds the ADR, not a debate.
- **OCP, gently.** New refund policies (line-item, EARS-5) extend by a new

calculator behind the same call shape — no modification of the verified path. We do *not* pre-build the abstraction: the flag stays in the spec until it's real. (Patterns are applied where they pay, not collected.)

- **CQS.** `calculate()` answers; `requestRefund()` changes state and reports. No method does both quietly.

Tests attach at two seams: **domain** (calculator: pure unit tests — fast, exhaustive) and **application** (service against in-memory adapters: the Gherkin and the 409 path). The HTTP adapter is covered by the OpenAPI contract; the cross-service promise by Pact (T5). Every layer has exactly one kind of test, and every test knows which layer it accuses when it fails.

§5 • Create the BDD

BDD is the **shared dialect**: the EARS criteria, re-expressed as Given-When-Then that the PO can read *and* the pipeline can execute. The translation is nearly mechanical — that's a feature:

EARS	Gherkin (in <code>contracts/refunds.feature</code>)
EARS-2: WHEN an event is cancelled...	<i>Given</i> a paid order for event "EVT-303" · <i>When</i> the event is cancelled · <i>Then</i> a refund is issued for price minus service fees · <i>And</i> the order status is "REFUNDED"
EARS-3: WHILE PENDING/ISSUED... 409	<i>Given</i> the order already has a refund · <i>When</i> a refund is requested again · <i>Then</i> the request is rejected with status 409

Rules for scenarios that stay healthy: declarative, domain-language steps (*"the event is cancelled"* — never "click the cancel button"); one behaviour per scenario; shared setup in `Background::`; ubiquitous-language nouns only. If a scenario needs UI verbs, it's testing the wrong layer.

Step definitions (`features/steps/refunds.steps.ts`) bind the scenarios to the **same in-memory harness the unit tests use** — one oracle, two dialects. Run: `npm run bdd` → both scenarios **fail**, correctly, because nothing is implemented. Red BDD before code is the proof that the scenarios can detect absence — the same logic as TDD's red, one level up.

§6 · Create the TDD

The failing test is derived **from the contract, never from the implementation** (there is no implementation). For T3, the contract names the oracle and the test writes itself (src/refunds/application/refunds.service.spec.ts):

```
it("refunds price minus fees when the event is cancelled", async () => {
  const order = await paidOrder("EVT-303");
  await events.cancel("EVT-303");
  expect(await refundFor(order.id)).toEqual(
    order.total.minus(order.fees) // RED — no implementation yet (T3)
  );
});
```

Run `npm test` → **RED**, with NotImplemented — task T3. This failure is an asset: it proves the detector works. A test written after the code passes *by construction* and proves nothing — which is why the discipline is:

1. **Red** — smallest failing test from the contract. No production code without one.
2. **Green** — the *minimum* to pass. Resist generality; the next red test earns it (triangulation: EARS-1's minor-units test and the 409 test pin the edges the first test doesn't).
3. **Refactor** — only while green: names to ubiquitous language, extract, de-dupe.
4. Repeat until the task's Done: line is literally true.

Agent-assisted variant: set the leash — `echo "checkout-refunds T3" > .claude/state/ACTIVE_TASK` — then `/implement T3`. The agent runs the same loop; the human reviews every diff **against the criteria** (conformance, not style). Try an edit outside Touches: and the hook answers exit 2: a hook is enforcement; a prompt is persuasion.

§7 · Implement

The green for T3 is deliberately anticlimactic — the whole pipeline exists to make it so (solution/refund-calculator.solution.ts, or live-typed):

```
export class RefundCalculator {
  calculate(order: Order): Money {
    const cents = Money.roundBankers(order.total.cents - order.fees.cents);
    return Money.ofCents(cents); // EARS-2 · minor units per EARS-1 · ADR-017
  }
}
```

Commentary: integer-cent subtraction is exact, so today the rounding guard is a no-op — it exists because ADR-017 says *all* refund math passes through one audited rounding point, so tomorrow’s percentage-fee change can’t reintroduce the origin-story bug. The function stays pure: the service (imperative shell) owns the store lookup → 409, the state transition, and the refund.issued publish — all behind ports.

Discipline while implementing: **the Out-list is law** — T4’s event templates and T5’s UI stay untouched even when “it would only take a minute”; anything learned that contradicts the spec goes back as an **open question or ADR**, never a silent interpretation. Then:

```
npm test && npm run bdd # GREEN – both dialects agree on the same oracle
```

§8 · Verify

Verification is the exit gate — conformance made explicit, in specs/checkout-refunds/verify (or /verify checkout-refunds):

1. **Criteria coverage table** — every EARS row maps to named green tests. EARS-2 flips Red – T3 open → **Green**. EARS-5 stays – (out of slice, *documented* as such).
2. **Per-task evidence** — suite names, results, and **deviations** → **ADRs** (T3’s row cites ADR-017: decided up-front, not during coding).
3. **Verdict** — CONFORMS – <date> only when every in-scope row is green; one red row and the feature does not merge — it reopens the spec or the task, visibly.
4. **First-pass entry** — did this task conform without rework? That single boolean, aggregated, is the purest spec-quality metric you will ever own.

The demo *is* the verification: run the live slice behind its flag. No screenshots. If Verify found a gap, the spec — not the definition of done — would have changed, and the loop tightens instead of drifting.

§9 · And at the end — DORA (plus the two SDD signals)

Metrics close the loop from *practice* to *proof*. **DORA — DevOps Research & Assessment** — four measures, each with its exact definition here and the mechanism by which this pipeline moves it:

DORA metric	Measured here as	How this pipeline moves it	Instrument from
Lead time for changes	spec-approved → running in prod	≤1-day slices kill queueing; contracts unblock parallel FE/BE/QA (PLT-4821: 4.2 days median spec→verified)	board timestamps (Spec col → Done)
Deployment frequency	deploys/week	small batches behind flags → daily merges of verified slices	CI/CD pipeline events
Change failure rate	% deploys causing incident/rollback	contracts catch regressions pre-prod ; Red never ships as “known issue”	incident tracker ÷ deploys
MTTR (mean time to restore)	failure → restored	thin slices = thin rollbacks; a red contract test <i>names the folder</i> (localize → reproduce → bisect)	incident timestamps

The two SDD-native signals (leading indicators DORA can’t see):

- **First-pass verification** = tasks conforming without rework ÷ total. Sprint 14: **81 %**, target 90. Falls out of §8’s boolean — zero extra tooling. When it dips, the retro reads *which spec section* failed.
- **Contract coverage** = EARS criteria with executable tests ÷ total. At Sprint-14 close: **4/4 in-scope** (EARS-5 consciously excluded). The Sprint-15 amendment (EARS-6/7) and the new wallet-credit spec enter *uncovered* — coverage dips at every sprint boundary and recovers as their tasks land: the dip is the signal working. Drift downward = spec rot, detected before the incident detects it for you.

Beyond DORA — SPACE, DevEx and GSM, worked with the same sprint
SPACE — sample ≥ 3 dimensions, always including one perceptual (survey) measure. ✓ marks the three this team samples:

Dim.	Question it answers	Worked example (Sprint 14)	Instrument	Sampled
S · Satisfaction & well-being	Do people feel good about how work works?	Spec-quality pulse: 4.2 / 5 (“criteria were clear before I started”)	5-question pulse	✓ (perceptual)
P · Performance	Outcomes, not effort	First-pass verification: 81 %	verification.md	optional
A · Activity	Output volume (context, never a target)	20 verified tasks	board count	✓
C · Communication & collaboration	Does knowledge flow?	Specs co-authored PO + engineer: 6 / 6	spec review log	optional
E · Efficiency & flow	How much waiting between steps?	Wait Contract → Red: 3 h median	board timestamps	✓

DevEx — pair every *perception* with a *workflow* measure; feelings are data too:

Driver	What it captures	Perception (pulse)	Workflow (system)
Feedback loops	How fast the system answers you	“I get test/review results quickly” — 4.5 / 5	suite 2 m 40 s · review turnaround 4 h
Cognitive load	How hard it is to think here	“It’s easy to know exactly what to build” — 4.1 / 5	new dev → first verified task: 3 days
Flow state	How often deep work survives the calendar	“I get uninterrupted focus time” — 3.9 / 5	WIP = 1 → 0 mid-task handoffs

GSM — Goal → Signal → Metric, strictly in that order; two worked chains:

Step	Rule	Chain 1 — spec quality	Chain 2 — review bottleneck	Goodhart trap avoided
Goal	In words; no numbers yet	Refunds ship with zero rework caused by ambiguity	Verification never starves the pipeline	starting from “what’s easy to count”
Signal	How you’d <i>know</i> , even if unmeasurable	Verified work rarely reopens	Finished work rarely waits on review	mistaking activity for the goal
Metric	A measurable <i>proxy</i> , held loosely	First-pass 81 % (→ 90) · reopened: 1	Wait Green → Verify: 4 h median (< 1 day)	optimizing the proxy (rushed reviews)

One instrumentation pass — the board, `verification.md`, and the five-question pulse — feeds all three frameworks *and* DORA: watch the pipeline, watch the humans, keep the “why” honest.

Can Jira track these? Mostly yes — here’s the split Precondition for everything: the Jira workflow statuses must mirror the pipeline columns (*Spec* → *Contract* → *In Progress* → *Green* → *Verify* → *Done/ Released*), because the issue **changelog timestamps every transition** — and those timestamps are the metrics.

Metric	In Jira?	How (Data Center)
Lead time	Yes — best source	changelog: <i>Spec approved</i> → <i>Released</i> ; Kanban Control Chart out of the box, or a nightly <code>/rest/api/2/search?expand=changelog</code> job / Time-in-Status app
First-pass verification	Yes — easiest	Automation rule: transition <i>Verify</i> → <i>In Progress</i> adds label <i>rework</i> ; then status = Done AND labels != <i>rework</i> ÷ all Done
MTTR	Yes, if incidents are issues	Incident issue type; created → resolved (or a <i>Restored</i> status) via resolution reports/JQL

Metric	In Jira?	How (Data Center)
Change failure rate	Partially	numerator: incidents linked to the causing release/story (linking discipline is the whole game); denominator (deploys) must be mirrored in
Deployment frequency	Mirror from CI	CD job calls Jira REST on each prod deploy (transition to <i>Released</i> / deployed label), or GitLab-for-Jira feeds the development panel; counting stays truthful in GitLab
Contract coverage	No — CI owns it	a CI step counts EARS- <i>n</i> IDs in specs vs. test annotations and posts to a dashboard gadget; source of truth stays in git

Step-by-step configuration for all of this — workflow statuses, the Automation rule, the CI jobs and the scheduled dashboard script — is in **METRICS-SETUP.md**.

Working architecture, matching “instrument once”: **Jira = workflow clock** (lead time, first-pass, MTTR) · **GitLab CI = delivery clock** (frequency, coverage) · incidents-as-issues bridge the two for CFR · one nightly REST script joins them into the dashboard. Caveat to say out loud: Jira only measures what people actually transition — batch-moving tickets on Friday turns lead time into fiction, which is exactly why an SDD board helps: status is *test status*, driven by the pipeline’s reality, not by memory.

Operating rules: capture the **baseline before the pilot** (this month), re-measure after two sprints, judge **trends per team — never absolutes across teams**, and never at individual level (Goodhart’s law: the fastest way to turn a measurement program into a gaming program). One instrumentation pass — the board, CI events, *verification.md*, and a 5-question pulse — feeds DORA today and SPACE/DevEx/GSM (deck slides 26-27) tomorrow.

Worked example — the dashboard after two SDD sprints, with numbers

Baseline captured over Sprints 11-12 (pre-pilot, same team of five); re-measured over Sprints 13-14 (the pilot). Same board, same CI, same incident tracker — only the practice changed.

Metric	Baseline · S11-12	SDD pilot · S13-14	Δ
Lead time (spec-approved → prod, median)	9.0 days	4.2 days	−53 %
Deployment frequency	2 / week	8 / week (16 deploys in 10 days, behind flags)	×4
Change failure rate	18 % (7 incidents / 39 deploys)	6 % (1 / 16)	−12 pp
MTTR	~6 h (multi-slice rollbacks, log archaeology)	40 min (flag-off + red test named the folder)	−89 %
<i>First-pass verification</i> (SDD)	— (not measurable pre-pilot)	74 % → 81 % (S13 → S14; target 90)	+7 pp
<i>Contract coverage</i> (SDD)	0 %	93 % (38 / 41 in-scope EARS criteria)	+93 pp

Where each number came from — no new tooling, just the artifacts this repo already produces:

- **Lead time:** board timestamps, Spec approved → Verified in prod. The five verified Sprint-14 stories took 3.1 · 3.8 · **4.2** · 4.6 · 5.0 days — median 4.2 (PLT-4830, rolled, is excluded from the median and *visible* as a roll, not hidden inside an average).
- **Deployment frequency:** CI deploy events. Twenty verified tasks merged behind flags produced 16 production deploys in 10 working days.
- **CFR:** incident tracker ÷ deploys. The one pilot incident was a flag mis-configuration — notably *not* a contract regression; the contracts caught those pre-merge (PLT-4830's rounding gap cost 2 h, not an incident).
- **MTTR:** flag-off restored service in 40 minutes; the failing contract test pointed at src/refunds/ before the call ended — localize → reproduce → bisect, as designed.

Reading the table — what each number actually says **Did they improve? Yes — all six.** “Better” points in different directions, so read the arrows with this rule:

Direction of good	Metrics	What happened	Verdict
Lower is better	Lead time · Change failure rate · MTTR	9.0→4.2 d · 18→6 % · 6 h→40 min — all down	✓ improved
Higher is better	Deployment frequency · First-pass verification · Contract coverage	2→8 /wk · 74→81 % · 0→93 % — all up	✓ improved

The strongest signal is the pair that normally *fights*: deploying more often usually means breaking more often. Here frequency rose $\times 4$ **while** failure rate fell — improving both sides of that trade-off at once is what tells you the process, not luck, did it.

- **Lead time (median), 9.0 → 4.2 days (−53 %).** Clock time from *spec approved* to *running in production* — the full journey of one slice, including all the waiting. It’s a **median** on purpose: half of all items now arrive within 4.2 days, and one disaster story can’t hide inside an average. The mechanism behind the drop is not faster typing — it’s less queueing, because one-day slices don’t wait behind each other.
- **Deployment frequency, 2 → 8 / week ($\times 4$).** How often code reaches production (16 deploys in 10 working days). “Behind flags” matters: each deploy is dark until the flag flips, so shipping often $\neq$ exposing users often. Frequency is really a proxy for **batch size** — you can only deploy $8\times$ a week if changes are small, and small changes are the easy ones to review, test and undo.
- **Change failure rate, 18 % → 6 % (−12 pp).** Of all deploys, the fraction that caused an incident or rollback: 7/39 before, 1/16 after. **pp = percentage points**, the absolute gap between two percentages ($18 - 6 = 12$) — written that way because “−12 %” would ambiguously suggest a relative drop. The *shape* of the remaining failure matters too: a flag misconfig, not a contract regression — that class died pre-merge (2 h, not an outage).

- **MTTR, ~6 h → 40 min (−89 %).** Mean Time To Restore: when a deploy does fail, detection → healthy again. Forty minutes is what “designed for debugging” buys — flag-off restores instantly, and the red contract test names the folder. MTTR reframes risk: the goal isn’t never failing; it’s making failure *cheap*.
- **First-pass verification, 74 → 81 % (target 90).** Share of tasks that passed verification **without rework** — 17 of 21 right the first time. The purest spec-quality signal there is: a first-pass failure almost always traces to an ambiguity that survived spec review. No baseline exists because the gate itself didn’t exist before the pilot.
- **Contract coverage, 0 → 93 %.** 38 of 41 in-scope EARS criteria have an executable test bound to them; the missing three are named, not forgotten. Watch the **direction** more than the level — drifting down means specs are outrunning tests: the early smell of spec rot.

One meta-observation for the Q&A: all six move for the *same underlying reason* — smaller, unambiguous, verified slices — which is what makes them credible together. If only one had moved, be suspicious.

Read it honestly: two sprints is a **direction, not a proof** — the deltas are large partly because the baseline was soft. Keep the same instruments running, report the trend per team every sprint, and let first-pass verification climb toward 90 % before declaring victory. The dashboard’s job is to make the next argument about data instead of vibes.

Appendix · stage → artifact → command

Stage	Artifact	Command / file
Feature arrives	jira/PLT-4821.json	scripts/jira-fetch.sh (live)
Spec (EARS)	specs/.../spec.md	/specify-from-jira PLT-4821
DDD	plan.md, docs/ddd.md	/plan checkout-refunds
Tasks	tasks.md	/tasks checkout-refunds
Architecture	src/refunds/application/ports.ts,— refunds.service.ts, docs/code-structure.md	
BDD	contracts/refunds.feature, features/steps/	npm run bdd
TDD (red)	refunds.service.spec.ts	npm test

Stage	Artifact	Command / file
Implement	refund-calculator.ts	/implement T3 (demo:green fallback)
Verify	verification.md	/verify checkout-refunds
DORA	metrics table above	board + CI + verification.md

Spec first. Verify always. Measure honestly.

Appendix I

Source Document: METRICS-SETUP.md

I.1 METRICS-SETUP.md — configuring Jira & GitLab to track the DORA + SDD dashboard

Step-by-step for Jira Data Center + self-managed GitLab (Cloud notes where they differ)

Goal: after this setup, the six-metric dashboard from WALKTHROUGH.md §9 computes itself — **Jira is the workflow clock** (lead time, first-pass verification, MTTR, and the incident side of CFR), **GitLab CI is the delivery clock** (deployment frequency, contract coverage, and the deploy side of CFR), and one scheduled script joins them.

Time budget: ~2 hours once, per project. Everything below uses only built-in features, Automation for Jira, and two scripts that ship in this repo (scripts/contract_coverage.sh, scripts/dora_dashboard.py).

Part A — Jira (Data Center)

A1 · Workflow statuses that mirror the pipeline (*prerequisite for everything*)

The changelog timestamps every status transition — those timestamps **are** the metrics. So the workflow must speak the pipeline's language.

1. **Admin → Issues → Statuses:** create Spec, Spec Approved, Contract, Green, Verify, Released (you already have In Progress and Done). Categories: Spec*/Contract → *To Do*; In Progress/Green/Verify → *In Progress*; Released/Done → *Done*.
2. **Project → Workflow (edit in designer):** order them Spec → Spec Approved → Contract → In Progress → Green → Verify → Done → Released. Allow Verify → In Progress (the rework path — we *want* it visible), and a global transition to Released (CI will drive it, §B2).
3. **Board → Board settings → Columns:** map one status per column so the board reads exactly like deck slide 19: *Spec · Contract · In Progress (Red) · Green · Verify · Done*.

Discipline note: Jira measures what people transition. If tickets get batch-moved on Friday, lead time is fiction. The SDD board resists this because status = test status — but say it out loud in the team kickoff.

A2 · Lead time (spec-approved → released)

Nothing to install:

- **Quick read:** Board → **Reports → Control Chart** → set the columns from Spec Approved to Released → the median line is your lead time; watch the trend, not single points.
- **Exact numbers:** the nightly script (§C1) reads `/rest/api/2/search?expand=changeLog` and computes the median from the first entry into Spec Approved to the first entry into Released.

A3 · First-pass verification (the rework label rule)

One Automation rule makes rework self-recording:

1. **Project settings → Automation → Create rule.**
2. **Trigger:** Issue transitioned · From Verify · To In Progress.
3. **Action:** Edit issue → Labels → **add** rework.
4. Name it SDD · mark rework, scope: this project. Done.

Read it with two saved filters (Dashboard → two *Issue Statistics* or *Filter Count* gadgets, or let §C1 compute the ratio):

```
-- all finished
project = PLT AND issuetype in (Story, Task) AND statusCategory = Done AND resolved >= -14d
-- finished first-pass
project = PLT AND issuetype in (Story, Task) AND statusCategory = Done AND resolved >= -14d
AND (labels IS EMPTY OR labels != rework)
```

First-pass % = second count ÷ first count.

A4 • Incidents → CFR numerator + MTTR

1. **Admin → Issues → Issue types:** add Incident (standard type) and add it to the project's scheme. Simple workflow Open → Restored → Done — or keep the default and treat *resolution* as restoration.
2. **Team rule (the whole game):** every production incident becomes an Incident issue **at detection time**, and gets an issue link caused by → <the story or the Deployment issue> (Admin → Issue linking: add link type causes / caused by if missing).
3. **MTTR** = mean(resolutiondate – created) over Incidents — the script computes it; the built-in *Average Resolution Time* gadget approximates it.

A5 • Deployments in Jira (the CFR denominator + frequency mirror)

Jira DC has no native deployments feed (that's a Cloud feature), so CI puts them there — two complementary conventions, both driven from GitLab in §B2:

- **One Deployment issue per production deploy** (Admin → Issue types: add Deployment). Counting them per week = deployment frequency inside Jira; linking incidents to them gives clean CFR.
- **Transition shipped stories to Released** so lead time ends at the truth.

A6 • The dashboard

Jira Dashboard with: Control Chart link, the two first-pass filter counters, *Created vs Resolved* for Incidents — plus the script's weekly table (§C1) posted wherever the team reads (Slack/Confluence/CI artifact).

Jira Cloud differences: Automation is built-in the same way; deployments can come natively from the GitLab for Jira Cloud app (development panel + deployments), so §A5's Deployment issues become

optional; JQL identical.

Part B — GitLab (self-managed)

B0 • Connect GitLab ↔ Jira (dev panel + smart transitions)

GitLab project → **Settings** → **Integrations** → **Jira issues**: Web URL = `https://jira.yourcompany.com`, auth = Jira PAT (Bearer). From now on any commit/MR/branch containing PLT-123 shows in the Jira issue's development panel — and **smart commits** work: `git commit -m "PLT-4821 verify green #transition Released"`.

B1 • CI variables

GitLab project → **Settings** → **CI/CD** → **Variables** (masked, protected):

Variable	Value
JIRA_BASE_URL	<code>https://jira.yourcompany.com</code>
JIRA_TOKEN	a service-account PAT (not a personal one)
JIRA_PROJECT	PLT

B2 • Deploy job → tell Jira (frequency + Released + CFR denominator)

Append to `.gitlab-ci.yml` — runs after every **production** deploy:

```
notify-jira:
  stage: .post
  rules:
    - if: '$CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH && $CI_ENVIRONMENT_NAME == "production"'
  script:
    # 1 • one Deployment issue per prod deploy (the CFR denominator)
    - |
      curl -sfS -X POST "$JIRA_BASE_URL/rest/api/2/issue" \
        -H "Authorization: Bearer $JIRA_TOKEN" -H "Content-Type: application/json" \
        -d '{"fields":{"project":{"key":"$JIRA_PROJECT"},
          "issuetype":{"name":"Deployment"},
          "summary":"deploy $CI_COMMIT_SHORT_SHA · $CI_PIPELINE_URL",
          "labels":["env:prod"]}}'
    # 2 • transition every story shipped in this deploy to Released
    - |
```

```
for KEY in $(git log "$CI_COMMIT_BEFORE_SHA..$CI_COMMIT_SHA" --pretty=%s%b \
    | grep -oE "$JIRA_PROJECT-[0-9]+" | sort -u); do
    TID=$(curl -sfS -H "Authorization: Bearer $JIRA_TOKEN" \
        "$JIRA_BASE_URL/rest/api/2/issue/$KEY/transitions" \
        | python3 -c 'import json,sys;
ts=json.load(sys.stdin)["transitions"];
print(next((t["id"] for t in ts if t["to"]["name"]=="Released"),""))')
    [ -n "$TID" ] && curl -sfS -X POST \
        "$JIRA_BASE_URL/rest/api/2/issue/$KEY/transitions" \
        -H "Authorization: Bearer $JIRA_TOKEN" -H "Content-Type: application/json" \
        -d '{"transition":{"id":"$TID"}}' || true
done
```

Deployment frequency now exists in *both* systems: GitLab (pipelines on the production environment — Deployments page / API) and Jira (Deployment issues) — they must agree, which is itself a useful health check.

B3 • Contract coverage job (the CI-owned SDD signal)

```
contract-coverage:
  stage: test
  script:
    - scripts/contract_coverage.sh "${COVERAGE_MIN:-0}"
```

The script counts EARS-n IDs in specs/*/spec.md versus IDs referenced by any *.feature or *.spec.ts (the repo's convention — a header line like Covers EARS-2 and EARS-3 or a test title (EARS-1) both count). On this repo today it prints, truthfully:

```
contract coverage: 3/11 criteria (27%)
uncovered: EARS-4 EARS-5 EARS-6 EARS-7 EARS-101 EARS-102 EARS-103 EARS-104
```

...which is correct — and a lesson in one line: EARS-4 lands with T6; EARS-5 is superseded by Amendment A1; EARS-6/7 arrived with the Sprint-15 amendment and EARS-101...104 with the brand-new wallet-credit spec — their contracts land with their tasks. **Coverage dips at every sprint boundary; the dip is the signal working.** Watch it recover as tasks land. Set COVERAGE_MIN once the pilot stabilizes to make drift a red pipeline instead of a surprise.

B4 • Weekly dashboard (scheduled pipeline)

```
dora-dashboard:
  stage: .post
```

```
rules: [{ if: '$CI_PIPELINE_SOURCE == "schedule"' }]
script:
  - python3 scripts/dora_dashboard.py --project "$JIRA_PROJECT" --days 14 | tee dora.txt
artifacts: { paths: [dora.txt], expire_in: 90 days }
```

GitLab → **Build → Pipeline schedules → New**: cron 0 6 * * 1 (Monday mornings). The script (stdlib-only, PAT auth) prints the table: lead-time median from the changelog, deployments/week from Deployment issues, CFR = incidents ÷ deployments, MTTR from incident resolution times, first-pass from the rework label — contract coverage it defers to §B3’s job.

Part C — Sanity checks & operating rules

C1 • Dry run the script (from any machine that can reach Jira):

```
export JIRA_BASE_URL=https://jira.yourcompany.com JIRA_TOKEN=<PAT>
python3 scripts/dora_dashboard.py --project PLT --days 14
```

C2 • Verify the plumbing, one item each: move a test issue Verify → In Progress and confirm the rework label appears; run one production deploy and confirm a Deployment issue + a Released transition; open the Control Chart and confirm the Spec Approved → Released span.

C3 • Operating rules (from the walkthrough, they bear repeating): capture the **baseline before the pilot**; re-measure after two sprints; judge **trends per team, never absolutes across teams**; never at individual level (Goodhart’s law); and when a number moves, ask *which spec section or pipeline stage* moved it — that’s the retro’s job.

Metric	Source of truth	Where configured
Lead time	Jira changelog	§A1–A2
Deployment frequency	GitLab prod pipelines (mirrored to Jira)	§B2 (§A5)
Change failure rate	Jira Incidents ÷ Deployment issues	§A4–A5, §B2
MTTR	Jira Incident timestamps	§A4
First-pass verification	Jira rework label	§A3
Contract coverage	git + CI	§B3

Instrument once. Argue with data.

Appendix J

Source Document: DEMO.md

J.1 DEMO.md — configuring and running the full SDD demo

Jira ticket → spec (EARS) → DDD → BDD → TDD → verified, in ~12 minutes

(Stage script only — for the fully reasoned, stage-by-stage method with all patterns and the DORA close, see WALKTHROUGH.md.)

This runbook turns the repo into a live, end-to-end demonstration of the whole pipeline: a deliberately vague Jira ticket becomes EARS criteria, the criteria become executable Gherkin and a failing Jest test, an agent (or you) drives red → green under a resource gate, and verification closes the loop. Every step has an offline fallback, so the demo cannot be killed by Wi-Fi.

§0 — Prerequisites (10 minutes, once)

Need	Check	Notes
Node.js ≥ 20	<code>node -v</code>	22 recommended
Claude Code	<code>claude --version</code>	<code>npm i -g @anthropic-ai/claude-code</code>

Need	Check	Notes
Model access	claude starts a session	Anthropic API key or Amazon Bedrock (CLAUDE_CODE_USE_BEDROCK=1 + AWS creds). If you keep a provider-toggle script, run it before the session and <i>never</i> switch mid-demo.
This repo	unzip sdd-starter.zip && cd sdd-starter	
Dependencies	npm install	installs jest, ts-jest, cucumber, ts-node
Sanity	npm test → RED on T3 · npm run bdd → scenario fails	red is the correct starting state
No-install check	npm run typecheck:noinstall	proves the skeleton compiles before any deps — nice opener line

Terminal setup for stage: font ≥ 18 pt, dark theme, two panes — left for Claude Code, right for npm test / npm run bdd. Rehearse the pane switch.

§1 — What’s in the repo for the demo

jira/PLT-4821.json the vague ticket (the villain)
 .claude/commands/
 specify-from-jira.md Jira → spec.md, gated on EARS quality
 plan.md · tasks.md · implement.md · verify.md
 .claude/hooks/resource_gate.sh the PreToolUse gate (exit 2 = blocked)
 .claude/state/ACTIVE_TASK which task an agent may touch
 specs/checkout-refunds/ the six artifacts (already worked, for reference)
 docs/ddd.md bounded-context map – the DDD talking slide
 features/steps/ BDD step definitions binding the .feature file
 solution/ RED stub + GREEN solution + demo:red / demo:green

Two demo modes: - **From scratch (impressive):** delete specs/checkout-refunds/spec.md body sections and rebuild them live via /specify-from-jira PLT-4821. - **From the worked example (safe):** keep the spec, *walk* it, and spend the live minutes on the red → green loop. Recommended for the webinar.

§2 — Configuring the Jira source (pick ONE)

A. Offline JSON — default, zero risk. Nothing to configure. `jira/PLT-4821.json` is a faithful issue export: vague description, untestable AC, two half-decisions hiding in comments. `/specify-from-jira` reads it first.

B. Live Jira via REST (Data Center or Cloud).

```
export JIRA_BASE_URL=https://jira.yourcompany.com
export JIRA_TOKEN=<personal-access-token>          # DC: PAT bearer
scripts/jira-fetch.sh PLT-4821                    # writes jira/PLT-4821.json
```

Jira Cloud instead: replace the Bearer header in the script with `-u you@company.com:$JIRA_API_TOKEN`. Fetch **before** the session and commit the JSON — never do live auth on stage.

C. Atlassian MCP (agent reads Jira itself). In `.mcp.json`:

```
{ "mcpServers": { "atlassian": {
  "type": "url", "url": "https://mcp.atlassian.com/v1/mcp" } } }
```

Authenticate once beforehand (`claude mcp list` to confirm). Then `/specify-from-jira PLT-4821` can pull the issue live — and you get a bonus talking point: *“note the token cost of that server — slide 23 in action.”* Remove the server after the demo; scope per repo, as preached.

§3 — The 12-minute demo script

Beat 1 • The villain (1 min). Open `jira/PLT-4821.json`. Read the AC out loud: *“refunds work for cancelled events.”* Read the comment: *“fees = our service fees... I think.”* Say: “every word of ambiguity here will be typed into code at machine speed. Let’s stop it now.”

Beat 2 • Jira → EARS, the /specify moment (3 min). In Claude Code:

```
/specify-from-jira PLT-4821
```

Watch it mine the description AND the comments; the “...I think” becomes an **open question**, not an assumption. Steer one criterion live — ask the room: *“‘automatic refunds’ — WHEN exactly?”* — until it reads: WHEN an event is cancelled, the system shall refund the ticket price minus service fees within 5 business days. Point at the gate: `/plan` refuses while Open

questions is non-empty. Answer the fees question on the spot (service fees, per the PO) and watch the list empty. **This conversation is the demo.**

Beat 3 · DDD in one breath (1 min). Open docs/ddd.md. One sentence: “Refunds is its own bounded context — the calculator lives in src/refunds/, downstream of Ordering’s frozen order snapshot — so when a refund test goes red, there is exactly one folder in the universe to look in.”

Beat 4 · BDD — the shared dialect (2 min).

```
npm run bdd
```

Open specs/checkout-refunds/contracts/refunds.feature side-by-side: the same EARS criterion as Given-When-Then that the PO can read — and it *executes*, and it *fails*, because nothing is implemented. One oracle, two dialects: the Gherkin and the Jest spec bind to the same in-memory harness.

Beat 5 · TDD + the gate (4 min).

```
npm test          # RED – refunds.service.spec.ts, by design
```

Set the task: `echo "checkout-refunds T3" > .claude/state/ACTIVE_TASK`. First, prove the gate: ask Claude to edit src/payments/infrastructure/psp.ts — the hook answers with exit 2: *outside T3’s declared resources*. Say: “a hook is enforcement; a prompt is persuasion.” Then implement: `/implement T3` and let the agent drive red → green against the contract (it’s ~4 lines: `total.cents – fees.cents`, banker’s rounding via `Money.roundBankers`, per ADR-017 — mention the two-hour Sprint-14 story).

```
npm test && npm run bdd    # GREEN – both dialects agree
```

Beat 6 · Verify (1 min). `/verify checkout-refunds` — the EARS-2 row flips to Green in verification.md; the exit gate opens. Close: “spec on the ticket this morning; verified behavior now. That’s the pipeline.”

§4 — Fallbacks & reset (do not skip)

- **Reset before every run:** `npm run demo:red && npm test` (must be RED), restore `ACTIVE_TASK`, `git checkout -- specs/` if you edited the spec live.
- **Agent slow or offline?** `npm run demo:green` swaps in `solution/refund-calculator.sol` — narrate the diff instead of typing it. The audience still sees red →

green.

- **Cucumber install trouble on the venue machine?** Skip Beat 4’s run; show the .feature file and lean on the Jest suite — the story holds.
- **Total tooling failure?** Slides 10-11 ARE the demo (criterion → contracts → failing test → task). Present them, promise the repo, move on.

§5 — Troubleshooting

Symptom	Fix
tsc deprecation error on moduleResolution	you’re on TS 6 globally — the repo pins TS 5.5 via npm install; use npm run typecheck
Hook doesn’t block	CLAUDE_PROJECT_DIR unset when testing manually — run via Claude Code, or export it
Cucumber “no steps”	run from repo root; cucumber.js resolves paths relative to it
Jest can’t find ts	npm install didn’t finish — rerun; typecheck:noinstall needs nothing
Agent edits refused everywhere	.claude/state/ACTIVE_TASK missing — that’s the gate working; set the task

§6 — What was verified where

Tested in a clean offline container: zero-dependency typecheck (stub **and** solution), the gate’s four modes (allow in-scope, block out-of-scope, allow specs/docs, block without ACTIVE_TASK), demo:red/demo:green round-trip, JSON/ YAML validity, cucumber config load. Requires npm install on your machine (not testable offline here): the jest run and the cucumber execution — both use pinned, current versions; run §0’s sanity checks the day before.

Spec first. Verify always — including this demo.